

# Apostila Definitiva

## Dataprev 2026

Analista de Tecnologia da Informação — Perfil 3  
Desenvolvimento de Software

---

Banca: **FGV** · Prova: **11/10/2026, 13h–17h** · Lotação: **Natal/RN**

70 questões · 115 pontos · Módulo I (Gerais) + Módulo II (Específicos, peso 2,5)

---

Conceitos + dicas de banca + o que já caiu nas provas reais,  
organizados a partir do roteiro, dos resumos e do banco de questões  
do repositório de estudo.

Compilado em 26 de julho de 2026

Uso pessoal de estudo — material derivado, não substitui o edital oficial.

# Sumário

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Como usar esta apostila</b>                                       | <b>1</b>  |
|          | Como a prova é montada (edital 2026) . . . . .                       | 1         |
|          | Onde a prova provavelmente se concentra . . . . .                    | 1         |
|          | IA entrou nas gerais (novidade 2026) . . . . .                       | 2         |
|          | Como usar este material no seu ciclo de estudo . . . . .             | 2         |
| <b>2</b> | <b>Técnica de prova FGV — como atacar qualquer questão</b>           | <b>4</b>  |
| 2.1      | Leia o comando antes de tudo . . . . .                               | 4         |
| 2.2      | Os padrões de distrator da FGV (o coração do método) . . . . .       | 4         |
| 2.3      | Estratégia de resolução . . . . .                                    | 5         |
| 2.4      | Gestão de tempo (4h, 70 questões) . . . . .                          | 5         |
| 2.5      | Regras de ouro . . . . .                                             | 5         |
| 2.6      | Depois de cada simulado . . . . .                                    | 5         |
| <b>I</b> | <b>Módulo II — Conhecimentos Específicos (peso 2,5, 30 questões)</b> | <b>6</b>  |
| <b>3</b> | <b>Engenharia de Software</b>                                        | <b>7</b>  |
| 3.1      | Ciclo de vida e modelos de processo . . . . .                        | 7         |
| 3.1.1    | RUP — as quatro fases e os dois eixos . . . . .                      | 8         |
| 3.2      | Maturidade de processo: CMMI e MPS.BR . . . . .                      | 9         |
| 3.3      | Engenharia de requisitos . . . . .                                   | 10        |
| 3.4      | Metodologias ágeis . . . . .                                         | 11        |
| 3.5      | Testes . . . . .                                                     | 12        |
| 3.6      | Manutenção de software . . . . .                                     | 13        |
| 3.7      | Mensuração: Ponto de Função × Story Points . . . . .                 | 14        |
| 3.8      | DevOps e entrega . . . . .                                           | 15        |
| 3.9      | Design de software . . . . .                                         | 15        |
| 3.10     | Alta probabilidade / pesquisa extra . . . . .                        | 16        |
| <b>4</b> | <b>Padrões de Projeto e UML</b>                                      | <b>17</b> |
| 4.1      | Padrões de Projeto (Design Patterns / GoF) . . . . .                 | 17        |
| 4.1.1    | Criacionais (como criar objetos) . . . . .                           | 18        |
| 4.1.2    | Estruturais (como compor objetos/classes) . . . . .                  | 19        |
| 4.1.3    | Comportamentais (como objetos interagem) . . . . .                   | 20        |
| 4.1.4    | GRASP . . . . .                                                      | 21        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 4.1.5    | Padrões arquiteturais (não são GoF, mas caem)                  | 21        |
| 4.1.6    | Reuso e anti-padrões                                           | 21        |
| 4.2      | UML                                                            | 22        |
| 4.2.1    | Os 14 diagramas em 2 famílias                                  | 23        |
| 4.2.2    | Diagrama de Classes (o mais cobrado)                           | 23        |
| 4.2.3    | Casos de Uso (requisitos funcionais)                           | 24        |
| 4.2.4    | Sequência (interação no tempo)                                 | 24        |
| 4.2.5    | Atividade e Estado                                             | 25        |
| <b>5</b> | <b>Arquitetura de Software</b>                                 | <b>27</b> |
| 5.1      | Servidor web × servidor de aplicação                           | 27        |
| 5.2      | Estilos de arquitetura                                         | 27        |
| 5.2.1    | Camadas lógicas (layers) × camadas físicas (tiers)             | 29        |
| 5.3      | Integração: REST × SOAP, Web Services                          | 29        |
| 5.3.1    | Mensageria (comunicação assíncrona)                            | 30        |
| 5.4      | Ambientes de rede corporativa                                  | 31        |
| 5.5      | Nuvem e escalabilidade                                         | 31        |
| 5.6      | Transações distribuídas                                        | 33        |
| 5.7      | Alta probabilidade / pesquisa extra                            | 34        |
| <b>6</b> | <b>Banco de Dados</b>                                          | <b>35</b> |
| 6.1      | Modelagem em três níveis                                       | 35        |
| 6.2      | Modelo relacional e integridade                                | 36        |
| 6.2.1    | Do MER para o relacional (como cada cardinalidade vira tabela) | 37        |
| 6.3      | Normalização (formas normais)                                  | 38        |
| 6.3.1    | A decomposição, passo a passo                                  | 38        |
| 6.4      | SQL: DDL × DML × DCL × TCL                                     | 39        |
| 6.4.1    | VIEW, GRANT e o controle da transação                          | 40        |
| 6.4.2    | Notações do MER e chave surrogada                              | 41        |
| 6.4.3    | Código no servidor: gatilho × procedimento armazenado          | 42        |
| 6.5      | Propriedades ACID (transações)                                 | 42        |
| 6.6      | Concorrência: níveis de isolamento                             | 43        |
| 6.7      | Relacional × Multidimensional (OLTP × OLAP)                    | 43        |
| 6.8      | NoSQL e novos armazenamentos                                   | 44        |
| 6.9      | ETL × ELT                                                      | 45        |
| 6.10     | Índices e notações                                             | 45        |
| 6.11     | Alta probabilidade / pesquisa extra                            | 47        |
| <b>7</b> | <b>Business Intelligence (BI)</b>                              | <b>48</b> |
| 7.1      | O que é BI                                                     | 48        |
| 7.2      | Data Warehouse (DW)                                            | 49        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 7.2.1    | Inmon × Kimball — por onde se começa . . . . .                | 49        |
| 7.3      | ETL — a ponte para o DW . . . . .                             | 49        |
| 7.4      | OLAP e modelagem dimensional . . . . .                        | 50        |
| 7.4.1    | Granularidade (o grão da fato) . . . . .                      | 52        |
| 7.4.2    | Tipos de fato e dimensão . . . . .                            | 52        |
| 7.4.3    | Dimensões lentamente mutantes (SCD) . . . . .                 | 53        |
| 7.5      | Sistemas de Suporte à Decisão (SSD/DSS) . . . . .             | 53        |
| 7.6      | Mapeamento de fontes de dados (boas práticas) . . . . .       | 53        |
| 7.7      | Data Mining e visualização . . . . .                          | 54        |
| 7.7.1    | CRISP-DM — o ciclo de um projeto de mineração . . . . .       | 54        |
| 7.8      | Alta probabilidade / pesquisa extra . . . . .                 | 55        |
| <b>8</b> | <b>Segurança da Informação</b>                                | <b>56</b> |
| 8.1      | Tríade CIA (a base de tudo) . . . . .                         | 56        |
| 8.2      | Criptografia . . . . .                                        | 57        |
| 8.3      | HTTPS, SSL e TLS . . . . .                                    | 58        |
| 8.4      | Controle de acesso . . . . .                                  | 59        |
| 8.5      | OWASP Top 10 — 2025 (vigente) e 2021 (leia os dois) . . . . . | 59        |
| 8.6      | Desenvolvimento seguro . . . . .                              | 60        |
| 8.7      | X.800 — a arquitetura de segurança OSI . . . . .              | 61        |
| 8.8      | Gestão de riscos: ameaça, vulnerabilidade e impacto . . . . . | 61        |
| 8.9      | Continuidade de negócio: RTO × RPO . . . . .                  | 62        |
| 8.10     | Deteção e resposta . . . . .                                  | 63        |
| 8.11     | Alta probabilidade / pesquisa extra . . . . .                 | 64        |
| <b>9</b> | <b>Programação</b>                                            | <b>65</b> |
| 9.1      | Ecossistema Spring (cai muito) . . . . .                      | 65        |
| 9.2      | Formatos de dados: XML, JSON, XSLT . . . . .                  | 66        |
| 9.3      | Mobile e low-code . . . . .                                   | 66        |
| 9.4      | Java EE / web server-side (JSF, Primefaces) . . . . .         | 67        |
| 9.5      | Versionamento com Git e DevOps . . . . .                      | 67        |
| 9.5.1    | CI, CD e CD — o trio que a FGV mais troca . . . . .           | 68        |
| 9.5.2    | Contêineres . . . . .                                         | 68        |
| 9.6      | Qualidade de código . . . . .                                 | 69        |
| 9.7      | Blockchain . . . . .                                          | 69        |
| 9.8      | Python aplicado a dados (cobrado no MPU) . . . . .            | 69        |
| 9.8.1    | PHP 8 — funções de sessão . . . . .                           | 70        |
| 9.9      | Estruturas de dados . . . . .                                 | 70        |
| 9.10     | Leitura ativa de código — técnica transversal . . . . .       | 71        |
| 9.11     | Alta probabilidade / pesquisa extra . . . . .                 | 73        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>10 Java</b>                                                             | <b>75</b> |
| 10.1 Exceções: checked × unchecked . . . . .                               | 75        |
| 10.2 Polimorfismo: overload × override . . . . .                           | 75        |
| 10.3 OO e SOLID . . . . .                                                  | 76        |
| 10.4 Interface × classe abstrata . . . . .                                 | 78        |
| 10.5 Coleções (Collections) . . . . .                                      | 78        |
| 10.5.1 String: imutabilidade e <b>StringBuilder</b> . . . . .              | 79        |
| 10.6 JVM, JPA e JavaEE . . . . .                                           | 79        |
| 10.6.1 Anotações que a FGV cobra pelo nome . . . . .                       | 80        |
| 10.7 Recursos modernos (Java 17/21 LTS) . . . . .                          | 81        |
| 10.7.1 <b>var</b> , <b>record</b> e <b>switch</b> como expressão . . . . . | 82        |
| 10.8 Leitura ativa de código em Java . . . . .                             | 83        |
| 10.8.1 Checked/unchecked disfarçadas em try-catch . . . . .                | 83        |
| 10.8.2 Escopo de variável — a pegadinha silenciosa . . . . .               | 83        |
| 10.8.3 Armadilhas de threads virtuais . . . . .                            | 84        |
| <b>11 Frontend</b>                                                         | <b>86</b> |
| 11.1 HTML e CSS . . . . .                                                  | 86        |
| 11.2 SPA × PWA (o coração deste bloco) . . . . .                           | 87        |
| 11.3 Frameworks . . . . .                                                  | 88        |
| 11.4 Ajax e comunicação . . . . .                                          | 89        |
| 11.5 UX, acessibilidade e usabilidade . . . . .                            | 89        |
| 11.6 Mobile multiplataforma × nativo . . . . .                             | 90        |
| 11.7 Leitura ativa de código — CSS e React . . . . .                       | 90        |
| 11.7.1 Pegadinhas visuais em CSS . . . . .                                 | 90        |
| 11.7.2 Escopo e closures em JavaScript . . . . .                           | 91        |
| 11.7.3 Renderização no React . . . . .                                     | 91        |
| 11.8 Alta probabilidade / pesquisa extra . . . . .                         | 92        |
| <b>12 Gestão e Governança de TI</b>                                        | <b>93</b> |
| 12.1 Gerenciamento de projetos (PMBOK) . . . . .                           | 93        |
| 12.2 Scrum, Kanban, Lean (ágil) . . . . .                                  | 94        |
| 12.3 ITIL 4 (gerenciamento de serviços) . . . . .                          | 95        |
| 12.4 COBIT 2019 (governança de TI) . . . . .                               | 96        |
| 12.5 BPMN (modelagem de processos) . . . . .                               | 97        |
| 12.6 Alta probabilidade / pesquisa extra . . . . .                         | 98        |
| <b>13 Redes de Computadores</b>                                            | <b>99</b> |
| 13.1 Modelos de referência . . . . .                                       | 99        |
| 13.2 Equipamentos . . . . .                                                | 100       |
| 13.3 TCP × UDP . . . . .                                                   | 100       |

|           |                                                                     |            |
|-----------|---------------------------------------------------------------------|------------|
| 13.4      | Protocolos e portas . . . . .                                       | 101        |
| 13.5      | Endereçamento e roteamento IP . . . . .                             | 101        |
| 13.5.1    | Sub-redes: máscara × hosts utilizáveis . . . . .                    | 101        |
| 13.5.2    | ACL Cisco e wildcard mask . . . . .                                 | 102        |
| 13.5.3    | MPLS, VLAN, NAT . . . . .                                           | 102        |
| 13.6      | Tipos de rede corporativa . . . . .                                 | 102        |
| 13.7      | Segurança de rede (interface com Segurança da Informação) . . . . . | 103        |
| 13.7.1    | Firewall stateful × stateless . . . . .                             | 103        |
| 13.7.2    | SSH × Telnet e o handshake do TLS . . . . .                         | 103        |
| <b>14</b> | <b>Órfãos — Administração de BD e Temas Coringa</b>                 | <b>105</b> |
| 14.1      | Administração de Dados × Administração de Banco de Dados . . . . .  | 105        |
| 14.2      | Recuperação e transações (ARIES) . . . . .                          | 105        |
| 14.3      | Armazenamento físico e desempenho . . . . .                         | 106        |
| 14.4      | Backup e recuperação . . . . .                                      | 106        |
| 14.5      | Segurança e acesso no SGBD . . . . .                                | 106        |
| 14.6      | Temas coringa genuínos . . . . .                                    | 106        |
| 14.7      | Onde cada tema coringa está detalhado . . . . .                     | 107        |
| 14.8      | IA/ML (tema quente neste bloco) . . . . .                           | 108        |
| <b>II</b> | <b>Módulo I — Conhecimentos Gerais (peso 1, 40 questões)</b>        | <b>110</b> |
| <b>15</b> | <b>Língua Portuguesa</b>                                            | <b>111</b> |
| 15.1      | Onde a prova se concentra . . . . .                                 | 111        |
| 15.2      | Pares e conceitos que a FGV cobra . . . . .                         | 111        |
| 15.3      | Concordância verbal — os verbos impessoais . . . . .                | 112        |
| 15.4      | Por que, porque, por quê, porquê . . . . .                          | 112        |
| 15.5      | Pontos de atenção / pesquisa extra . . . . .                        | 113        |
| <b>16</b> | <b>Língua Inglesa</b>                                               | <b>114</b> |
| 16.1      | O que a FGV cobra . . . . .                                         | 114        |
| 16.2      | Conectivos que caem muito . . . . .                                 | 114        |
| 16.3      | Modais e tempos (para o sentido) . . . . .                          | 115        |
| 16.4      | Pontos de atenção / pesquisa extra . . . . .                        | 116        |
| <b>17</b> | <b>Raciocínio Lógico Matemático</b>                                 | <b>117</b> |
| 17.1      | Ferramentas mínimas de matemática . . . . .                         | 118        |
| 17.2      | Lógica formal (não subestime) . . . . .                             | 118        |
| 17.3      | Alta probabilidade / pesquisa extra . . . . .                       | 120        |
| <b>18</b> | <b>Atualidades e Inteligência Artificial</b>                        | <b>121</b> |
| 18.1      | Parte 1 — Atualidades (viés socioambiental) . . . . .               | 121        |

|           |                                                                                   |            |
|-----------|-----------------------------------------------------------------------------------|------------|
| 18.2      | Parte 2 — Fundamentos de IA (novo, alto retorno)                                  | 122        |
| 18.2.1    | Conceitos                                                                         | 122        |
| 18.2.2    | Tipos de aprendizado                                                              | 123        |
| 18.2.3    | Ajuste do modelo e métricas de avaliação                                          | 123        |
| 18.2.4    | Modelos generativos e LLMs                                                        | 124        |
| 18.2.5    | Ética, governança e privacidade em IA                                             | 124        |
| 18.2.6    | IA aplicada a negócios e políticas públicas — como a FGV cobra                    | 124        |
| 18.2.7    | Interseção IA × ESG: energia, governança e regulação                              | 125        |
| 18.3      | Alta probabilidade / pesquisa extra                                               | 126        |
| <b>19</b> | <b>Legislação — Segurança da Informação e Proteção de Dados</b>                   | <b>127</b> |
| 19.1      | LGPD (Lei 13.709/2018)                                                            | 127        |
| 19.1.1    | Agentes de tratamento — o papel vem do poder de decisão                           | 128        |
| 19.1.2    | Bases legais — o art. 7º e o art. 11 são listas diferentes                        | 128        |
| 19.1.3    | Princípios (art. 6º) — o trio que a FGV confunde                                  | 129        |
| 19.1.4    | Decisão automatizada e explicabilidade (art. 20)                                  | 129        |
| 19.1.5    | Segurança e boas práticas (Capítulo VII) — a LGPD que o desenvolvedor implementa  | 130        |
| 19.1.6    | Sanções administrativas (art. 52) — é um regime, não uma tabela de multa          | 131        |
| 19.1.7    | ANPD e CNPD — dois colegiados, e um deles mudou de nome em 2026                   | 132        |
| 19.2      | Marco Civil (Lei 12.965/2014)                                                     | 132        |
| 19.2.1    | Guarda de registros                                                               | 132        |
| 19.2.2    | Sanções (art. 12)                                                                 | 133        |
| 19.2.3    | Art. 19 — responsabilidade dos provedores (atualização crítica pós-STF, jun/2025) | 133        |
| 19.3      | LAI (Lei 12.527/2011) — prazos de sigilo                                          | 135        |
| 19.4      | Delitos Informáticos (Lei 12.737/2012, art. 154-A do CP)                          | 135        |
| 19.5      | Alta probabilidade / pesquisa extra                                               | 136        |
| <b>A</b>  | <b>Mapa de pesos e prioridades</b>                                                | <b>137</b> |
| A.1       | Estrutura oficial da prova                                                        | 137        |
| A.2       | Distribuição real do Módulo II na Dataprev 2024 (30 questões)                     | 137        |
| A.3       | Comportamento da FGV em outras provas de TI                                       | 138        |
| A.4       | As cinco conclusões que moldam a priorização                                      | 138        |
| A.5       | Onde investir o tempo de revisão                                                  | 139        |
| A.6       | Checklist do dia da prova                                                         | 139        |
| <b>B</b>  | <b>Glossário de pares que a FGV inverte</b>                                       | <b>140</b> |

## Capítulo 1

# Como usar esta apostila

Este material foi montado a partir de quatro fontes do repositório de estudo: o conteúdo programático do edital (Anexo I, Perfil 3), as dicas de banca (**dicas/**), a resolução das questões do banco (**banco.json** + provas reais da FGV em **banco-provas.json**) e pesquisa em fontes primárias. A ideia é que ela funcione como par do quiz de terminal: leia o capítulo do bloco antes de resolver as questões daquele assunto, e volte a ele quando errar algo que não entendeu.

## Como a prova é montada (edital 2026)

Prova objetiva, **11/10/2026, 13h–17h**, 70 questões, 5 alternativas, uma correta. Portões fecham 12h30. Sem consulta. Questão com duas marcações ou nenhuma vale zero.

| Módulo                  | Disciplina                                               | Questões  | Peso       | Pontos     |
|-------------------------|----------------------------------------------------------|-----------|------------|------------|
| I — Gerais              | Língua Portuguesa                                        | 12        | 1          | 12         |
| I — Gerais              | Língua Inglesa                                           | 12        | 1          | 12         |
| I — Gerais              | Raciocínio Lógico Matemático                             | 5         | 1          | 5          |
| I — Gerais              | Atualidades e Inteligência Artificial                    | 6         | 1          | 6          |
| I — Gerais              | Legislação (Segurança da Informação e Proteção de Dados) | 5         | 1          | 5          |
| <b>II — Específicos</b> | <b>Conhecimentos Específicos</b>                         | <b>30</b> | <b>2,5</b> | <b>75</b>  |
| <b>Total</b>            |                                                          | <b>70</b> |            | <b>115</b> |

### A prova se decide no Módulo II

30 questões a 2,5 pontos cada = 75 dos 115 pontos (**65% da nota**). Errar um específico custa  $2,5 \times$  errar um geral. A eliminação exige no mínimo 57,5 pontos — isso é o piso legal, não a meta. Desenvolvimento de Software foi o perfil mais concorrido em 2024 (6.257 inscritos de 23.423), então a nota de corte real tende a ser bem mais alta que o piso.

## Onde a prova provavelmente se concentra

Estimativa a partir da Dataprev 2024 e da ênfase do edital 2026 — use como guia de prioridade, não como garantia.



| Bloco                   | Peso esperado | Por quê                                                                                    |
|-------------------------|---------------|--------------------------------------------------------------------------------------------|
| Engenharia de Software  | muito alto    | Maior bloco em 2024 (~10 questões). Requisitos, ágil, testes, ponto de função caem sempre. |
| Banco de Dados          | alto          | Eixo duplo com Eng. Software. No MPU superou tudo. SQL, normalização, NoSQL.               |
| Arquitetura de Software | alto          | Microserviços, REST×SOAP, nuvem, containers, hexagonal.                                    |
| Segurança               | alto          | ISO 27001/27002:2022, OWASP, OAuth2/SSO, SAST/DAST.                                        |
| Programação             | médio-alto    | Spring, XML/JSON/REST, DevOps, Git, mobile, low-code.                                      |
| BI                      | médio         | DW, OLAP, ETL, data mining — costuma vir junto com BD.                                     |
| Governança              | médio         | ITIL 4, COBIT 2019, Scrum/Kanban, BPMN.                                                    |
| Frontend                | médio         | SPA×PWA, React/Angular/Vue, HTTPS/TLS.                                                     |
| Java                    | médio         | Linguagem-base do edital, mas caiu pouco na amostra.                                       |

### ATENÇÃO — Dois alertas que valem pontos

**1. Redes cai mesmo fora do edital do Perfil 3.** Redes de Computadores está nos Perfis 2 e 5, não no 3. Ainda assim a Dataprev 2024 trouxe ~3 questões de redes (OSI, protocolo, segurança de rede) para Desenvolvimento de Software. Ignorar isso custou 7,5 pontos em 2024 — ver Capítulo de Redes.

**2. OWASP: 2021 vs. 2025.** A Dataprev 2024 cobrou o OWASP Top 10:2021. Mas em novembro de 2025 saiu o OWASP Top 10:2025, com mudanças grandes (Security Misconfiguration subiu para A02, “Software Supply Chain Failures” entrou como A03, SSRF foi absorvido em A01, surgiu A10 “Mishandling of Exceptional Conditions”). O edital 2026 aponta para a página do projeto sem fixar ano — saiba as duas listas.

## IA entrou nas gerais (novidade 2026)

A disciplina antes chamada “Atualidades” agora é “**Atualidades e Inteligência Artificial**” (6 questões) e o edital pede fundamentos de IA: conceitos, aprendizado de máquina, modelos generativos e de linguagem (LLMs), e ética, governança e privacidade em IA. Conteúdo novo e de retorno alto — ver o capítulo de Atualidades.

## Como usar este material no seu ciclo de estudo

1. **Leia o capítulo do bloco** antes de atacar as questões dele.
2. **Resolva as questões:** `./quiz.py <bloco>` (ou `-prova` para as provas reais da FGV).

3. **Veja a dica de banca dentro do capítulo** — os quadros vermelhos (“Como a FGV arma a pegadinha”) resumem o padrão de distrator.
4. **Revise o que errou:** os erros vão automaticamente para `erros/<bloco>.md`; use `./quiz.py -erradas` para repetição espaçada.

Ordem de prioridade sugerida: comece pelos blocos “muito alto”/“alto” da tabela acima — é onde os 2,5 pontos por questão se acumulam. O Apêndice A tem o mapa completo de pesos, e o Apêndice B consolida todos os pares que a FGV inverte, capítulo por capítulo, num glossário único de revisão rápida.

#### Regra de ouro para estudar com esta apostila

Não decore definição solta. Cada capítulo tem uma caixa azul de *conceito*, uma vermelha de *pegadinha* e uma verde de *como se sair melhor*. Se você souber reproduzir as três de cabeça para um bloco, está pronto para fazer as questões dele no quiz sem abrir a apostila.

## Capítulo 2

# Técnica de prova FGV — como atacar qualquer questão

Guia transversal, válido para todas as matérias, destilado da resolução de ~500 questões reais da FGV em 7 provas. Os capítulos por assunto dizem *o que* cada matéria cobra; este guia diz *como* atacar a questão na hora da prova. Leia-o primeiro e volte a ele na última semana.

## 2.1 Leia o comando antes de tudo

O comando é a última linha do enunciado. O erro número um em prova é responder o oposto do que foi pedido.

- “**Assinale a correta**” × “**assinale a INCORRETA / a que NÃO...**” — a FGV usa muito o comando negativo. Circule o “incorreta”/“não” no papel antes de ler as alternativas.
- “**Julgue as afirmativas I, II, III**” → resolva cada item isolado como V/F, depois case com a alternativa (“I e II apenas”, “I, II e III”, etc.). Uma afirmativa falsa derruba toda alternativa que a inclua — use isso para eliminar em bloco.
- **Sequência (V) (F) (F)**: mesma lógica. Acertar 1 ou 2 itens já elimina metade das alternativas.

## 2.2 Os padrões de distrator da FGV (o coração do método)

A FGV constrói a alternativa errada de poucas formas. Reconhecê-las é meio caminho andado.

### ATENÇÃO — Os sete padrões de distrator

1. **Absolutos.** “sempre”, “nunca”, “exclusivamente”, “apenas”, “todo”, “invariavelmente”, “impossível”, “garante”. No mundo técnico quase nada é absoluto — a alternativa com absoluto é quase sempre o distrator.
2. **Inversão de conceitos.** Troca dois conceitos do mesmo par: OLTP↔OLAP, PUT↔PATCH, agregação↔composição, checked↔unchecked, controlador↔operador, governança↔gestão, IDS↔IPS, SAST↔DAST, TCP↔UDP, incidente↔problema, Factory↔Abstract Factory. Se você domina o par lado a lado, a inversão salta aos olhos.
3. **A “quase certa”.** Acerta a primeira metade e erra a segunda (“X é um índice bitmap, adequado para *alta* cardinalidade” — bitmap é para *baixa*). Leia a alternativa inteira; a FGV esconde o erro no fim.
4. **Extrapolação (interpretação).** Em português/inglês, o distrator diz mais do que o texto diz, ou inverte a tese do autor. Se não está sustentado no texto, é distrator — não importa se é verdade no mundo real.
5. **Troca de número.** “5 domínios do COBIT”, “4 dimensões do ITIL”, “3 formas normais”, “camada 7 do OSI”. A FGV troca o número. Decore as quantidades.

6. **Troca de ordem.** Fases do ARIES (Analysis → Redo → Undo), ciclo de vida, camadas, formas normais. O distrator embaralha a sequência.
7. **Contradição interna.** “SOAP sem contrato formal” (SOAP usa WSDL), “microserviço com banco único compartilhado” (contra o princípio). A alternativa se contradiz — descarte.

## 2.3 Estratégia de resolução

1. **Elimine primeiro os absolutos e as contradições internas** — costumam sumir 2 alternativas de cara.
2. **Compare as 2 que sobraram.** A FGV quase sempre deixa uma “quase certa” como pegadinha final; a diferença está num detalhe (uma palavra, a segunda metade da frase, um número).
3. **Volte ao enunciado** e confirme a escolhida contra o que foi pedido (o cenário e o comando), não contra sua memória solta.
4. **Interpretação:** volte ao trecho exato. Nunca responda “pelo que faz sentido”; responda pelo que o texto afirma.

## 2.4 Gestão de tempo (4h, 70 questões)

### Ritmo de prova

- **Específicos primeiro.** Valem  $2,5 \times$  (75 dos 115 pontos). Comece por eles com a cabeça fresca; deixe português/inglês/RLM para depois.
- **Não trave.** Marque a questão difícil, siga, volte no fim. Uma questão de RLM não vale mais que uma de eng. de software — e costuma custar mais tempo.
- **Ritmo de referência:**  $\sim 3$  min por questão em média. Treine com `./quiz.py -simulado` (cronometrado, sem feedback até o fim).

## 2.5 Regras de ouro

- **Filtro travado em FGV.** Cada banca tem uma gramática de pegadinha própria; reflexo de CESPE ou de outra banca atrapalha aqui.
- **Cartão de resposta:** questão com duas marcações ou nenhuma = zero. Confira. Reserve tempo para preencher com calma.
- **Não mude resposta no chute.** Só troque se tiver um motivo concreto (releu e viu o erro), não por insegurança de última hora.
- **Anulada acontece.** Se a questão parecer ter duas certas ou nenhuma, marque a “menos errada” e siga — não perca tempo brigando com ela.

## 2.6 Depois de cada simulado

Toda questão errada vira material de revisão: `./quiz.py -erradas` (repetição espaçada) e a anotação automática em `erros/<bloco>.md`. O caderno de erros é o material principal de revisão nas duas últimas semanas antes da prova.

## Parte I

### Módulo II — Conhecimentos Específicos (peso 2,5, 30 questões)

## Capítulo 3

# Engenharia de Software

### Ficha do edital

Engenharia de requisitos (classificação, processo, técnicas de elicitação); testes (unitários, integração, ágeis, usabilidade, automatizados, TDD, ciclo de vida, RPA); metodologias ágeis (Scrum, Kanban, XP); padrões de desenvolvimento e reuso; codificação; Ponto de Função e Story Points; DevOps; design de software.

**Peso esperado: MUITO ALTO** — foi o maior bloco da Dataprev 2024 (~10 questões). É metade do “eixo duplo” da FGV em TI (a outra metade é Banco de Dados).

### 3.1 Ciclo de vida e modelos de processo

#### Os modelos clássicos

- **Cascata (waterfall):** fases sequenciais estritas, sem sobreposição, requisitos congelados no início. Adequado quando os requisitos são **estáveis e bem compreendidos**. Rígido a mudanças.
- **Incremental:** entrega em incrementos que somam funcionalidade ao longo do tempo.
- **Iterativo:** refina o mesmo produto em ciclos sucessivos.
- **Espiral (Boehm):** iterativo + **análise de risco** a cada volta da espiral.
- **Prototipação:** constrói um protótipo (às vezes descartável) para validar requisitos com o cliente.
- **Ágil:** iterativo-incremental, com entregas curtas e adaptação contínua a mudanças.

#### ATENÇÃO — Como a FGV arma a pegadinha

O enunciado descreve “fases sequenciais, requisitos congelados” → é **cascata**, nunca incremental nem ágil. Esse é o distrator clássico de abertura do bloco.

#### Modelo V — cada fase tem o seu teste

Variação do cascata que **espelha** construção e verificação: o ramo descendente vai do requisito ao código, o ascendente sobe testando, e cada nível de teste confere o que foi decidido no nível equivalente do outro lado.

| Fase (ramo descendente)                     | Nível de teste que a verifica |
|---------------------------------------------|-------------------------------|
| Requisitos do usuário                       | <b>Teste de aceitação</b>     |
| Especificação do sistema                    | Teste de sistema              |
| Projeto arquitetural (módulos e interfaces) | <b>Teste de integração</b>    |
| Projeto detalhado (algoritmos)              | Teste de unidade              |
| Codificação                                 | (vértice do V)                |

A lógica é “quem definiu, confere”: o que foi **acordado com o usuário** é conferido na **aceitação**; o que foi decidido no **projeto arquitetural** — a divisão em módulos e suas interfaces — é conferido na **integração**.

### ATENÇÃO — Como a FGV arma a pegadinha

A FGV embaralha os pares do V. Requisito nunca casa com teste de unidade (unidade verifica o **projeto detalhado**), e codificação nunca casa com aceitação. Se estiver em dúvida, use as pontas: a de cima é sempre requisito ↔ aceitação; a de baixo, código ↔ unidade.

### 3.1.1 RUP — as quatro fases e os dois eixos

O RUP (*Rational Unified Process*) é iterativo-incremental e dirigido por casos de uso, mas o que a banca cobra dele é a **estrutura em dois eixos** — e é justamente aí que ela arma o item.

#### Fase × disciplina: o eixo do tempo e o eixo do conteúdo

**Fase** é *quando*: o projeto passa por Concepção, Elaboração, Construção e Transição, nessa ordem, uma vez só, e cada fase termina num marco. **Disciplina** (ou *workflow*) é *o quê*: Requisitos, Análise e Projeto, Implementação, Teste, Implantação e as de apoio (Gerência de Projeto, Configuração e Mudança, Ambiente).

O ponto que importa: **as disciplinas atravessam todas as fases**. Não existe “a fase de testes” no RUP — testa-se em todas as fases, só que em intensidade diferente. Se você desenhar o gráfico clássico do RUP, as fases são as colunas e as disciplinas são as linhas: cada linha tem uma corcova onde aquele trabalho concentra, mas nenhuma linha começa e termina dentro de uma coluna só.

| Fase       | Pergunta que ela responde                                                                                               | Marco                          |
|------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| Concepção  | vale a pena fazer? escopo, viabilidade, caso de negócio                                                                 | Objetivos do ciclo de vida     |
| Elaboração | <b>como fazer sem que dê errado?</b> — é aqui que se atacam os <b>riscos</b> e se firma a <b>arquitetura executável</b> | Arquitetura do ciclo de vida   |
| Construção | construir o grosso do produto, iteração a iteração                                                                      | Capacidade operacional inicial |
| Transição  | entregar ao usuário: implantação, treinamento, ajuste                                                                   | Liberação do produto           |

**A âncora:** risco e arquitetura vivem na **Elaboração**. A lógica é econômica — resolver o risco

arquitetural antes de gastar o orçamento da Construção. Quem deixa a arquitetura para a Construção descobre o problema quando já é caro demais mudar.

### ATENÇÃO — Como a FGV arma a pegadinha

Duas trocas prontas. (i) Tratar disciplina como fase — “a fase de testes do RUP” não existe; teste é disciplina, e acontece nas quatro fases. (ii) Deslocar o risco: dizer que os riscos principais são tratados na **Concepção** (lá se avalia viabilidade) ou na **Construção** (lá já é tarde). É a **Elaboração**. Cuidado também com “o RUP é cascata” — ele é iterativo *dentro* de cada fase; o que é sequencial é a ordem das quatro fases, não o trabalho.

## 3.2 Maturidade de processo: CMMI e MPS.BR

Modelos que avaliam **quão maduro é o processo** da organização — não a qualidade de um produto específico.

### O que é maturidade de processo, e que problema o CMMI resolve

O CMMI nasceu de uma pergunta prática do Departamento de Defesa americano nos anos 1980: *como saber, antes de contratar, se um fornecedor vai entregar?* Olhar o produto anterior não bastava — um projeto que deu certo pode ter dado certo por acaso, ou porque um herói virou noites. O que se queria medir era a **capacidade de repetir** o resultado.

Daí a definição: **maturidade é o quanto o resultado depende do processo e não das pessoas**. Uma organização madura entrega parecido com equipes diferentes, porque o jeito de trabalhar está descrito, é seguido e é medido. Uma imatura entrega bem quando calha de ter a pessoa certa.

**Como se reconhece cada nível na prática** — e o corte que a banca mais cobra é o do 2 para o 3:

|         |                                                                                                                                                                                                                                      |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nível 2 | Cada projeto se organiza <b>do seu jeito</b> . O projeto A tem cronograma e controle de mudanças; o projeto B, ao lado, faz diferente — e os dois podem ser bem gerenciados. O que existe é disciplina <i>por projeto</i> .          |
| Nível 3 | Existe um <b>processo-padrão da organização</b> , e cada projeto o <i>adapta (tailoring)</i> dentro de regras. Um desenvolvedor que muda de projeto reconhece o jeito de trabalhar. O aprendizado de um projeto vira ativo de todos. |
| Nível 4 | O processo passa a ser <b>medido estatisticamente</b> : não se diz “costuma dar certo”, diz-se “a densidade de defeito fica entre tais limites”. Previsibilidade quantitativa.                                                       |
| Nível 5 | A medição do nível 4 é usada para <b>mudar o processo</b> de propósito e verificar se melhorou. Melhoria contínua baseada em dado, não em opinião.                                                                                   |

A frase-âncora: **o 2 é “gerenciado por projeto”; o 3 é “padronizado na organização”**. Todo cenário de prova que descreve “cada equipe tem seu próprio jeito, embora todas planejem e monitorem” é nível 2.

**E por que existem duas representações?** Porque as duas perguntas são legítimas. *Por estágios* responde “que nota tem esta **organização?**” — é um número só, comparável entre



fornecedores, e foi para isso que o modelo nasceu. *Contínua* responde “quão capaz somos **nesta área específica?**” — permite investir em Engenharia de Requisitos sem ter de subir tudo junto, o que serve a quem quer melhorar e não a quem quer se certificar. Daí o vocabulário diferente: estágios dá **maturidade** à organização; contínua dá **capacidade** a cada área de processo.

**CMMI — representação por estágios** (a que a banca cobra): cinco níveis de maturidade da organização inteira.

| Nº | Nível                        | O que caracteriza                                                              |
|----|------------------------------|--------------------------------------------------------------------------------|
| 1  | Inicial                      | processo imprevisível, reativo, dependente de heróis                           |
| 2  | Gerenciado                   | gerenciado <b>por projeto</b> (planejado, monitorado)                          |
| 3  | <b>Definido</b>              | processo <b>padronizado no âmbito da organização</b> , e não projeto a projeto |
| 4  | Gerenciado Quantitativamente | processo <b>medido e controlado por estatística</b>                            |
| 5  | Em Otimização                | melhoria contínua a partir da medição                                          |

**As duas representações:** *por estágios* dá um número de **maturidade** à organização (os cinco níveis acima); *contínua* dá um nível de **capacidade** a cada área de processo isolada, permitindo evoluir umas antes das outras.

**MPS.BR** (modelo brasileiro, MR-MPS-SW): mesma ideia, escala própria com **sete níveis identificados por letras, de G até A** — G (Parcialmente Gerenciado), F (Gerenciado), E (Parcialmente Definido), D (Largamente Definido), C (Definido), B (Gerenciado Quantitativamente), A (Em Otimização). Evolui-se **de G para A**: o G é o ponto de partida e o A é o topo.

**ATENÇÃO — Como a FGV arma a pegadinha**

Três trocas prontas: (i) dizer que o **nível 3 é o Gerenciado** — ele é o **Definido**; (ii) inverter a ordem do topo, pondo o Gerenciado depois do Definido — acima do 3 vem **Gerenciado Quantitativamente** e só então Em Otimização; (iii) afirmar que o MPS.BR **começa no A**. É o contrário: começa no G. Cuidado também com “a escala tem quatro níveis” — são **cinco** no CMMI e **sete** no MPS.BR.

### 3.3 Engenharia de requisitos

O par que mais cai no bloco inteiro (Dataprev, MPU e TJ-RJ todos cobraram):

| Tipo                       | O que descreve                                     | Exemplo                                                   |
|----------------------------|----------------------------------------------------|-----------------------------------------------------------|
| <b>Funcional (RF)</b>      | O QUE o sistema faz (uma função, um comportamento) | “emitir saldo da conta”                                   |
| <b>Não funcional (RNF)</b> | COMO / QUÃO BEM (qualidade, restrição)             | “saldo em tempo real”, desempenho, segurança, usabilidade |

**Regra prática:** “em tempo real”, “seguro”, “rápido”, “disponível” → sempre **não funcional**.

**Processo:** elicitação → análise → especificação → validação → gerenciamento. A **elicitação** (levantamento) usa: entrevista, questionário, **brainstorming**, workshop, observação, prototipação, análise de documentos.

#### ATENÇÃO — Como a FGV arma a pegadinha

A FGV afirma que “brainstorming é técnica inadequada, use só entrevista formal” — isso é **falso**; brainstorming é técnica legítima de elicitação. Outro distrator comum: dizer que o requisito surge *depois* da implementação (a ordem certa é elicitar antes de codificar).

#### O que já caiu nas provas

**Em prova real da FGV:** pedir para **contar** quantos RF e quantos RNF há num enunciado longo é item recorrente — caiu no **MPU** (rol de funcionalidades de um sistema de processo seletivo), no **TJ-RJ** (entrevista de elicitação sobre relatórios dinâmicos) e na **ALERO 2026** (LGPD e acessibilidade entrando como RNF). A banca também pede o *tipo* do RNF (usabilidade, desempenho, produto, organizacional, externo). Ao ler, marque cada verbo de ação (RF) e cada “deve ser rápido/seguro/exportável” (RNF).

## 3.4 Metodologias ágeis

### Scrum — é framework, não metodologia

Trabalho organizado em **Sprints** de tamanho fixo (time-boxed).

**Papéis/accountabilities:** **Product Owner** (valor e Product Backlog), **Scrum Master** (remove impedimentos, serve ao time, *não manda*), **Developers**.

**Eventos:** Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective.

**Artefatos:** Product Backlog, Sprint Backlog, Increment.

**Compromissos (Scrum Guide 2020):** cada artefato tem um **compromisso** associado, que lhe dá foco e permite medir progresso. São três pares fixos:

| Artefato        | Compromisso                           | O que ele fixa                    |
|-----------------|---------------------------------------|-----------------------------------|
| Product Backlog | <b>Meta do Produto</b> (Product Goal) | o objetivo de longo prazo         |
| Sprint Backlog  | <b>Meta da Sprint</b> (Sprint Goal)   | o objetivo único da Sprint        |
| Increment       | <b>Definition of Done</b>             | o padrão de qualidade do “pronto” |

- **Sprint Review** = **produto** (inspeciona o incremento com stakeholders).
- **Sprint Retrospective** = **processo** (o que melhorar no jeito de trabalhar do time).
- No Daily, com risco à Sprint, o Scrum Master **facilita a solução do time** — não redistribui tarefas sozinho, nem assume a tarefa do desenvolvedor.

| Framework                | Ideia central                                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------------------------|
| Kanban                   | Fluxo contínuo, <b>sem sprints fechadas</b> ; limita WIP (trabalho em progresso); entrega conforme conclui. |
| XP (Extreme Programming) | Práticas de engenharia: programação em par, TDD, integração contínua, refatoração, cliente presente.        |
| Lean                     | Eliminar desperdício, otimizar o fluxo de valor.                                                            |
| Ágil híbrido             | Combina práticas ágeis com métodos tradicionais.                                                            |

### ATENÇÃO — Como a FGV arma a pegadinha

Kanban **não** tem sprint — o distrator clássico é “Kanban organiza o trabalho em sprints”. Scrum **tem** time-box; Kanban é fluxo contínuo. Nos **compromissos**, a banca mantém os nomes certos e **embaralha as ligações**: Definition of Done é do **Incremento** (nunca do Product Backlog nem do Sprint Backlog), e a Meta do Produto é do **Product Backlog** (nunca do Incremento). Guie-se pelo horizonte: produto (longo prazo) → Product Backlog; Sprint → Sprint Backlog; “pronto” → Incremento.

### O que já caiu nas provas

**Em prova real da FGV:** Sprint Planning prioriza pela **capacidade real da equipe**, nunca por pressão externa — gabarito literal da **Dataprev 2024**. Mudança tardia de escopo: trata-se com o PO, ajustando o backlog, nunca “recusar por comprometer o plano” nem “suspender o projeto” — **TJ-RJ**, no cenário do projeto com 90% do escopo já definido.

## 3.5 Testes

| Nível/tipo     | O que verifica                                          |
|----------------|---------------------------------------------------------|
| Unitário       | menor unidade isolada (função/método)                   |
| Integração     | interação entre módulos/componentes                     |
| Sistema        | o sistema completo                                      |
| Aceitação      | atende ao cliente/requisito                             |
| Usabilidade    | experiência/interface intuitiva                         |
| Regressão      | mudança não quebrou o que já funcionava                 |
| Fumaça (smoke) | verificação rápida e superficial das funções principais |

### Testes de desempenho — carga, estresse, volume

Os três se confundem porque todos “sobrecarregam” o sistema. O que muda é **até onde** se vai:

- **Carga (load):** roda o volume de uso **esperado em produção** e observa se o sistema aguenta o previsto.
- **Estresse (stress):** vai **além** do previsto, aumentando até achar o **ponto de ruptura** — e como o sistema se comporta ao quebrar (e ao se recuperar).
- **Volume:** muita **massa de dados** armazenada, não muitos usuários simultâneos.

**ATENÇÃO — Como a FGV arma a pegadinha**

“Ultrapassar o previsto até o sistema deixar de responder” é **estresse**; “comportar-se bem no volume contratado” é **carga**. A FGV oferece carga como quase-certa em cenário de estresse — o gatilho é a palavra **além/ultrapassa**. Volume é massa de **dados**, não de usuários.

**Cobertura de caixa-branca**

CrITÉRIOS que medem **quanto do código** os testes exercitam:

- **Comandos** (instruções/*statement*): cada linha executada ao menos uma vez.
- **Decisões** (ramos/*branch*): cada decisão avaliada **como verdadeira e como falsa**.
- **Caminhos**: cada combinação de caminhos pelo código — o mais forte e, em geral, inviável de atingir por completo.

Força crescente: comandos < decisões < caminhos. **100% de decisões implica 100% de comandos; a recíproca é falsa.**

**ATENÇÃO — Como a FGV arma a pegadinha**

O caso-teste da banca é o **if sem else**. Com `if (a > 10) { x = 1; }` e um único caso `a = 20`, todas as linhas rodam — **comandos em 100%** — mas a condição nunca foi avaliada como falsa, então **decisões fica em 50%**, ainda que não exista bloco `else` escrito. Nunca aceite que os dois critérios são equivalentes.

## 3.6 Manutenção de software

Classificação clássica (ISO/IEC 14764) do que se faz com o software **depois de entregue**. O critério é a **causa** da alteração, não o momento nem o tamanho:

| Tipo       | Causa da alteração                                                                                  |
|------------|-----------------------------------------------------------------------------------------------------|
| Corretiva  | <b>corrigir defeito</b> já detectado (erro relatado, falha em produção)                             |
| Adaptativa | acompanhar <b>mudança do ambiente</b> : sistema operacional, SGBD, plataforma, hardware, legislação |
| Perfectiva | <b>melhorar</b> o que já funciona: desempenho, manutenibilidade, requisito novo pedido pelo usuário |
| Preventiva | corrigir <b>falha latente</b> antes que ela se manifeste (reduzir risco futuro)                     |

**ATENÇÃO — Como a FGV arma a pegadinha**

Atualização obrigatória do banco de dados, sem defeito relatado e sem funcionalidade nova, é **adaptativa** — a causa é o **ambiente**. Não é corretiva (não há defeito a corrigir), não é perfectiva (não se melhora nada) e não é preventiva (não se antecipa falha latente alguma). Cuidado também com “evolutiva”, rótulo de fora dessa classificação que a banca oferece como se fosse um dos quatro tipos.

**TDD e BDD**

**TDD (Test-Driven Development):** escreve o **teste antes** do código (ciclo red → green → refactor). Foco em design e cobertura.

**BDD (Behavior-Driven Development):** evolução do TDD, especifica comportamento em linguagem acessível ao negócio (Gherkin: Dado-Quando-Então).

**Caixa-preta** (sem ver o código, só entrada/saída) × **caixa-branca** (baseado na estrutura interna/código).

**RPA (Robotic Process Automation):** automatiza tarefas repetitivas de interface (robôs de software) — **não é teste**.

**3.7 Mensuração: Ponto de Função × Story Points****O que o Ponto de Função mede — e por que independe da linguagem**

A APF resolve um problema de contrato: *como pagar por software sem pagar por linha de código?* Medir por linhas premia quem escreve mal e pune quem usa uma linguagem expressiva — a mesma funcionalidade sai em 500 linhas de Java ou 80 de Python, e não faz sentido a segunda valer menos.

A saída foi medir do **lado de fora**: conta-se o que o sistema **faz para o usuário**, não como foi construído. Por isso a contagem se faz a partir de **telas, relatórios e arquivos lógicos** — artefatos visíveis — e o resultado é o mesmo qualquer que seja a linguagem, a equipe ou a ferramenta. É exatamente essa independência que a FGV cobra.

O que se conta são cinco tipos, em dois grupos:

|                             |                                                                                                                                                    |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Funções de transação</b> | o que <i>atravessa</i> a fronteira do sistema: <b>EE</b> (entrada externa), <b>SE</b> (saída externa), <b>CE</b> (consulta externa)                |
| <b>Funções de dados</b>     | o que o sistema <i>guarda ou lê</i> : <b>ALI</b> (arquivo lógico interno, mantido aqui dentro), <b>AIE</b> (arquivo de interface externa, só lido) |

Os dois critérios que resolvem quase toda questão de classificação: **ALI × AIE se decide pela manutenção**, não pela leitura — se o sistema *mantém* o dado, é ALI; se só consulta o que outro mantém, é AIE. E **SE × CE se decide pelo dado derivado** — se há cálculo, totalização ou lógica que gera informação nova, é Saída Externa; se é recuperação pura, é Consulta Externa.

|                    | <b>Ponto de Função (APF)</b>                          | <b>Story Points</b>                 |
|--------------------|-------------------------------------------------------|-------------------------------------|
| Natureza           | <b>objetiva</b> , padronizada (IF-PUG)                | <b>relativa/subjetiva</b> , do time |
| Independe do time? | sim                                                   | não (calibrado por time)            |
| Bom para           | contrato de escopo fechado, comparação entre projetos | estimativa ágil interna             |
| Baseado em         | funções do sistema (EE, SE, CE, ALI, AIE)             | esforço/complexidade percebidos     |

**ATENÇÃO — Como a FGV arma a pegadinha**

A FGV troca os dois: diz que Story Point é objetivo/padronizável entre times, ou que serve a contrato de escopo fechado — é o contrário. Ponto de Função é o objetivo e comparável; Story Points é o relativo do time.

**Classificando telas em APF**

- Tela que **ENTRA** e grava dado → **EE** (Entrada Externa).
- Tela que só **mostra dado processado/calculado** → **SE** (Saída Externa).
- Tela que só **consulta e exibe sem cálculo** → **CE** (Consulta Externa).
- Grupo lógico de dados **mantido dentro** do sistema → **ALI** (Arquivo Lógico Interno).
- Referência externa só **lida** (não mantida) → **AIE** (Arquivo de Interface Externa).

### 3.8 DevOps e entrega

- **CI (Integração Contínua)**: integra e testa o código com frequência (a cada commit).
- **CD (Entrega/Implantação Contínua)**: leva a nova versão à produção rapidamente, com mínima interrupção. “Fornecer rapidamente nova versão em produção” = **CD**, não **CI**.
- **DevOps** une desenvolvimento e operações; **DevSecOps** injeta segurança no pipeline.

### 3.9 Design de software

- **Design de alto nível** = estrutura geral, módulos e sua interação (próximo da arquitetura). **Design de baixo nível** = detalhe de funções, métodos, algoritmos.
- **Arquitetura** = decisões amplas e estruturais; **design** = decisões detalhadas. Não é verdade que “arquitetura só serve para projeto grande”.
- Padrões de projeto e UML têm capítulo próprio (Capítulo 4).

**O que já caiu nas provas**

**Em prova real da FGV: 39 questões — e é o bloco específico mais bem distribuído do corpus**, o único que aparece forte em *todas* as provas (13 na ALERO, 9 na Dataprev, 10 no MPU, 7 no TJ-RJ). Outros blocos têm total maior por causa de uma prova de perfil diferente; este não depende de nenhuma. É o bloco mais confiável para investir tempo. Requisito funcional × não funcional no cenário do “saldo em tempo real”, com **brainstorming** como elicitação válida; papel do **Scrum Master no Daily**; **Sprint Planning** pela capacidade da equipe; metodologia ágil para mudança frequente (Scrum, com Kanban/XP/Lean/Cascata como distratores); **CD** no DevOps; **Ponto de Função** × **Story Points**; **design alto** × **baixo nível** (a banca inverte os dois numa alternativa e na seguinte); níveis de teste + **TDD** em item I/II/III/IV; **ágil híbrido**; **Liskov** em leitura de código — **Dataprev 2024**. Contagem de RF/RNF, **BPMN** (Data Object × Data Association e leitura de diagrama), **CBOK 4.0** (*handoffs* entre departamentos), **APF** sobre telas, **SNAP** (medição não funcional), testes automatizados, **Liskov** pela definição formal e **UML 2.5.1** — MPU. Mudança tardia de escopo, **DDD** com especialistas de domínio, **APF**, contagem de RF/RNF e o **MoReq-Jus** (Resolução CNJ nº 522/2023) — TJ-RJ. **CMMI** e **MPS.BR** no mesmo enunciado, **Ponto de Função** como métrica independente de linguagem, **modelo cascata** (validação só na fase final), **RUP** (as quatro fases, risco na Elaboração), **prototipação** para requisito volátil, **verificação** ×

**validação** e negociação de requisitos conflitantes — **ALERO 2026**. Versionamento (SVN × Git, GitLab CI) vem tagueado aqui, mas o conteúdo está no Capítulo 9.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): Sprint Review × Retrospective; os três compromissos do Scrum (Meta do Produto, Meta da Sprint, Definition of Done); modelo V (requisitos ↔ aceitação, arquitetural ↔ integração); tipos de manutenção (adaptativa na troca de SGBD); cobertura de comandos × decisões; teste de estresse × carga; cascata × incremental como par explícito. **APF em detalhe** — ALI × AIE pelo critério da manutenção (e não da leitura), e EE/CE/SE pela intenção do processo, com o dado derivado separando Consulta de Saída Externa; APF × LOC × *story point* como métrica de contrato. **Partição de equivalência** × **análise de valor limite**. **Estratégias de implantação**: *blue-green* × *canary* × *rolling update* × *feature toggle*. **RUP**: fase (eixo do tempo, iterativa) × disciplina (eixo do conteúdo, atravessa as fases) — o par que sustenta o item de comando negativo.

Rode `./quiz.py eng-software`.

#### ATENÇÃO — Resumo dos pares que a FGV inverte neste bloco

Review↔Retro, funcional↔não funcional, PF↔Story Points, cascata↔incremental, TDD↔BDD, alto↔baixo nível. Absolutos (“sempre”, “exclusivamente”, “apenas”, “impossível”) quase sempre marcam o distrator. E cuidado com distorcer o papel do Scrum Master: ele facilita e serve, nunca comanda nem executa a tarefa do desenvolvedor.

### 3.10 Alta probabilidade / pesquisa extra

- **Scrum Guide 2020**: usa “accountabilities” (não “papéis”); removeu a figura do “time de desenvolvimento” como subgrupo — hoje é um **Scrum Team** único com Developers. Sprint Goal, Product Goal, Definition of Done.
- **Clean Code** / **SonarQube** (no edital): análise **estática** de código (sem executar) para dívida técnica, code smells, cobertura.
- **MVP**, **design thinking**, **UX** aparecem no edital do Perfil 1 mas permeiam o Perfil 3; MVP = menor versão que entrega valor e valida uma hipótese.



## Capítulo 4

# Padrões de Projeto e UML

### Ficha do edital

“Padrões de projeto”, “design de software”, “padrões de desenvolvimento e reuso”, “análise e projeto orientados a objetos”. UML não está listada com todas as letras no edital do Perfil 3 (aparece explícita no Perfil 2), mas é vizinha direta desses itens e a FGV cobra com frequência em provas de TI.

**Peso esperado: ALTA PROBABILIDADE** — a FGV dá um cenário (“preciso garantir uma única instância...”, “preciso criar objetos sem acoplar à classe concreta...”) e pergunta qual padrão resolve. Decore o gatilho de cada um, não só o nome.

### 4.1 Padrões de Projeto (Design Patterns / GoF)

O enunciado descreve um **problema de design** e você identifica o padrão que o resolve. A chave é reconhecer a **intenção** (o “para quê”) de cada padrão. O catálogo GoF (*Gang of Four*) reúne **23 padrões**, divididos em três grupos: **criacionais** (5), **estruturais** (7) e **comportamentais** (11). O número e a classificação em três grupos são cobrados diretamente.

Antes do catálogo, o essencial: um padrão de projeto **não é código pronto, nem biblioteca, nem framework**. É a descrição de uma solução que já foi reinventada tantas vezes que virou vocabulário — e, como todo verbete, tem quatro partes: um **nome**, o **problema** em que ele se aplica, a **solução** em termos de classes e objetos, e as **consequências** (porque toda escolha tem preço). Quem lê só a solução decora estrutura; quem lê o problema reconhece o gatilho no enunciado.

Isso também explica o anti-padrão inverso: aplicar padrão *sem* ter o problema é *over-engineering* — uma fábrica abstrata para criar um único tipo de objeto acrescenta duas classes e nenhuma flexibilidade.

### O que os 23 padrões têm em comum — e o critério das três famílias

Quase todo padrão do catálogo faz a *mesma* coisa por dentro: **isola o que varia atrás de uma interface estável** e prefere **composição a herança** para poder trocar essa parte em tempo de execução. Guarde essa frase: ela é o fio que liga Strategy, State, Bridge, Decorator, Abstract Factory e quase todos os outros.

O que muda de família para família é **o que** está sendo isolado:



| Família              | Isola...                                             | Pergunta que ela responde                                                     |
|----------------------|------------------------------------------------------|-------------------------------------------------------------------------------|
| Criacionais (5)      | <i>qual</i> objeto se instancia, e quando            | “como criar isto sem amarrar o código à classe concreta?”                     |
| Estruturais (7)      | <i>como</i> as peças se encaixam                     | “como compor/embrulhar objetos para obter uma interface ou um comportamento?” |
| Comportamentais (11) | <i>quem</i> conversa com quem, e qual algoritmo roda | “como distribuir a responsabilidade da interação em runtime?”                 |

Na dúvida entre duas alternativas, classifique primeiro pela família: se o enunciado fala em **criar** algo, os sete estruturais e os onze comportamentais caem fora de uma vez.

#### 4.1.1 Criacionais (como criar objetos)

| Padrão           | Intenção (gatilho no enunciado)                                                                                                             |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Singleton        | garantir <b>uma única instância</b> e ponto global de acesso (“apenas uma conexão/configuração no sistema todo”)                            |
| Factory Method   | delegar a criação a subclasses; criar objeto <b>sem acoplar à classe concreta</b> (“decidir em runtime qual objeto criar”)                  |
| Abstract Factory | criar <b>famílias</b> de objetos relacionados sem especificar classes concretas (“kit de UI para Windows e Mac”)                            |
| Builder          | construir objeto complexo <b>passo a passo</b> , separando construção da representação (“montar um objeto com muitos parâmetros opcionais”) |
| Prototype        | criar novos objetos <b>clonando</b> um protótipo (“copiar um objeto existente em vez de criar do zero”)                                     |

#### ATENÇÃO — Como a FGV arma a pegadinha

**Factory Method** (uma criação, via herança) × **Abstract Factory** (famílias de produtos, via composição) — a FGV troca os dois. Singleton = **uma** instância, nunca “mais de uma quando necessário”.

### 4.1.2 Estruturais (como compor objetos/classes)

| Padrão    | Intenção (gatilho)                                                                                                |
|-----------|-------------------------------------------------------------------------------------------------------------------|
| Adapter   | fazer interfaces <b>incompatíveis</b> trabalharem juntas (“adaptar uma API legada à nova”)                        |
| Facade    | interface <b>simplificada</b> para um subsistema complexo (“uma fachada única para vários serviços”)              |
| Proxy     | um <b>substituto</b> que controla acesso ao objeto real (lazy load, cache, segurança)                             |
| Decorator | adicionar responsabilidades a um objeto <b>dinamicamente</b> , sem herança (“empilhar comportamentos em runtime”) |
| Composite | tratar objetos individuais e composições <b>de forma uniforme</b> (árvore parte-todo)                             |
| Bridge    | separar <b>abstração</b> da <b>implementação</b> para variarem independentes                                      |
| Flyweight | compartilhar objetos para economizar memória (muitos objetos semelhantes)                                         |

#### ATENÇÃO — Como a FGV arma a pegadinha

**Adapter** (compatibiliza o que já existe) × **Facade** (simplifica um subsistema) × **Decorator** (adiciona comportamento) — os três se confundem na estrutura, mas a intenção é diferente. **Proxy** controla **acesso**; **Decorator** acrescenta **função**.

### 4.1.3 Comportamentais (como objetos interagem)

| Padrão                  | Intenção (gatilho)                                                                                                                                          |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Strategy                | família de algoritmos <b>intercambiáveis</b> , escolhidos em runtime (“trocar a regra de cálculo sem if gigante”)                                           |
| Observer                | notificar automaticamente <b>dependentes</b> quando o estado muda (publish-subscribe, eventos)                                                              |
| Command                 | encapsular uma <b>requisição como objeto</b> (undo/redo, fila de operações)                                                                                 |
| State                   | mudar o comportamento conforme o <b>estado interno</b> (“o objeto age diferente conforme sua situação”)                                                     |
| Template Method         | definir o <b>esqueleto</b> de um algoritmo, deixando passos para subclasses                                                                                 |
| Iterator                | percorrer uma coleção <b>sem expor sua estrutura</b>                                                                                                        |
| Chain of Responsibility | passar a requisição por uma <b>cadeia</b> de handlers até alguém tratar                                                                                     |
| Mediator                | centralizar a comunicação entre objetos (reduz acoplamento N-para-N)                                                                                        |
| Memento                 | capturar/restaurar o estado de um objeto (snapshot)                                                                                                         |
| Visitor                 | adicionar operações a uma estrutura sem alterar as classes                                                                                                  |
| Interpreter             | definir uma <b>gramática</b> para uma linguagem e um interpretador que avalia suas sentenças (“interpretar expressões/regras escritas numa mini-linguagem”) |

#### ATENÇÃO — Como a FGV arma a pegadinha

**Strategy** (troca o algoritmo) × **State** (troca conforme o estado) — parecidos na estrutura, diferentes na intenção. **Observer** é o padrão de eventos/notificação; não confundir com Mediator (que centraliza a comunicação, não notifica mudança de estado). **Interpreter** é o 11º comportamental e o mais esquecido — some da lista de quem conta de cabeça, e é justamente aí que a FGV pergunta o número.

**Exemplo trabalhado — do if gigante ao Strategy.** O cálculo do frete começa assim, dentro do serviço de pedidos:

| Antes                            | Depois                                                                                             |
|----------------------------------|----------------------------------------------------------------------------------------------------|
| if tipo == "sedex">{...}         | interface <code>CalculoFrete</code> com um método <code>calcular(pedido)</code>                    |
| else if tipo == "pac">{...}      | uma classe por regra: <code>FreteSedex</code> , <code>FretePac</code> , <code>FreteRetirada</code> |
| else if tipo == "retirada">{...} | o serviço <b>recebe</b> a estratégia pronta e só chama <code>calcular</code>                       |

O que se ganhou tem nome: acrescentar a transportadora nova passou a ser **criar uma classe**, não **editar** o método existente — aberto à extensão, fechado à modificação, que é o *Open/Closed* do

SOLID. E o serviço deixou de conhecer as regras concretas: ele depende só da interface, o que é o *Dependency Inversion* do mesmo conjunto. Repare que o padrão não eliminou a decisão de qual regra usar; ele a **moveu para fora**, para quem monta o objeto.

Guarde o contraste, porque é a pegadinha do bloco: **Strategy e State têm a mesma estrutura** — um objeto delegando a outro por uma interface. O que muda é quem decide a troca. No Strategy, **quem escolhe é o cliente**, de fora, e as estratégias não sabem umas das outras. No State, **quem troca é o próprio objeto** conforme seu estado interno, e os estados conhecem a transição seguinte (“pedido pago passa a enviado”). Se o enunciado descreve um ciclo de vida com transições, é State; se descreve alternativas intercambiáveis para a mesma tarefa, é Strategy.

#### 4.1.4 GRASP

O edital cita GRASP no Perfil 2, mas o conceito permeia o Perfil 3. São princípios (não “caixas de código”) para decidir **qual classe recebe qual responsabilidade**: Information Expert, Creator, Controller, Low Coupling, High Cohesion, Polymorphism, Pure Fabrication, Indirection, Protected Variations.

Dois deles respondem sozinhos quase toda questão de “a qual classe cabe esta responsabilidade”. **Information Expert**: a responsabilidade vai para a classe que **tem os dados** necessários para cumpri-la — quem calcula o total do pedido é o **Pedido**, que conhece os itens, não um **CalculadoraDeTotais** que precisaria pedir tudo emprestado. **Creator**: quem cria o objeto é quem o **agrega, contém ou usa de perto** — o **Pedido** cria o **ItemDePedido**. Os dois existem pelo mesmo motivo: manter juntos o dado e o comportamento que o usa é o que produz **alta coesão e baixo acoplamento**, que são eles próprios princípios da lista.

#### 4.1.5 Padrões arquiteturais (não são GoF, mas caem)

- **MVC** (Model-View-Controller): separa dados, apresentação e controle.
- **MVVM / MVP**: variações com binding/presenter.
- **DAO / Repository**: isola o acesso a dados.
- **DTO**: objeto para transportar dados entre camadas.
- **Front Controller**, Singleton de configuração — aparecem em Java EE.

#### 4.1.6 Reuso e anti-padrões

**Reuso**: herança (“é-um”) × **composição** (“tem-um”) — prefira composição (*favor composition over inheritance*). Também: bibliotecas, frameworks, componentes.

**Anti-padrões**: God Object (classe que faz tudo), Spaghetti Code, Golden Hammer (usar sempre a mesma solução), Copy-Paste. A FGV pergunta “qual é o anti-padrão” ou “qual prática viola SOLID”.

#### SOLID — os cinco, para consultar aqui

Os princípios aparecem **com todas as letras no edital do Perfil 2** (item 16) e são a linguagem em que a banca cobra “qual prática viola” e “que princípio o padrão X concretiza”. O tratamento completo, com as armadilhas de Java, está no Capítulo 10; aqui fica o essencial para ligar padrão a princípio sem sair do capítulo:

- **S** — *Single Responsibility*: uma responsabilidade (um motivo para mudar) por classe.
- **O** — *Open/Closed*: aberta à **extensão**, fechada à **modificação**.
- **L** — *Liskov*: o subtipo substitui o tipo base sem quebrar o programa.

- **I** — *Interface Segregation*: interfaces pequenas e específicas, em vez de uma interface gorda.
- **D** — *Dependency Inversion*: depender de **abstração**, não de implementação concreta.

O par que a FGV inverte é **ISP** (interface enxuta) × **SRP** (uma responsabilidade) — os dois falam em “pequeno e específico”, mas um trata de *interface* e o outro de *classe*.

### Como se sair melhor

1. Leia a **intenção**, não a implementação. O enunciado descreve o problema; case com o “para quê” das tabelas acima.
2. Memorize os pares confundíveis: Factory Method × Abstract Factory; Adapter × Facade × Decorator; Strategy × State; Proxy × Decorator.
3. Ligue ao SOLID: Strategy e Template Method concretizam o Open/Closed; Dependency Inversion aparece em Factory/Abstract Factory.

Os mais cobrados pela FGV: **Singleton, Factory Method/Abstract Factory, Strategy, Observer, Adapter, Facade, Decorator, MVC**. O Spring usa Singleton (beans), Factory, Proxy (AOP), Template Method (JdbcTemplate), Observer (eventos) — ligar padrão a framework ajuda a fixar.

### O que já caiu nas provas

**Em prova real da FGV: uma questão na amostra inteira** — **ALERO 2026**, cache de documentos que precisa de uma *única instância ativa* em toda a aplicação, gabarito **Singleton**, com Factory Method, Builder, Observer e Strategy como distratores. O enunciado fala em “Padrão de Projeto da **classificação GoF**” — por isso a tabela das três famílias acima continua valendo o estudo: é assim que a banca ancora o item.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas — 23 questões, subtag **padroes-projeto**): os demais criacionais em cenário (Abstract Factory para famílias de componentes; Builder com muitos atributos opcionais; Prototype quando criar do zero é caro; Factory Method); Adapter para biblioteca de terceiros com interface incompatível; Proxy como substituto de imagem pesada; Decorator para responsabilidade dinâmica; Composite em árvore de pastas e arquivos; Facade acionando vários subsistemas em sequência; Strategy trocando o cálculo de frete em runtime; Observer na planilha que recalcula dependentes; Template Method nas etapas compartilhadas; Command na fila com desfazer; Chain of Responsibility nas validações sucessivas; State no pedido que muda de comportamento por estágio; MVC; e GRASP (a qual classe cabe a responsabilidade).

**Cuidado ao garimpar “já caiu” por palavra-chave**: o corpus tem quatro falsos positivos que *não* são GoF — “Decorators (@)” numa questão de Python (é metaprogramação da linguagem), “Portas e Adaptadores” na arquitetura hexagonal (não é o Adapter), os vários “proxy” das questões de rede, e o MVC, que só aparece descrito sem ser nomeado. Rode `./quiz.py padroes-projeto`.

## 4.2 UML

A UML é uma **linguagem de notação, não um método**: ela padroniza o desenho, não diz quando desenhar — isso é papel do processo (RUP, Scrum; Capítulo 3). O problema que ela resolve é banal e caro: antes dela, cada empresa tinha a própria simbologia, e o mesmo losango significava coisas diferentes em dois documentos da mesma sala. Padronizar a notação é o que permite discutir projeto sem discutir desenho. A especificação vigente da OMG é a **2.5.1**, e a FGV costuma imprimir esse

número no enunciado — é sinal de que o item vai cobrar a semântica exata de um símbolo.

#### 4.2.1 Os 14 diagramas em 2 famílias

| Família         | Foca em                | Diagramas principais                                                                                                               |
|-----------------|------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Estruturais     | estrutura estática     | <b>Classe</b> , Objeto, Componente, Implantação (deployment), Pacote, Estrutura composta, Perfil                                   |
| Comportamentais | comportamento dinâmico | <b>Caso de uso</b> , <b>Sequência</b> , <b>Atividade</b> , <b>Estado</b> (máquina de estados), Comunicação, Interação geral, Tempo |

#### ATENÇÃO — Como a FGV arma a pegadinha

A FGV troca estrutural  $\leftrightarrow$  comportamental. **Classe** é estrutural; **sequência, atividade, estado, caso de uso** são comportamentais. Nunca o contrário.

#### 4.2.2 Diagrama de Classes (o mais cobrado)

Mostra classes, atributos, métodos e relacionamentos:

| Relacionamento          | Significado                                        | Notação                     |
|-------------------------|----------------------------------------------------|-----------------------------|
| Associação              | ligação entre classes                              | linha                       |
| Agregação               | todo-parte <b>fraca</b> (a parte vive sem o todo)  | losango <b>vazio</b>        |
| Composição              | todo-parte <b>forte</b> (a parte morre com o todo) | losango <b>cheio</b>        |
| Herança / Generalização | é-um (subtipo)                                     | seta triângulo <b>vazio</b> |
| Dependência             | usa temporariamente                                | seta tracejada              |
| Realização              | implementa interface                               | tracejada + triângulo       |

**Multiplicidade:** 1, 0..1, 1..\*, \* (muitos). **Visibilidade:** + público, - privado, # protegido, ~ pacote.

#### Ler a caixa e a linha — a sintaxe que a FGV cobra de perto

A classe tem **três compartimentos**: nome, atributos, operações. E o atributo tem uma forma completa, que é onde mora o item de leitura:

```
- status : String [1] = "aberto"
```

visibilidade, nome, tipo, multiplicidade e **valor padrão** — o valor que a instância assume quando ninguém informa outro na criação.

Do lado da linha, duas notações que aparecem como distrator justamente por serem pouco lidas:

- **Extremidade de associação**  $\times$  **atributo**. Cada ponta da associação tem papel, multiplicidade e navegabilidade — e uma ponta navegável **pode ser representada como**

**atributo** da classe do outro lado. “Pedido associa-se a 1 Cliente” e “Pedido tem o atributo **cliente: Cliente**” dizem a *mesma* coisa em UML; são duas notações para um modelo só, não duas modelagens diferentes.

- **Associação qualificada.** Um qualificador (uma chave, desenhado num retângulo na ponta) **particiona** o conjunto do outro lado e tipicamente derruba a multiplicidade de \* para 0..1: dado um banco *e* um número de conta, chega-se a no máximo uma conta. É um índice, não um todo-parte — e é por isso que ela funciona como distrator de agregação.

**Vocabulário da especificação:** o que este capítulo chama de composição a UML chama de **agregação composta** (o losango cheio). Se a alternativa usar esse nome, é composição — não é um terceiro tipo de relacionamento.

#### ATENÇÃO — Como a FGV arma a pegadinha

Pegadinha central do bloco: **agregação** (losango vazio, parte independente) × **composição** (losango cheio, parte dependente). A FGV inverte qual dos dois “morre com o todo”.

### 4.2.3 Casos de Uso (requisitos funcionais)

Ator (boneco), caso de uso (elipse), fronteira do sistema.

- «**include**»: comportamento **sempre** incluído (obrigatório).
- «**extend**»: comportamento **opcional/condicional**.

#### ATENÇÃO — Como a FGV arma a pegadinha

**include** (sempre executa) × **extend** (às vezes executa) — a FGV inverte esse par com frequência.

### 4.2.4 Sequência (interação no tempo)

Linhas de vida (objetos no topo), mensagens trocadas na ordem temporal (de cima para baixo), barras de ativação. Mensagem síncrona (seta cheia) × assíncrona (seta aberta) × retorno (tracejada).

#### Os quatro diagramas de interação — onde a FGV monta os distratores

Quatro diagramas mostram **objetos trocando mensagens**. Como todos “mostram interação”, a banca usa os outros três como distratores do de sequência — e a diferença não é o conteúdo, é **o que cada um enfatiza**:

| Diagrama                        | Enfatiza                                                                                         | Como se lê a ordem                                                              |
|---------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Sequência</b>                | a <b>ordem cronológica</b> das mensagens                                                         | pela posição vertical: de cima para baixo                                       |
| <b>Comunicação</b>              | a <b>estrutura de ligações</b> entre os objetos (quem está conectado a quem)                     | pela <b>numeração</b> das mensagens (1, 1.1, 1.2) — não há eixo de tempo        |
| <b>Tempo</b> ( <i>timing</i> )  | a <b>mudança de estado</b> de cada participante ao longo de uma <b>escala de tempo explícita</b> | pelo eixo horizontal de tempo; usado quando o requisito é de duração/tempo real |
| <b>Visão geral de interação</b> | como <b>vários cenários</b> se encadeiam                                                         | é um diagrama de atividades cujos nós são interações                            |

A regra de decisão cabe numa linha: pediu a **ordem exata em que as mensagens acontecem**, é **sequência**. Sequência e comunicação são **semanticamente equivalentes** (dá para converter um no outro sem perder informação) — o que muda é a ênfase, e é justamente aí que a alternativa “comunicação” parece defensável e não é, quando o enunciado fala em cronologia. Não confunda com **atividade**: lá não há troca de mensagens entre participantes, há **fluxo de trabalho** — ações, decisões e paralelismo.

#### 4.2.5 Atividade e Estado

**Atividade:** fluxo de trabalho/algoritmo — nós de ação, decisão (losango), bifurcação/junção (barra, paralelismo), início/fim. Parecido com fluxograma; usado para modelar processos.

**Máquina de estados:** os estados de um objeto e as transições disparadas por eventos (o objeto reage diferente conforme o estado — casa com o padrão de projeto State).

##### Como se sair melhor

1. Decore a família de cada diagrama (estrutural × comportamental) — é a pegadinha mais comum.
2. No diagrama de classes, memorize agregação × composição (vazio × cheio) e as multiplicidades.
3. Caso de uso: include (obrigatório) × extend (opcional).
4. Ligue ao resto: caso de uso ↔ requisitos funcionais; classes ↔ OO/SOLID; estado ↔ padrão State.

Diagramas mais cobrados: **Classe, Caso de Uso, Sequência, Atividade**. Deployment e Componente aparecem em questões de arquitetura (Capítulo 5).

##### O que já caiu nas provas

**Em prova real da FGV:** ao contrário dos padrões, UML **cai**, e sempre citando a versão. **Diagrama de sequência** para a ordem cronológica exata das mensagens — e os distratores eram os outros diagramas de interação (**comunicação, timing, visão geral de interação**) mais o de atividades; **composição** para a parte que não existe sem o todo e some junto com ele (distratores: agregação, dependência, associação qualificada); **diagrama de casos**



**de uso** combinado com o **diagrama de objetos** para montar plano de teste (o de objetos mostra os *valores* dos atributos das instâncias num ponto da execução) — **ALERO 2026**, sempre com “UML 2.5.1” no enunciado. Leitura de **diagrama de classes** (extremidade de associação representável como atributo, agregação composta, valor default de atributo) — **MPU**. **Generalização/especialização total e disjunta** caiu na **ALERO 2026** pelo lado do modelo ER, com a mesma semântica.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas — 10 questões, subtag `uml`): as duas famílias na UML 2.x; multiplicidade em diagrama de classes; «**include**» × «**extend**»; atividade para decisão, paralelismo e sincronização; máquina de estados no ciclo de vida do objeto; direção da seta da generalização; componentes × implantação; realização de interface.

Rode `./quiz.py uml`.

## Capítulo 5

# Arquitetura de Software

### Ficha do edital

Arquitetura de software; interoperabilidade; arquitetura e linguagem orientada a serviços; web services; mensageria; API, Swagger; arquitetura orientada a objetos; aplicações para ambiente web; servidor de aplicações × servidor web; internet/extranet/intranet/portal; arquitetura hexagonal; microserviços (orquestração e API gateway); containers; transações distribuídas.

**Peso esperado: ALTO** — nuvem, REST×SOAP, microserviços e containers são recorrentes.

### 5.1 Servidor web × servidor de aplicação

|          | Servidor web                                  | Servidor de aplicação                             |
|----------|-----------------------------------------------|---------------------------------------------------|
| Função   | atende HTTP, serve conteúdo estático/dinâmico | executa <b>lógica de negócio</b> , integra com BD |
| Exemplos | Apache, NGINX                                 | JBoss/WildFly, WebLogic, Tomcat (parcial)         |

No mundo Java a distinção fica literal: o servidor de aplicação implementa a especificação **Java EE/Jakarta EE** completa — ele traz o **container web** (Servlet/JSP) e **mais** o **container EJB** para os componentes de negócio, além de transações distribuídas, JMS e injeção de dependência. O Tomcat, por isso, é servidor web + container web, mas não é servidor de aplicação completo: não tem container EJB.

### ATENÇÃO — Como a FGV arma a pegadinha

A FGV troca os papéis (“servidor web processa lógica de negócio”). O **web** recebe a requisição HTTP; o **de aplicação** roda a regra de negócio. Outros distratores do par: dizer que o servidor de aplicação “é sempre mais leve” (é o oposto — ele carrega containers a mais), que “dispensa a JVM” (roda sobre ela) ou que “não suporta HTTP” (suporta; ele contém o container web). O “sempre” já entrega o primeiro.

### 5.2 Estilos de arquitetura

Os estilos não são uma lista de opções paralelas: são uma **sequência histórica**, e cada um nasceu resolvendo a dor do anterior. O monólito incomodava porque uma equipe travava a outra — subir uma correção de uma tela obrigava a reimplantar o sistema inteiro. A SOA respondeu quebrando em serviços com contrato explícito, mas concentrou a inteligência no barramento, e o barramento virou o novo gargalo. Os microserviços responderam à SOA invertendo isso: *canais burros, serviços espertos*.

Saber a sequência é o que permite responder às questões de cenário, que nunca perguntam “o que é X” e sim “qual estilo resolve esta dor”. A dor descrita no enunciado aponta o estilo.

### Do monólito aos microsserviços

- **Monolítica:** tudo num artefato único, implantado junto. Simples de começar; acopla e dificulta escalar partes isoladas.
- **Cliente-servidor, N camadas, P2P, barramento de mensagens.**
- **SOA (orientada a serviços):** serviços reutilizáveis, contrato explícito, baixo acoplamento, interoperabilidade. Web services são a tecnologia comum. O barramento dessa arquitetura é o **ESB (Enterprise Service Bus)**: o intermediário que **roteia, transforma** (converte formato/protocolo entre sistemas que não se falam) e **orquestra** as mensagens trocadas entre os serviços. Concentra a inteligência da integração — o que simplifica o cliente e, em troca, cria um ponto único de falha e de gargalo. É a diferença de filosofia para os microsserviços, que preferem canais burros e serviços espertos.
- **Microsserviços:** serviços pequenos, autônomos, **implantáveis independentemente**, cada um com **seu próprio banco** (não compartilham base — isso reduz acoplamento). Comunicação leve (REST/mensageria).
- **Arquitetura hexagonal (Ports & Adapters):** separa a **lógica de negócio** do mundo externo por **portas** (interfaces) e **adaptadores**; permite trocar UI/BD/serviços sem tocar no núcleo.

### O preço dos microsserviços — por que nem sempre valem

A banca gosta de cenários em que microsserviço é a resposta *errada*, e quem só decorou a lista de vantagens cai. O que se ganha vem com uma conta:

| Ganho                                     | Preço correspondente                                                                                                                  |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| implantar um serviço sem tocar nos outros | precisa de automação de <i>deploy</i> e de observabilidade distribuída — sem isso, achar a causa de um erro fica pior que no monólito |
| escalar só a parte que aperta             | a chamada entre serviços virou <b>chamada de rede</b> : pode falhar, tem latência, exige repetição e <i>timeout</i>                   |
| cada serviço com seu banco                | acabou o JOIN entre eles, e acabou a transação ACID que atravessa os dois — daí Saga e consistência eventual                          |
| equipes autônomas                         | só compensa se as equipes <i>forem</i> várias; para um time pequeno, é custo sem benefício                                            |

Daí a regra prática: microsserviço resolve problema **organizacional** e de **escala independente**. Se o enunciado descreve sistema pequeno, equipe única ou forte consistência transacional entre módulos, o monólito (eventualmente modular) é a resposta certa — e “migrar para microsserviços” é a quase-certa plantada.

**ATENÇÃO — Como a FGV arma a pegadinha**

“Microsserviços compartilham o mesmo banco” = **falso** (aumenta acoplamento, contra o princípio). “Monólito não pode ser distribuído” = falso. Baixo acoplamento significa que mudança em um serviço **não** obriga mudança no outro — se a alternativa disser que “se reflete diretamente”, descarte. Num item que descreve “o barramento que intermedeia, roteia e transforma mensagens entre serviços”, a resposta é **ESB** — os distratores costumam ser siglas de outras prateleiras (ETL move dados para o DW, CDN entrega conteúdo estático, DNS resolve nome, VPN faz túnel).

**5.2.1 Camadas lógicas (layers) × camadas físicas (tiers)**

Duas coisas diferentes que a tradução para o português funde em “camada”:

|                       |                                                                                                     |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| <b>Layer (lógica)</b> | como as <b>responsabilidades</b> do software estão organizadas: apresentação, negócio, persistência |
| <b>Tier (física)</b>  | em <b>quantos nós</b> o software está implantado: máquinas, processos, servidores                   |

Uma coisa não obriga a outra: uma aplicação pode ter **três layers e um único tier** — as três responsabilidades separadas no código, tudo rodando no mesmo processo, e a aplicação continua **monolítica**. O inverso também existe: cliente-servidor em dois tiers pode manter apresentação e negócio bem separados logicamente.

**ATENÇÃO — Como a FGV arma a pegadinha**

A troca clássica é definir layer como “distribuição entre máquinas” e tier como “agrupamento de responsabilidades” — está invertido. O absoluto também cai: “três camadas lógicas são **necessariamente** implantadas em três nós físicos”. E o MVC **não** corresponde a três tiers: é padrão de organização (apresentação), não de implantação.

**5.3 Integração: REST × SOAP, Web Services****O que REST é de verdade — recurso, representação, verbo**

REST não é “API que devolve JSON”. É um **estilo arquitetural** com uma ideia central: tudo que interessa é um **recurso**, identificado por uma URI, e o cliente nunca manipula o recurso — ele troca **representações** dele (JSON, XML, HTML). Daí decorre o resto:

- **Interface uniforme.** O conjunto de verbos é fixo e igual para todo recurso: GET lê, POST cria, PUT substitui, PATCH altera em parte, DELETE remove. É por isso que a URI deve nomear *coisa* e não *ação*: `/pedidos/42` está certo, `/buscarPedido?id=42` contraria o estilo, porque põe o verbo na URI quando ele já está no método.
- **Sem estado (*stateless*).** Cada requisição carrega tudo o que é preciso para ser entendida sozinha. O ganho não é ideológico: se o servidor não guarda sessão, **qualquer** instância atende **qualquer** requisição — e é exatamente isso que permite pôr um balanceador na frente e escalar horizontalmente sem afinidade de sessão.
- **Cliente-servidor, cache, camadas.** O cliente não sabe se está falando com o servidor final ou com um intermediário; por isso proxy, gateway e CDN cabem no caminho sem

quebrar nada.

Repare no encadeamento: *ausência de estado* → *qualquer instância serve* → *escala horizontal*. Quando a FGV pergunta “o que no REST sustenta a escalabilidade”, é essa corrente que ela está cobrando.

|          | REST                                  | SOAP                           |
|----------|---------------------------------------|--------------------------------|
| Estilo   | arquitetural, sobre HTTP              | protocolo baseado em XML       |
| Formato  | JSON (comum), leve                    | XML (envelope), verboso        |
| Contrato | OpenAPI/Swagger                       | WSDL                           |
| Estado   | <b>stateless</b>                      | pode ter padrões WS-*          |
| Vantagem | leve, escala, independe de plataforma | contratos formais, WS-Security |

- **REST é stateless:** cada requisição carrega tudo; o servidor não guarda **estado de sessão** → escala horizontalmente sem afinidade de sessão.
- **Cacheable é outra restrição:** a resposta se declara cacheável ou não, e quem a guarda pode ser o **cliente** ou um **intermediário** — proxy, gateway, CDN. Cache no caminho é **permitido** pelo REST; é justamente o que deixa uma CDN servir o conteúdo estático de um nó próximo do usuário (Seção 5.5). Não confunda: o que não pode ficar no servidor é o **estado da sessão**, não o cache.
- **API Gateway:** ponto único de entrada dos microsserviços (roteamento, autenticação, rate limit, agregação).
- **Métodos HTTP:** GET (ler), POST (criar), **PUT (substituir inteiro)**, **PATCH (atualizar parcial)**, DELETE. Códigos: 200 OK, 201 Created, 204 No Content, 400, 401, 403, 404, 500.
- **UDDI:** diretório (legado) para **descoberta/registro** de web services SOAP, junto do WSDL (contrato) e SOAP (mensagem). Trio clássico WS-\*: **SOAP + WSDL + UDDI**.
- **Swagger / OpenAPI:** o **OpenAPI** é a especificação padrão para **documentar e contratar APIs REST** (endpoints, parâmetros, respostas); **Swagger** é o conjunto de ferramentas (Swagger UI, editor, codegen) em cima dela. É “o WSDL do REST”.

#### ATENÇÃO — Como a FGV arma a pegadinha

**PUT × PATCH** (substituição total × parcial); dizer que SOAP não tem contrato formal contradiz o próprio SOAP (que usa WSDL); UDDI é **descoberta**, WSDL é **contrato**, SOAP é a **mensagem** — não misture os três papéis.

### 5.3.1 Mensageria (comunicação assíncrona)

Desacopla produtor e consumidor: quem envia não espera quem processa.

| Modelo                     | Como funciona                                                | Exemplo       |
|----------------------------|--------------------------------------------------------------|---------------|
| Fila (queue)               | mensagem consumida por <b>um</b> consumidor (point-to-point) | RabbitMQ, SQS |
| Publish/Subscribe (tópico) | mensagem entregue a <b>vários</b> assinantes                 | Kafka, SNS    |

**Apache Kafka:** plataforma de **streaming** de eventos — produtores publicam em **tópicos** (particionados), consumidores leem em **grupos**; guarda o log de eventos (permite reprocessar). Alta vazão, escalável.

Benefícios da mensageria: desacoplamento, absorção de picos (buffer), resiliência. O **broker** é o intermediário (Kafka, RabbitMQ). Casa com **microserviços** e com os padrões **Saga** (eventos de compensação) e **Event Sourcing**.

#### ATENÇÃO — Como a FGV arma a pegadinha

Fila (**um** consumidor) × tópico/pub-sub (**vários**) — não inverta. Mensageria é **assíncrona** (diferente da chamada REST síncrona).

## 5.4 Ambientes de rede corporativa

| Termo    | Alcance                                                     |
|----------|-------------------------------------------------------------|
| Internet | rede pública global                                         |
| Intranet | rede interna da organização (só funcionários)               |
| Extranet | acesso controlado a parceiros/clientes externos autorizados |
| Portal   | ponto único que centraliza informação/serviços              |

#### ATENÇÃO — Como a FGV arma a pegadinha

Trocar intranet ↔ extranet ↔ internet. Extranet = interno + parceiros externos autorizados (o meio-termo).

## 5.5 Nuvem e escalabilidade

#### IaaS, PaaS, SaaS — onde passa a linha de responsabilidade

Os três modelos respondem a uma pergunta só: **até onde vai o provedor e onde começa você**. A pilha é sempre a mesma — rede, armazenamento, servidores, virtualização, sistema operacional, *middleware*, tempo de execução, dados, aplicação — e o modelo apenas move a linha de corte para cima.

|             | Você cuida de                                                   | Exemplo típico                           |
|-------------|-----------------------------------------------------------------|------------------------------------------|
| <b>IaaS</b> | sistema operacional para cima: patch, runtime, aplicação, dados | máquina virtual alugada                  |
| <b>PaaS</b> | aplicação e dados; o resto é do provedor                        | plataforma que recebe seu código e roda  |
| <b>SaaS</b> | só os <b>dados</b> e a configuração de uso                      | software pronto, acessado pelo navegador |

Duas consequências que a banca cobra. Primeira: **quanto mais “as a Service”, menos controle** — não é só menos trabalho, é também menos liberdade para customizar. Segunda, a

**responsabilidade compartilhada** em segurança: mesmo em SaaS, **o dado continua sendo seu**. O provedor responde pela segurança *da* nuvem (infraestrutura); o cliente responde pela segurança *na* nuvem — quem tem acesso, como o dado é classificado, se a conta usa segundo fator. A alternativa que diz que o SaaS “transfere integralmente a responsabilidade pelos dados ao provedor” é falsa, e é a plantada.

- **Modelos de serviço: IaaS** (infra), **PaaS** (plataforma), **SaaS** (software pronto). Regra: quanto mais “as a Service”, menos você gerencia.
- **Modelos de implantação:** privada, pública, híbrida.
- **Escalabilidade horizontal (scale out):** adicionar instâncias/nós.
- **Escalabilidade vertical (scale up):** adicionar recursos à instância existente (mais CPU/RAM).
- **Elasticidade:** ajustar capacidade automaticamente conforme a demanda. **Cloud bursting:** estende para nuvem pública no pico de demanda.
- **Serverless / FaaS (Function as a Service):** o código é publicado como **função** e o provedor executa por **evento**. Não significa “sem servidor” — significa que o **cliente não gerencia** servidor. Características cobradas: escala **de zero** a muitas execuções conforme os eventos; **cobrança pelo tempo e pelos recursos consumidos** (não por instância parada); execução **sem estado** entre invocações (o estado vai para armazenamento externo); **cold start**, a latência maior na primeira invocação após um período ocioso; e **limite de tempo por execução**, o que o desaconselha para processamento longo e contínuo.
- **Balanceador de carga × CDN:** atacam gargalos **diferentes** e por isso se somam. O **balanceador** reparte as requisições entre as instâncias da aplicação — resolve **sobrecarga de processamento**. A **CDN** replica o conteúdo **estático** em pontos de presença espalhados e o entrega do nó mais próximo — resolve **latência geográfica**.
- **Containers (Docker) × VM:** container compartilha o **kernel** do SO (leve, rápido); VM tem SO convidado completo sobre um hipervisor (isola mais, pesa mais). **Kubernetes** orquestra containers.
- **Hipervisor Tipo 1** (bare-metal, roda direto no hardware: ESXi, Hyper-V, KVM) × **Tipo 2** (hosted, roda sobre um SO: VirtualBox, VMware Workstation).
- **Virtualização total** (SO convidado não sabe que é virtual) × **paravirtualização** (SO convidado é modificado e coopera com o hipervisor — mais rápido). **VDI** = virtualização de desktops entregues remotamente.

### ATENÇÃO — Como a FGV arma a pegadinha

“Adicionar recursos à instância” é sempre **vertical**, jamais horizontal. Container  $\neq$  VM (kernel compartilhado  $\times$  SO próprio). Em questões de custo-benefício, some a capacidade das novas instâncias e compare o preço antes de decidir entre escalar vertical ou horizontal.

Em **serverless**, cada distrator nega uma característica: “não há servidor, o código roda no dispositivo que dispara o evento” (há servidor, do provedor); “o estado da execução anterior permanece” (é sem estado); “a latência da primeira invocação é igual à das demais” (ignora o *cold start*); “remove o limite de tempo por execução” (o limite existe — é justamente o que desaconselha processamento longo).

Em **balanceador × CDN**, a inversão é trocar os papéis (“a CDN distribui as requisições dinâmicas e o balanceador replica o estático na borda”) ou declarar um dispensável/redundante. Guie-se pelo sintoma: “instâncias sobrecarregadas”  $\rightarrow$  balanceador; “lentidão para usuários distantes / arquivos estáticos”  $\rightarrow$  CDN.



## 5.6 Transações distribuídas

O problema nasce no instante em que cada microsserviço ganha o próprio banco. Dentro de um SGBD, o ACID é de graça: existe um só log, um só gerenciador de bloqueios, e ele decide sozinho. Distribuído, não há árbitro — “debitar no serviço de contas e reservar no serviço de estoque” são **duas** transações, em **dois** bancos, ligadas por uma rede que pode cair no meio. Não existe operação que efetive as duas ao mesmo tempo; o que existe são protocolos que tentam *coordenar* isso, cada um pagando um preço diferente. É o mesmo dilema do teorema CAP (Capítulo 6), visto do lado de quem escreve a aplicação.

- **2PC (Two-Phase Commit):** coordenador + participantes; exige **unanimidade** (todos confirmam) para commit. Bloqueante.
- **Saga:** sequência de transações locais com **compensação** em caso de falha (padrão para microsserviços).
- **Raft:** algoritmo de **consenso** — resolve um problema *diferente* do 2PC. Não pergunta “todos aceitam efetivar esta transação?”, e sim “em que **sequência de operações** este grupo de réplicas concorda?”. Funciona por **eleição de líder** (o líder ordena e replica a *log*) e decide por **maioria** (quórum), o que o torna **tolerante a falhas**: sobrevive à queda da minoria, inclusive do líder, que é substituído por nova eleição. É o que sustenta o *etcd* (e, por tabela, o Kubernetes).

### ATENÇÃO — Como a FGV arma a pegadinha

**2PC × Raft** é o par do capítulo. O 2PC é *commit atômico*: exige **unanimidade** e **bloqueia** se o coordenador cair (ponto único de falha). O Raft é *consenso*: decide por **maioria** e **continua funcionando** com a minoria fora. Trocar “unanimidade” por “maioria” de um para o outro é o distrator pronto — e a alternativa que diz que o 2PC “tolera a falha do coordenador” é falsa.

### O que já caiu nas provas

**Em prova real da FGV:** servidor web × de aplicação; SOA e web services (baixo acoplamento); arquitetura hexagonal × microsserviços — a afirmativa falsa é justamente “microsserviços compartilham o mesmo banco”; **design × arquitetura** (amplo/estrutural contra detalhado/específico) — **Dataprev 2024**, que também cobrou internet/intranet/extranet/portais, ali como item de *redes*. Escalabilidade horizontal × vertical **com cálculo de custo-benefício**; **REST stateless** (a escalabilidade vem da ausência de estado no servidor); **taints/tolerations** no Kubernetes; **cloud bursting**; container × VM em cenário de latência e isolamento; **object storage** de espaço de nomes plano; cloud-native × híbrida; cliente-servidor à Sommerville (o modelo é **lógico**); **WSDL** como descrição do web service; RabbitMQ e a entrega *exactly-once* — **TJ-RJ**. DDD depois da modelagem do domínio e escolha de distribuição do Kubernetes sob o Rancher — **MPU**. Balanceador de carga com **afinidade de sessão** (e o preço que ela cobra em tolerância a falhas), framework de arquitetura corporativa por *views* e *viewpoints*, **VDI**, PaaS gerenciado, **paravirtualização** com *hypercalls*, responsabilidade compartilhada em SaaS e **WSDL** de novo — **ALERO 2026**.

**Fora do edital:** 11 questões que vinham marcadas como **arquitetura** eram **arquitetura de computadores e sistema operacional** — Von Neumann e seu gargalo, ULA e unidade de controle, *overflow × carry*, conversão de base, ROM e firmware, ciclo busca-decodificação-execução, MMU/TLB e *thrashing*, tabela de páginas, DMA. Vieram de provas de **outro perfil** (ALERO 2026 e TJ-RJ), e o edital do Perfil 3 fala em arquitetura *de software*. Foram **movidas para orfaos** (Capítulo 14), de onde não poluem mais a contagem deste bloco: sobram **23** questões reais de arquitetura de software. Saiba que existem, não gaste tempo nelas.



**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): **ESB** como barramento da SOA; **API gateway**; **Raft** (o 2PC, esse sim, caiu na ALERO 2026 pelo lado de banco distribuído); layers × tiers como par; serverless/FaaS e CDN como item próprio — os dois só apareceram como nome de produto entre distratores. Rode `./quiz.py arquitetura`.

## 5.7 Alta probabilidade / pesquisa extra

- **12-Factor App** (boas práticas de app nativa de nuvem).
- **DDD (Domain-Driven Design): Aggregate** (fronteira de consistência, protege invariantes — não expõe referência a cada entidade interna), **Repository** (coleção/persistência do agregado, não instancia por métodos externos), **Factory** (criação), **Ubiquitous Language** (vocabulário comum domínio↔código), eventos de domínio **imutáveis** (fato passado). Caiu no **MPU 2025** (o gabarito era o *Aggregate* garantindo a consistência das mudanças num modelo de associações complexas) e no **TJ-RJ 2** (gabarito: eventos de domínio são ordinariamente imutáveis, por registrarem algo já ocorrido).
- **Service mesh** (Istio) × **API gateway**: mesh cuida da comunicação serviço-a-serviço (leste-oeste); gateway cuida da entrada (norte-sul).
- **IaC (Infraestrutura como Código)**: Terraform, Ansible — provisiona ambiente de forma declarativa e versionada.
- **Kubernetes** — *taint* × *toleration*: são o par de **repulsão**, e funcionam ao contrário do que o nome sugere. O *taint* vai no **nó** e *afasta* pods: “não me escalone nada, a menos que aceite esta marca”. A *toleration* vai no **pod** e diz “eu *tolero* essa marca”, tornando-o *elegível* àquele nó. Serve para reservar nós especiais (GPU, banco, nó mestre). Cuidado: tolerar **não é atrair** — o pod que tolera o taint *pode* ir para lá, não *tem* que ir; quem atrai é *node affinity/nodeSelector*. Distribuições: RKE1/RKE2/K3s/EKS, Rancher.
- **Armazenamento de objetos**: flat namespace, API REST.
- **Fundamentos de SO que a FGV cobra como “arquitetura”**: paginação/memória virtual, E/S bloqueante, troca de contexto.

### Como se sair melhor

Na dúvida entre duas alternativas, os gatilhos de distrator deste bloco denunciam a errada: “sempre”, “exclusivamente”, “elimina a necessidade de”, “idêntico ao”. Os pares que a FGV inverte aqui — vertical×horizontal, REST×SOAP, container×VM, fila×tópico, web×aplicação — estão nas caixas vermelhas acima: revise por elas, não por um resumo que as repita.

## Capítulo 6

# Banco de Dados

### Ficha do edital

**Atenção ao recorte:** o Perfil 3 *não tem* “Banco de Dados” como disciplina. O assunto entra por duas portas: **Inteligência de Negócios** (“arquitetura e aplicações de *data warehouse* com ETL e OLAP”, “*data warehouse* e *data mining*”, “visualização de dados: BD individuais e cubos”) e o item 20 de Desenvolvimento de Sistemas (“Conceitos de Inteligência Artificial, **Análise de Dados** e Big Data”).

A lista fechada — modelagem e normalização, arquitetura, estrutura de dados, SQL (ANSI), administração de dados, backup/restauração, engenharia de dados/Big Data — e os **SGBD nomeados** (Oracle 19C, MySQL, PostgreSQL, MongoDB, MS-SQL Server 2019) estão no **Perfil 2**. Estudamos assim mesmo porque a **FGV cobrou banco de dados no Perfil 3 da Dataprev 2024** — do mesmo jeito que cobrou redes, que também não está no perfil.

**Peso esperado:** ALTO por evidência de prova, não por peso de edital — metade do “eixo duplo” com Engenharia de Software; no MPU 2025 foi o bloco que mais caiu.

## 6.1 Modelagem em três níveis

Modelar um banco é responder três vezes à mesma pergunta, com liberdade decrescente a cada rodada. **O que existe no mundo do negócio?** — existem servidores, e cada servidor tem dependentes. **Como isso vira estrutura de dados?** — vira duas tabelas, ligadas por uma chave. **Como isso roda neste produto?** — vira um `CREATE TABLE` com tipos concretos, um índice na coluna de busca e uma escolha de armazenamento.

A separação não é burocracia: os três níveis **mudam em ritmos diferentes**. A regra “todo dependente pertence a um servidor” sobrevive à troca do Oracle pelo PostgreSQL; o `NUMBER(5)` não. Quem mistura os níveis redesenha o negócio toda vez que troca de infraestrutura — e é exatamente essa independência que a banca cobra ao perguntar qual nível independe do SGBD.

| Nível      | O que é                                    | Artefato                                 |
|------------|--------------------------------------------|------------------------------------------|
| Conceitual | visão de negócio, independente de SGBD     | Modelo Entidade-Relacionamento (MER/DER) |
| Lógico     | estrutura no modelo escolhido (relacional) | tabelas, chaves, tipos genéricos         |
| Físico     | implementação no SGBD específico           | DDL, índices, particionamento            |

### O que cada nível pode decidir — e o que ele não pode

O critério para saber em que nível você está é **o que já foi decidido**:

- **Conceitual.** Decide *entidades, atributos e relacionamentos*. Não sabe o que é tabela nem o que é chave estrangeira. Se o artefato fala em “FK” ou em `VARCHAR`, ele já não é conceitual. É o nível que se discute **com o usuário**.
- **Lógico.** Já escolheu o *paradigma* (relacional, documento, grafo) e traduz o conceitual para ele: entidades viram tabelas, relacionamentos viram chaves estrangeiras ou tabelas associativas, atributos ganham tipos genéricos. Ainda **não** escolheu o produto.
- **Físico.** Já escolheu o produto e decide o que só existe lá dentro: tipo exato da coluna, índice, partição, *tablespace*, compressão. É o único nível em que “desempenho” é resposta legítima.

Repare que a informação **só se acrescenta**: cada nível carrega tudo o que o anterior decidiu e soma o seu. É por isso que a ordem não é invertível — não se parte do índice para descobrir qual é a entidade.

### ATENÇÃO — Como a FGV arma a pegadinha

Ordem correta: conceitual → lógico → físico. A FGV inverte a ordem ou troca o artefato de cada nível.

## 6.2 Modelo relacional e integridade

O modelo relacional (Codd, 1970) tem uma ideia central e uma consequência inevitável. A ideia: **todo dado é uma tabela, e a única forma de ligar duas tabelas é o valor de um campo coincidir com o de outro** — não há ponteiro, não há endereço, não há ordem garantida. A consequência: se o vínculo é por valor, alguém precisa **garantir que o valor referenciado exista**, senão a ligação aponta para o nada. É disso que trata integridade.

O caso concreto: se `DEPENDENTE.servidor_id` vale 47 e não existe servidor 47, aquela linha é lixo — ninguém consegue dizer de quem é o dependente, e nenhuma consulta com `JOIN` vai trazê-la de volta. A restrição de integridade referencial é o que impede a linha de nascer. Sem ela, o erro não aparece no `INSERT`: aparece meses depois, num relatório que não fecha.

- **Chave primária (PK):** identifica unicamente a tupla; não nula, única.
- **Chave estrangeira (FK):** referencia a PK de outra (ou da mesma) tabela.
- **Integridade referencial:** toda FK aponta para uma PK existente (ou é nula). Ações: `CASCADE`, `SET NULL`, `RESTRICT/NO ACTION`.
- **Integridade de entidade:** PK não pode ser nula.
- **Álgebra relacional:** seleção ( $\sigma$ ) = `WHERE` = filtra **linhas**; projeção ( $\pi$ ) = filtra **colunas**.

### As ações referenciais — o que fazer quando o pai morre

A pergunta que o `ON DELETE` responde é esta: apagaram o servidor 47, e existem dependentes apontando para ele. O que acontece com os filhos?

| Ação        | O que o SGBD faz                                                                                                                                   |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| CASCADE     | apaga (ou atualiza) os filhos junto — o dependente some com o servidor                                                                             |
| SET NULL    | mantém o filho e zera a FK — exige que a coluna aceite NULL                                                                                        |
| SET DEFAULT | mantém o filho e aponta a FK para um valor padrão, que precisa existir                                                                             |
| RESTRICT    | <b>barra</b> a operação de imediato                                                                                                                |
| NO ACTION   | barra também, mas a checagem fica para o <b>fim do comando</b> — permite que um passo intermediário viole, desde que o estado final esteja íntegro |

RESTRICT e NO ACTION são o par que a banca usa como “quase certa”: os dois recusam, e a diferença é só *quando* a verificação acontece. CASCADE é o oposto dos dois — ele não recusa nada, ele propaga. E note o que isso significa na prática: CASCADE numa tabela central pode apagar em silêncio meia base a partir de um único DELETE.

### 6.2.1 Do MER para o relacional (como cada cardinalidade vira tabela)

| Cardinalidade | Implementação no modelo relacional                                                                          |
|---------------|-------------------------------------------------------------------------------------------------------------|
| 1:1           | FK em uma das duas tabelas (de preferência no lado de participação obrigatória), com restrição de unicidade |
| 1:N           | <b>FK no lado N</b> — a tabela “muitos” guarda a chave da tabela “um”                                       |
| N:M           | <b>tabela associativa</b> (intermediária), com as FKs das duas pontas formando a chave primária composta    |

#### ATENÇÃO — Como a FGV arma a pegadinha

N:M **nunca** se resolve com FK direta em uma das tabelas (isso só atende 1:N) nem com coluna multivalorada dos dois lados — coluna multivalorada viola a **1FN**. Trigger e índice composto não implementam cardinalidade nenhuma: são distratores de outra prateleira.

#### Entidade fraca × entidade associativa

**Entidade fraca:** não se identifica sozinha. Sua chave é a chave da **entidade proprietária** (forte) somada a uma **chave parcial** (discriminador) — ex.: DEPENDENTE só é identificado dentro do cadastro de um SERVIDOR, pelo nome. O relacionamento com o proprietário é **identificador**, e a existência da fraca depende da forte.

**Entidade associativa:** é outra coisa — é o **relacionamento N:M promovido a entidade**, quando ele próprio precisa se relacionar com uma terceira entidade ou ganhar atributos próprios.

#### ATENÇÃO — Como a FGV arma a pegadinha

A quase-certa diz “entidade fraca, que **dispensa** a chave do proprietário” — é justamente o oposto: sem a chave do proprietário ela não se identifica. E ter atributos próprios **não** torna a

entidade forte; o que define a força é conseguir se identificar sozinha.

## 6.3 Normalização (formas normais)

Normalizar resolve um problema concreto; não é obediência a uma regra estética. O problema chama-se **anomalia**, e ele aparece sempre que uma tabela guarda **dois assuntos ao mesmo tempo**.

### Dependência funcional — a ferramenta que sustenta tudo

Escreve-se  $A \rightarrow B$  e lê-se “**A determina B**”: dado um valor de A, existe **um único** valor de B possível.  $CPF \rightarrow nome$  é dependência funcional — um CPF, um nome.  $nome \rightarrow CPF$  não é: homônimos existem.

Toda forma normal é uma restrição sobre *quais* dependências funcionais uma tabela tem direito de ter. A chave determina todo o resto — isso é o que significa ser chave. O que a normalização proíbe é que **alguma coisa além da chave** determine alguma coisa:

- determinar a partir de **parte** da chave  $\rightarrow$  viola a **2FN** (dependência parcial) — só é possível com chave composta, e é por isso que a 2FN não tem o que fazer numa tabela de chave simples;
- determinar a partir de um atributo **não-chave**  $\rightarrow$  viola a **3FN** (dependência transitiva);
- determinar sendo algo que **não é chave candidata**  $\rightarrow$  viola a **BCNF**.

Repare que as três são a mesma frase ficando mais rigorosa. Daí a ordem ser cumulativa: quem está em 3FN já está em 2FN e em 1FN.

| FN          | Elimina                                                          |
|-------------|------------------------------------------------------------------|
| 1FN         | atributos multivalorados / não atômicos (cada célula um valor)   |
| 2FN         | dependência parcial da chave (só relevante com PK composta)      |
| 3FN         | dependência transitiva (atributo que depende de outro não-chave) |
| FNBC (BCNF) | anomalia quando há mais de uma chave candidata sobreposta        |

Ordem:  $1FN \rightarrow 2FN \rightarrow 3FN \rightarrow BCNF$ . Normalizar reduz redundância; **desnormalizar de propósito** (ex.: Data Warehouse) é aceitável por desempenho de leitura — objetivo é leitura, nunca escrita.

### 6.3.1 A decomposição, passo a passo

Uma tabela de matrículas guardando aluno, curso e nota — do jeito que um sistema mal projetado a criaria:

| aluno | curso | nota | nome_aluno | depto      | chefe_depto |
|-------|-------|------|------------|------------|-------------|
| 101   | BD01  | 8,5  | Ana        | Computação | Prof. Silva |
| 101   | ES02  | 7,0  | Ana        | Computação | Prof. Silva |
| 102   | BD01  | 9,0  | Bruno      | Computação | Prof. Silva |

A chave é composta: **(aluno, curso)** — é o par que identifica uma matrícula. E as dependências funcionais são estas três:

| Dependência                                                 | Status                                             |
|-------------------------------------------------------------|----------------------------------------------------|
| $(\text{aluno}, \text{curso}) \rightarrow \text{nota}$      | legítima: depende da chave <b>inteira</b>          |
| $\text{aluno} \rightarrow \text{nome\_aluno}, \text{depto}$ | <b>parcial</b> : depende de <i>metade</i> da chave |
| $\text{depto} \rightarrow \text{chefe\_depto}$              | <b>transitiva</b> : um não-chave determina outro   |

**O estrago que isso causa** — as três anomalias, nesta tabela:

- **De atualização:** mudou o chefe da Computação. Ele está repetido em toda linha de todo aluno do departamento; se um UPDATE escapar, a tabela passa a afirmar duas coisas contraditórias ao mesmo tempo.
- **De inserção:** criou-se um departamento novo, ainda sem aluno nenhum. Não há onde guardá-lo — a chave exige (aluno, curso), e não existe aluno.
- **De exclusão:** o aluno 102 trancou, e sua única matrícula foi apagada. Junto com ela sumiu o registro de que Bruno existe.

Agora a decomposição, uma forma normal por vez.

**1FN** — já está. Toda célula tem um valor só. Se a coluna **curso** guardasse “BD01, ES02” na mesma célula, não estaria.

**2FN** — eliminar a dependência parcial. O nome e o departamento do aluno não têm nada a ver com *curso*; pertencem ao aluno. Sai uma tabela:

```
MATRICULA(aluno, curso, nota)
ALUNO(aluno, nome_aluno, depto, chefe_depto)
```

**3FN** — eliminar a transitiva. Dentro de ALUNO, o chefe não depende do aluno: depende do *departamento*, que por sua vez depende do aluno. Sai mais uma:

```
MATRICULA(aluno, curso, nota)
ALUNO(aluno, nome_aluno, depto)
DEPARTAMENTO(depto, chefe_depto)
```

Cada fato passou a morar num lugar só, e as três anomalias sumiram junto: trocar o chefe virou um UPDATE numa linha; cadastrar departamento novo não exige inventar aluno; apagar a última matrícula não apaga o aluno.

**Achar a chave a partir das dependências** é o formato em que a FGV costuma cobrar isso (“considere  $R(A, B, C, D, E)$  e  $F = \{A \rightarrow B, BC \rightarrow DE, \dots\}$ ; a forma normal mais alta é”). O procedimento é mecânico: parta dos atributos que **nunca aparecem do lado direito** de nenhuma dependência — eles obrigatoriamente fazem parte de toda chave candidata. Feche esse conjunto aplicando as dependências até parar de crescer; se ele alcançar todos os atributos, é chave. Só então classifique cada dependência restante como parcial, transitiva ou fora de chave candidata — e a forma normal mais alta é a última que *nenhuma* dependência viola.

## 6.4 SQL: DDL × DML × DCL × TCL

A divisão não é de nomenclatura: é de **o que cada grupo faz com a transação**. DDL muda a *estrutura* e, na maioria dos SGBD, confirma sozinha (*auto-commit*) — não se desfaz um DROP

TABLE com ROLLBACK. DML muda os *dados* dentro de uma transação, e por isso é desfazível. É essa diferença, e não a semelhança do efeito, que separa TRUNCATE (DDL) de DELETE (DML).

| Sublinguagem      | Comandos                       | Observação                               |
|-------------------|--------------------------------|------------------------------------------|
| DDL (definição)   | CREATE, ALTER, DROP, TRUNCATE  | TRUNCATE é DDL (não DML), auto-commit    |
| DML (manipulação) | SELECT, INSERT, UPDATE, DELETE | DELETE é DML e pode ter WHERE + rollback |
| DCL (controle)    | GRANT, REVOKE                  | permissões                               |
| TCL (transação)   | COMMIT, ROLLBACK, SAVEPOINT    |                                          |

### ATENÇÃO — Como a FGV arma a pegadinha

**TRUNCATE (DDL) × DELETE (DML)**; UNION remove duplicatas / UNION ALL mantém; WHERE filtra linhas / HAVING filtra grupos após GROUP BY; JOIN sem ON vira produto cartesiano. Ordem de execução lógica: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Leia palavra por palavra os comandos SQL da questão — a FGV troca uma cláusula só (LIMIT/OFFSET, DISTINCT, ILIKE, NOT EXISTS por JOIN comum) e testa leitura de código.

### A ordem lógica de execução, e o que ela explica

Você escreve o SELECT primeiro, mas o banco o resolve quase por último:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Duas consequências que a banca cobra sem avisar que está cobrando isto:

- **O WHERE não enxerga apelido do SELECT.** Em SELECT preco\*2 AS dobro ... WHERE dobro > 100, o WHERE roda antes de dobro existir — é erro de sintaxe. Já o ORDER BY dobro funciona, porque roda depois.
- **WHERE filtra linha, HAVING filtra grupo.** Não é questão de estilo: o WHERE acontece *antes* do GROUP BY, quando grupo nenhum existe ainda; o HAVING acontece depois, e é por isso que ele é o único que pode comparar com COUNT(\*) ou SUM().

Guarde a ordem: ela transforma três pegadinhas diferentes em uma regra só.

#### 6.4.1 VIEW, GRANT e o controle da transação

**VIEW (visão):** uma **tabela virtual** — é a consulta guardada com nome, não os dados copiados. Consultar a visão executa o SELECT que está por trás dela, então o resultado é **sempre atual**. Serve para simplificar consulta complexa e para **restringir acesso** (a visão expõe só as colunas/linhas permitidas, e o GRANT é dado sobre ela em vez da tabela). Visão **simples** costuma aceitar atualização; visão com JOIN, agregação ou DISTINCT, não. A **materialized view** é a exceção que confirma a regra: essa *sim* armazena o resultado em disco e precisa ser atualizada.

**GRANT/REVOKE (DCL):** GRANT SELECT, INSERT ON tabela TO usuario concede; REVOKE retira. Com WITH GRANT OPTION, quem recebeu pode repassar o privilégio a terceiros — detalhe que a banca gosta de cobrar. **GRANT é DCL**, não DDL nem DML.



**SAVEPOINT e ROLLBACK TO (TCL):** o **SAVEPOINT** nome marca um ponto **dentro** da transação; **ROLLBACK TO** nome desfaz só o que veio **depois** daquela marca — e a transação **continua aberta**, sem **COMMIT** nem **ROLLBACK** total. É desfazer parcial.

#### ATENÇÃO — Como a FGV arma a pegadinha

Três trocas prontas: dizer que a **visão armazena os dados** fisicamente (isso é a *materialized view*; a visão comum é virtual); classificar **GRANT** como **DDL** (é **DCL**); e afirmar que o **ROLLBACK TO** **encerra** a transação ou desfaz tudo — ele volta ao *savepoint* e a transação segue viva.

### 6.4.2 Notações do MER e chave surrogada

**Crow's Foot (pé de galinha):** a notação de cardinalidade mais usada em ferramenta. Lê-se **no ponto onde a linha encosta na entidade**, e cada lado traz *dois* símbolos — o de fora é o **máximo**, o de dentro é o **mínimo**:

| Símbolo                     | Leitura                                    |
|-----------------------------|--------------------------------------------|
| Traço ( <i>barra</i> )      | exatamente <b>um</b> (mínimo 1 / máximo 1) |
| Círculo ( <i>o</i> )        | <b>zero</b> — opcional                     |
| Pé de galinha (três riscos) | <b>muitos</b>                              |
| Círculo + pé de galinha     | <b>zero ou muitos</b>                      |
| Traço + pé de galinha       | <b>um ou muitos</b> (obrigatório)          |

Daí a tradução para DDL: o lado com **pé de galinha** é o lado **N**, e é nele que entra a **FK**; o círculo indica participação **opcional** → a FK aceita **NULL**; o traço indica participação **obrigatória** → **NOT NULL**.

**Chave surrogada (substituta/artificial):** chave primária **sem significado de negócio**, gerada pelo sistema (sequência, *identity*, **UUID**), em oposição à **chave natural**, que vem do domínio (CPF, matrícula). Vantagens: é estável (o dado de negócio pode mudar — CPF corrigido, matrícula reemitida), curta e uniforme para *join*. É padrão em **Data Warehouse**, onde ela também permite guardar **versões históricas** da mesma entidade de negócio (dimensão de mudança lenta) — detalhe no Capítulo 7.

#### ATENÇÃO — Como a FGV arma a pegadinha

Chave surrogada **não é** chave estrangeira nem chave composta, e não dispensa a chave natural: o campo de negócio continua na tabela, em geral com restrição **UNIQUE**. Em Crow's Foot, a inversão clássica é ler a cardinalidade **do lado errado** da linha — o símbolo vale para a entidade que ele *toca*.



### 6.4.3 Código no servidor: gatilho × procedimento armazenado

|                | Gatilho (trigger)                                                       | Procedimento armazenado                                     |
|----------------|-------------------------------------------------------------------------|-------------------------------------------------------------|
| Quem dispara   | o <b>próprio SGBD</b> , em resposta a um <b>evento</b> de dados         | a <b>aplicação</b> (ou outro código), por chamada explícita |
| Como se invoca | não se invoca — é automático                                            | CALL/EXECUTE                                                |
| Amarrado a     | uma tabela + evento (INSERT, UPDATE, DELETE)                            | nada; é código nomeado e reutilizável                       |
| Parâmetros     | não recebe                                                              | recebe (e pode retornar)                                    |
| Bom para       | auditoria, log, regra que <b>não pode depender</b> da aplicação lembrar | rotina de negócio chamada sob demanda                       |

#### ATENÇÃO — Como a FGV arma a pegadinha

A troca é sempre a mesma: dizer que o **procedimento** é “acionado automaticamente pelo SGBD a cada UPDATE” (isso é o gatilho) ou que o **gatilho** é “invocado com um CALL antes do UPDATE” (isso é o procedimento). O gatilho **também não substitui** restrição de integridade referencial declarada — são mecanismos diferentes. Palavra-chave do enunciado: “**sem depender de a aplicação lembrar**” → gatilho.

## 6.5 Propriedades ACID (transações)

Uma transação é um conjunto de operações que o banco trata como **uma só**. A transferência bancária é o exemplo canônico porque expõe as quatro letras de uma vez: debitar R\$ 100 da conta A e creditar R\$ 100 na conta B são dois UPDATES, e nenhum estado intermediário entre eles é aceitável.

#### ACID na transferência de R\$ 100 de A para B

- **Atomicidade** — *tudo ou nada*. Se o servidor cair entre o débito e o crédito, o débito é desfeito. Nunca existe “saiu de A e não chegou em B”. Quem entrega: o ROLLBACK e o log de transações.
- **Consistência** — *estado válido antes, estado válido depois*. A soma dos saldos não mudou, e nenhuma restrição declarada (CHECK saldo >= 0, chave estrangeira, UNIQUE) foi violada. Repare: consistência é sobre **as regras do banco**, não sobre concorrência.
- **Isolamento** — *uma transação não enxerga o meio da outra*. Se um extrato rodar no exato instante da transferência, ele não pode ver o dinheiro debitado e ainda não creditado. É a letra mais cobrada porque é a única que tem **graus** — são os quatro níveis da seção seguinte.
- **Durabilidade** — *confirmado é para sempre*. Depois do COMMIT, o efeito sobrevive à queda de energia. Quem entrega: a gravação do log em disco **antes** da confirmação (WAL) — por isso durabilidade e recuperação são o mesmo assunto.

O par que se confunde é **Consistência × Isolamento**: a primeira olha para as **regras**, a segunda olha para as **outras transações**. E cuidado com o falso amigo: o “C” do **CAP** (adiante) é outro conceito — lá ele significa que todas as réplicas mostram o mesmo dado.

## 6.6 Concorrência: níveis de isolamento

O “I” do ACID não é tudo-ou-nada: o padrão SQL define **quatro níveis**, e cada um *admite* certos fenômenos de concorrência. Quanto maior o isolamento, **menor** a concorrência — é um trade-off, não um ganho de graça.

Os três fenômenos:

- **Leitura suja** (*dirty read*): lê dado alterado por outra transação que **ainda não confirmou** (e pode desfazer).
- **Leitura não repetível** (*non-repeatable read*): relê a **mesma linha** e o valor mudou, porque outra transação a atualizou e confirmou no meio.
- **Leitura fantasma** (*phantom read*): relê a **mesma faixa** e aparecem **linhas novas**, inseridas e confirmadas por outra transação.

| Nível            | Leitura suja  | Não repetível | Fantasma      |
|------------------|---------------|---------------|---------------|
| READ UNCOMMITTED | admite        | admite        | admite        |
| READ COMMITTED   | <b>impede</b> | admite        | admite        |
| REPEATABLE READ  | impede        | <b>impede</b> | admite        |
| SERIALIZABLE     | impede        | impede        | <b>impede</b> |

### ATENÇÃO — Como a FGV arma a pegadinha

Duas inversões prontas: dizer que o **READ UNCOMMITTED** é o **mais restritivo** (é o **mais permissivo** — é justamente ele que deixa ler o não confirmado) e dizer que o **REPEATABLE READ** **elimina o fantasma** (elimina a **não repetível**; o fantasma só cai no **SERIALIZABLE**). E o absoluto invertido: “quanto mais alto o isolamento, maior a concorrência” — é o contrário. Marque o par pelo objeto: linha que **muda de valor** = não repetível; linha **nova que aparece** = fantasma.

## 6.7 Relacional × Multidimensional (OLTP × OLAP)

Os dois modelos existem porque as duas cargas de trabalho são opostas. O sistema que registra a venda precisa gravar **uma linha por vez, muitas vezes por segundo**, sem perder nada — normalizar serve a ele, porque cada fato mora num lugar só e a escrita fica barata. O sistema que responde “quanto vendemos por região no último trimestre” precisa **varrer milhões de linhas e somar** — e aí a normalização vira inimiga, porque cada *join* a mais é trabalho a mais numa consulta que já é pesada.

Rodar os dois na mesma base é o erro clássico: o relatório trava a produção. Separar em OLTP e OLAP é a resposta — e é isso que justifica o Data Warehouse existir como cópia desnormalizada, em vez de ser um capricho de arquiteto.

|                 | OLTP (transacional)    | OLAP (analítico)           |
|-----------------|------------------------|----------------------------|
| Uso             | operações do dia a dia | apoio à decisão / análise  |
| Escrita/leitura | muitas escritas curtas | leitura pesada, agregação  |
| Modelo          | relacional normalizado | multidimensional (cubos)   |
| Estrutura       | tabelas                | dimensões e métricas/fatos |

### ATENÇÃO — Como a FGV arma a pegadinha

Pegadinha central do bloco: a FGV troca OLTP↔OLAP. “Cubos, dimensões, agregação, análise” = OLAP; “transação, muitas gravações” = OLTP. Nunca “relacional é usado em OLAP e multidimensional em OLTP”.

## 6.8 NoSQL e novos armazenamentos

- **NoSQL**: alta disponibilidade e **escalabilidade horizontal**; costuma relaxar ACID (modelo **BASE**: Basically Available, Soft state, Eventual consistency). Tipos: **chave-valor** (Redis), **documento** (MongoDB), **colunar** (Cassandra), **grafo** (Neo4j, nós e arestas). Não é “sempre grafo” nem “segue ACID estritamente”; não substitui ERP/CRM transacional por padrão.
- **Teorema CAP**: em sistema distribuído, escolha 2 de 3 — **C**onsistency, **A**vailability, **P**artition tolerance. Com partição (P inevitável em rede), decide-se entre C e A. Cenário de registro oficial/consistência crítica (sentença, transação) → CP.
- **Banco em memória** (in-memory, ex.: Redis): baixa latência.
- **Data Lake × Data Warehouse × Lakehouse**: Lake guarda dado bruto (schema-on-read, estruturado e não estruturado); DW guarda dado tratado e modelado (schema-on-write); Lakehouse combina os dois.

### Por que o CAP obriga a escolher — o cenário do cabo cortado

Duas réplicas do mesmo banco, uma no Rio e outra em Natal. O cabo entre elas cai: é a **partição** (P). Chega uma escrita no Rio. O sistema tem duas saídas, e só duas:

- **Aceitar a escrita**. O Rio responde ao usuário, mas Natal segue com o dado velho — as réplicas divergiram. Você abriu mão de **Consistência** para manter **Availability**: perfil **AP**.
- **Recusar a escrita** (ou travar até o cabo voltar). Ninguém lê dado divergente, mas o serviço ficou indisponível para aquela operação. Você abriu mão de **A** para manter **C**: perfil **CP**.

Não existe terceira saída — é por isso que o teorema é um teorema. O “escolha 2 de 3” é uma simplificação didática: em rede real a partição **não é opcional**, cabo cai. P está sempre dentro, e a escolha verdadeira é sempre **C ou A**. Um banco monolítico num servidor só não é “CA”: ele simplesmente não é distribuído, e o teorema não se aplica a ele.

Como decidir diante do enunciado, pergunte *o que dói mais*. Saldo, sentença, registro oficial, estoque de item único → dado errado é inaceitável → **CP**. Carrinho de compras, curta, feed, telemetria → ficar fora do ar é inaceitável e a divergência se resolve depois → **AP**.

### ATENÇÃO — Como a FGV arma a pegadinha

Absolutos denunciam o distrator: “NoSQL segue ESTRITAMENTE ACID”, “NoSQL usa SEMPRE modelo relacional”, “dados SEMPRE em grafos”, “desnormalização é requisito OBRI-

GATÓRIO”. A FGV adora “sempre/apenas/nunca/estritamente” — quase sempre é a alternativa errada. No teorema CAP, desconfie de cenários que priorizam disponibilidade/velocidade (AP) quando o contexto pede consistência crítica (deveria ser CP).

## 6.9 ETL × ELT

|               | ETL                                  | ELT                                                        |
|---------------|--------------------------------------|------------------------------------------------------------|
| Ordem         | Extrai → <b>Transforma</b> → Carrega | Extrai → Carrega → <b>Transforma</b>                       |
| Transformação | antes de carregar (fora do destino)  | depois, no próprio destino                                 |
| Bom para      | DW tradicional                       | destino com muito poder de processamento (nuvem, big data) |

### ATENÇÃO — Como a FGV arma a pegadinha

A pegadinha é a **posição do T**. ELT aproveita o processamento do destino (bom para grande volume); ETL é melhor quando a transformação ocorre antes/fora. Cuidado com a alternativa que inverte quem transforma antes/depois do carregamento, ou que diz que ELT é melhor para volume pequeno (é o contrário).

## 6.10 Índices e notações

### O que um índice é, e por que ele tem tipos

Um índice é uma **estrutura separada da tabela**, que guarda os valores de uma coluna *já organizados* junto com o endereço da linha correspondente. Ele não muda a tabela: acrescenta um caminho de acesso. A troca é sempre a mesma — **leitura mais rápida, escrita mais lenta** (todo INSERT tem de atualizar o índice também) e mais espaço em disco. Índice não é grátis, e é por isso que “criar índice em tudo” nunca é a resposta certa.

Daí decorrem os tipos: cada um organiza de um jeito, e cada jeito responde a uma pergunta diferente.

- **B+ Tree** — árvore balanceada com os valores *ordenados* nas folhas, encadeadas entre si. Como está ordenado, atende faixa (**BETWEEN**, **>**, **<**), **ORDER BY** e igualdade. É o padrão de todo SGBD e o tipo certo em coluna de **alta cardinalidade** — muitos valores distintos: CPF, data, valor.
- **Hash** — aplica uma função ao valor e vai direto ao balde. Ótimo para **=** e **inútil** para faixa: o hash embaralha a ordem de propósito, então “entre 10 e 20” não quer dizer nada dentro dele.
- **Bitmap** — um vetor de bits por valor distinto. Só compensa em **baixa cardinalidade** (situação ativo/inativo; UF, com 27 valores), porque o número de vetores é o número de valores distintos. Combina vários filtros com operação lógica de bits, o que o torna excelente em consulta analítica — e ruim em tabela com muita escrita concorrente.

As duas perguntas que decidem o tipo: *a consulta pede faixa ou igualdade?* e *quantos valores*

*distintos a coluna tem?*

- **Índices:** **B+ Tree** (padrão, bom para faixas e alta cardinalidade), **bitmap** (baixa cardinalidade, ex.: sexo/status), **hash** (igualdade exata). **Hashing extensível:** hash dinâmico com um **diretório** de ponteiros e *global depth*; quando um bucket enche, ele faz **split** e dobra o diretório se preciso — cresce sem reorganizar tudo.
- **IDEF1X:** notação de modelagem de dados (comum em ferramentas como o erwin). Relacionamento **identificador** = linha **sólida**; entidade dependente = retângulo de **cantos arredondados**.
- Administração física (tablespaces, ARIES/recuperação, backup, otimizador) está no Capítulo 14 (Órfãos).

### O que já caiu nas provas

**Em prova real da FGV:** quase toda a lista deste bloco tem lastro. Relacional × multidimensional (OLTP/OLAP); NoSQL; ETL × ELT (“assinale a **incorreta**”); ETL como ponte para o DW — **Dataprev 2024**. Álgebra relacional (a seleção  $\sigma$  e sua tradução para o WHERE); **VIEW** como tabela virtual; **GRANT** de privilégios; **SAVEPOINT** com ROLLBACK TO e RELEASE; **Crow’s Foot** → DDL, que no mesmo item cobra **N:M por tabela associativa** com chave composta e **integridade referencial**; DROP × TRUNCATE × DELETE; LIKE; MongoDB (relação ↔ coleção, \$size, ETL de NoSQL para relacional); NoSQL de **grafos** para rede de relacionamentos; DAMA-DMBOK e governança de dados — **MPU**. Teorema **CAP** (o perfil CP); desnormalização intencional no DW; Star × Snowflake; **Data Warehouse** × **Data Lake** × **Lakehouse**; **chave surrogada**; LIMIT/OFFSET; DISTINCT; anti-join (Partes sem Audiência); *constraints* (UNIQUE, NOT NULL, CHECK, PK autogerada) — **TJ-RJ**. Entidade **fraca** na notação IDEF1X; integridade referencial; normalização (chave candidata e a forma normal mais alta, a partir das dependências funcionais); **gatilho** × **procedimento** × **função**, em três questões distintas; VIEW para esconder a coluna de salário; NoSQL chave-valor para cache de sessão — **ALERO 2026**.

**E tem mais 34, que só apareceram no fechamento do ITEM 7:** o bloco de **administração física** da **ALERO 2026** vinha classificado como “órfão” por um defeito do importador. São **B+ Tree** (eficiência e fator de ramificação) e **hashing extensível**; **índice clustered**; **ARIES/WAL** e a ordem das três fases; **otimizador** de consultas e estatísticas desatualizadas; **tablespaces**; *connection pooling*; **XML Schema** e XQuery no SGBD; **backup** com RPO/RTO; **Administração de Dados** × **DBA**; **JDBC/ODBC**; **dicionário de dados**; e **sharding**. Com elas, banco de dados salta de 25 para **59** questões reais. O conteúdo de administração física continua sendo ensinado no Capítulo 14.

**Não leia esse 59 como peso de prova. 28 das 59 vêm de uma prova só** — a da **ALERO** para o perfil *Banco de Dados*, que não é o nosso. Nas provas mais próximas de desenvolvimento, banco de dados fica em **30**, atrás de engenharia de software (39). O número grande mede *material de treino disponível*, não a chance de cair: na Dataprev 2024, banco de dados e BI somaram **4** questões. Estude como pilar — é o eixo duplo com engenharia de software —, mas sem inflar.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): **níveis de isolamento** e os fenômenos que cada um admite (leitura suja, não repetível, fantasma) — nenhuma das 432 questões reais menciona READ COMMITTED ou SERIALIZABLE.

Rode `./quiz.py banco-dados` e `./quiz.py bi`.

## 6.11 Alta probabilidade / pesquisa extra

- **SGBD nomeados no edital:** Oracle 19C, MySQL, PostgreSQL, MongoDB, MS-SQL Server 2019 — item **7.1 do Perfil 2**, não do 3. Nenhuma questão deve depender de sintaxe proprietária de um deles; o que pode vir para nós é o **conceito** (SQL ANSI, o par relacional × documento).
- **Modelagem dimensional (Kimball): Star Schema** (uma tabela fato + dimensões desnormalizadas, mais rápido) × **Snowflake** (dimensões normalizadas, menos redundância, mais joins) — detalhe completo no Capítulo 7 (BI).
- **Big Data — os Vs.** O trio clássico (Gartner, 2001) é **Volume, Velocidade e Variedade**. **Veracidade** e **Valor** são **extensões** posteriores (daí os modelos “4V” e “5V”), assim como **Variabilidade**. Saiba qual é o trio original: a pegadinha é trocar um membro do trio por um V de extensão — “volume, velocidade e valor” ou “volume, variedade e veracidade” são as “quase certas”.
- **MongoDB:** query, \$size, coleção/documento (vs. tabela/linha do MySQL); **banco orientado a grafos** tem nós e arestas.

### Como se sair melhor

Memorize lado a lado: OLTP = transacional, normalizado, escrita, linha; OLAP = analítico, dimensional, leitura, cubo/agregação. ETL = transforma ANTES (DW tradicional); ELT = carrega e transforma no destino (nuvem, grande volume). Desnormalização = menos joins, mais espaço, risco de anomalia de atualização — serve à leitura, não à escrita. NoSQL: BASE, alta disponibilidade e escalabilidade horizontal; ruim para ERP/CRM fortemente relacional.

## Capítulo 7

# Business Intelligence (BI)

### Ficha do edital

Conceitos, fundamentos, técnicas e métodos de BI; sistemas de suporte à decisão (SSD) e gestão de conteúdo; arquitetura e aplicações de data warehouse com ETL e OLAP; data warehouse e data mining; visualização (BD individuais e cubos); mapeamento das fontes de dados; arquitetura de BI.

Peso esperado: MÉDIO — costuma cair junto com Banco de Dados.

## 7.1 O que é BI

Conjunto de conceitos e ferramentas para transformar **dado** em **informação** e apoiar a **tomada de decisão**. Pirâmide clássica: dado → informação → conhecimento → inteligência/sabedoria.

A pergunta que fundou o assunto é mais concreta: *por que não rodar o relatório direto no banco do sistema?* Por três motivos que aparecem, um a um, nos enunciados de cenário. Primeiro, **concorrência**: uma consulta que varre cinco anos de vendas segura recursos que o sistema transacional precisa para atender o caixa *agora*. Segundo, **história**: o banco transacional guarda o estado *atual* — ele sabe onde o cliente mora, não onde morava quando comprou. Terceiro, e o mais caro, **integração**: os dados estão em vários sistemas que discordam entre si (a mesma unidade com dois códigos, a mesma data em dois formatos). O BI existe para resolver os três de uma vez, e é isso que o resto do capítulo descreve.

### A arquitetura de BI — o caminho do dado, ponta a ponta

fontes (OLTP, planilhas, externas) → **extração** → *staging* → **transformação** → **DW** (e seus *data marts*) → **OLAP/cubo** → visualização

Cada etapa existe por um motivo, e é o motivo que a banca cobra:

- A **extração** lê das fontes sem incomodá-las mais do que o necessário (janela de carga, leitura incremental do que mudou).
- A **staging area** é o pouso intermediário: dado bruto já fora da fonte, ainda não tratado. Existe para que a fonte seja liberada depressa e para que o tratamento possa ser feito sem voltar a ela.
- A **transformação** é onde a integração acontece — é a etapa cara, não a cópia.
- O **DW** guarda o resultado: integrado, histórico e **não volátil** (não se atualiza um fato passado; acrescenta-se).
- **OLAP e visualização** são **consumo**. Nenhum dos dois transforma dado — quem transforma é o ETL, e essa separação é a resposta de vários itens.

O **data mart** é o recorte do DW por assunto ou departamento (vendas, RH). Ele não é uma cópia menor por economia: é a entrega no vocabulário de quem vai usar.



## 7.2 Data Warehouse (DW)

Repositório central de dados **integrados, históricos e orientados a assunto**, para análise (não para transação). Características de Inmon: **orientado a assunto, integrado, não volátil, variante no tempo**.

- **Data Mart:** recorte departamental do DW.
- **DW × Data Lake:** DW guarda dado **tratado e modelado** (schema-on-write); Lake guarda dado **bruto** estruturado e não estruturado (schema-on-read).
- **Staging area:** área intermediária onde o ETL trata os dados.
- **Lakehouse:** une a governança do DW com a flexibilidade do Lake. Camadas típicas: **Bronze** (bruto) → **Silver** (limpo) → **Gold** (dimensional, pronto para BI). Machine Learning consome o bruto; BI consome o Gold.

### ATENÇÃO — Como a FGV arma a pegadinha

Não diga que o Data Lake “substitui” o DW, nem que ele guarda só dado estruturado. Lakehouse é o que combina os dois — não é sinônimo de nenhum dos dois isoladamente.

### 7.2.1 Inmon × Kimball — por onde se começa

As duas escolas discordam sobre **qual peça se constrói primeiro**, e a discordância tem consequência prática.

#### Top-down × bottom-up

**Inmon (top-down).** Constrói-se primeiro o **DW corporativo**, normalizado (até a 3FN), como fonte única de verdade da organização; os **data marts** dimensionais são *derivados* dele. Integra melhor e resiste melhor à mudança, mas o primeiro relatório demora — a área usuária espera o modelo corporativo ficar de pé.

**Kimball (bottom-up).** Começa-se por um **data mart dimensional** de um processo de negócio (vendas, por exemplo), já em esquema estrela, e vão se acrescentando outros. Entrega valor cedo. A integração não vem do modelo central: vem das **dimensões conformadas** — a mesma dimensão **Produto**, com a mesma chave e o mesmo significado, compartilhada por todos os marts (a *bus architecture*).

**O risco de cada um é o espelho do outro.** Em Inmon, o projeto longo que nunca chega ao usuário; em Kimball, o **silo**: marts que não conversam porque as dimensões não foram conformadas. Por isso “dimensão conformada” não é detalhe de vocabulário — é a peça que faz o bottom-up funcionar.

## 7.3 ETL — a ponte para o DW

**Extract → Transform → Load.** Objetivo principal: extrair de várias fontes, transformar em formato padronizado e consistente, e carregar no DW. **Não é** “criar dashboards” nem “treinar modelo de ML” — isso vem depois, como consumo do dado já carregado. ETL × ELT: ver Capítulo 6 (a posição do T é a mesma pegadinha).

### O que acontece em cada fase — e por que a FGV pergunta isso

O item típico descreve uma tarefa e pergunta **a que fase ela pertence**. Como quase tudo o que é interessante mora no *T*, o critério tem de ser mais fino que “transformar é mudar o dado”.



| Fase             | O que é dela                                                                                                                                                                                                                                                                                                    |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Extract</b>   | ler das fontes (banco, arquivo, API), tipicamente só o que mudou desde a última carga. <b>Sem regra de negócio</b> — ela lê, não julga                                                                                                                                                                          |
| <b>Transform</b> | <b>limpeza</b> (nulos, formatos, valores inválidos), <b>padronização/conformidade</b> (o de-para que faz dois sistemas falarem a mesma língua), <b>deduplicação</b> , <b>junção</b> de fontes diferentes, <b>derivação</b> de medidas calculadas, geração da <b>chave surrogate</b> e aplicação da regra de SCD |
| <b>Load</b>      | gravar no destino, <b>dimensões antes da fato</b> (a fato referencia as chaves das dimensões), carga completa × incremental, índices e agregados no fim                                                                                                                                                         |

**A regra prática:** se a tarefa *decide* alguma coisa sobre o conteúdo — padronizar, deduplicar, casar duas fontes, calcular —, é **Transform**. Ler é Extract; gravar é Load.

**ELT** inverte a ordem por um motivo econômico: quando o destino tem poder de processamento de sobra (nuvem, *lakehouse*), carrega-se o bruto e transforma-se **dentro** dele, com SQL. Não é uma versão moderna do ETL que torne a outra obsoleta — é a mesma decisão de onde gastar o processamento.

## 7.4 OLAP e modelagem dimensional

OLAP (analítico) × OLTP (transacional): ver Capítulo 6.

**Cubo:** dados organizados em **dimensões** (tempo, produto, local) e **métricas/fatos** (vendas, quantidade).

### Modelo relacional × modelo multidimensional

São **duas formas de organizar o mesmo dado**, para finalidades diferentes — e a FGV cobra a comparação diretamente.

|                 | Relacional (OLTP)                                   | Multidimensional (OLAP)                            |
|-----------------|-----------------------------------------------------|----------------------------------------------------|
| Organiza em     | tabelas normalizadas (até 3FN)                      | fatos × dimensões (cubo)                           |
| Otimizado para  | <b>escrita</b> curta e concorrente, sem redundância | <b>leitura</b> analítica, agregação em massa       |
| Redundância     | evitada                                             | <b>aceita de propósito</b> (desnormalização)       |
| Pergunta típica | “qual o saldo do cliente X agora?”                  | “como as vendas evoluíram por região e trimestre?” |

Formas de **armazenar** o cubo: **ROLAP** (o dado fica em tabelas relacionais e o cubo é montado por consulta — escala melhor), **MOLAP** (cubo pré-calculado em estrutura própria — consulta mais rápida, carga mais pesada) e **HOLAP** (híbrido: detalhe no relacional, agregados pré-calculados).

**ATENÇÃO — Como a FGV arma a pegadinha**

Duas trocas frequentes: dizer que o modelo multidimensional serve para **transação** (é o relacional/OLTP) e inverter ROLAP com MOLAP — lembre que o **M** de MOLAP é de *multidimensional*, o cubo pré-calculado. Absoluto a descartar: “ferramentas de BI são incompatíveis com tabelas normalizadas” — não são; a desnormalização é escolha de desempenho, não exigência.

| Operação OLAP      | O que faz                                             |
|--------------------|-------------------------------------------------------|
| Drill-down         | desce ao detalhe (ano → mês → dia)                    |
| Roll-up (drill-up) | sobe à agregação                                      |
| Slice              | fatia <b>uma</b> dimensão (fixa um valor)             |
| Dice               | subcubo — filtro em <b>várias</b> dimensões e valores |
| Pivot (rotate)     | gira a visão                                          |

**Star Schema × Snowflake Schema**

**Star Schema:** uma tabela fato + dimensões **desnormalizadas**. Menos joins, mais rápido para BI — é o padrão recomendado por Kimball.

**Snowflake Schema:** dimensões **normalizadas** em várias tabelas. Mais joins, mais lento, mas economiza espaço e ajuda integridade.

**ATENÇÃO — Como a FGV arma a pegadinha**

Star × Snowflake invertido: a FGV chama de “estrela” a dimensão normalizada em várias tabelas (isso é floco de neve/Snowflake), ou diz que Snowflake é o padrão recomendado por Kimball (é o Star). Também drill-down ↔ roll-up e OLAP ↔ OLTP invertidos.

**Exemplo trabalhado — montando o esquema estrela de vendas.** O projeto dimensional tem uma ordem, e ela é sempre a mesma:

1. **Escolher o processo de negócio:** a venda no varejo.
2. **Declarar o grão:** “**uma linha por item de um pedido**”. É a decisão que governa todo o resto — veja a seção seguinte.
3. **Escolher as dimensões:** tudo o que responde a “por qual recorte eu quero olhar?” — Tempo, Produto, Loja, Cliente.
4. **Escolher as medidas:** o que é *numérico e somável* no grão declarado — quantidade, valor\_unitário, valor\_total, desconto.

O resultado é uma **tabela fato** que contém **só duas coisas**: as **chaves estrangeiras** para as dimensões e as **medidas numéricas**. Nada de texto descritivo — o nome do produto mora na dimensão Produto, não na fato. A única exceção com nome próprio é a **dimensão degenerada**: o `número_do_pedido` fica na fato, porque é um identificador que não tem atributo nenhum para formar dimensão.

Agora a diferença que a banca mede em *joins*. Para responder “vendas do **departamento X** no trimestre”:

|                      |           |                                                                                                                                 |
|----------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Estrela</b>       |           | Produto guarda categoria e departamento como colunas dela mesma (desnormalizada): <b>2 joins</b> — fato → Produto, fato → Tempo |
| <b>Floco de neve</b> | <b>de</b> | Produto → Categoria → Departamento, cada uma em sua tabela: <b>4 joins</b> para a mesma resposta                                |

É daí que sai tudo o mais: menos *joins* → consulta mais rápida → esquema estrela como padrão do BI. O floco de neve troca velocidade por economia de espaço e por integridade da hierarquia — uma escolha legítima, só não a padrão.

#### 7.4.1 Granularidade (o grão da fato)

**Granularidade é o que uma linha da tabela fato representa.** Definir o grão é a primeira decisão do projeto dimensional, e ela é irreversível na prática: tudo o mais (dimensões, medidas, agregações) é escolhido depois dela.

| Grão                                                       | Consequência                                                                                                |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Fino</b> (detalhado) — “uma linha por atendimento”      | <b>amplia</b> as análises possíveis (dá para agregar depois em qualquer nível); volume maior                |
| <b>Grosso</b> (agregado) — “uma linha por unidade por dia” | tabela menor e consulta mais rápida, mas o detalhe está <b>perdido para sempre</b> — não dá para desagregar |

Regra de Kimball: **declare o grão no nível mais atômico que a fonte permitir.** Agregados são construídos **a partir** do detalhe, como tabelas adicionais — nunca no lugar dele.

#### ATENÇÃO — Como a FGV arma a pegadinha

A FGV inverte a consequência: diz que escolher o grão mais detalhado **restringe** as análises (é o contrário — quem restringe é o agregado) ou que a granularidade é definida pela quantidade de dimensões (é definida pelo que **uma linha da fato representa**). Não confunda **grão** com **nível de agregação de uma consulta**: o grão é do modelo, a agregação é do drill-up.

#### 7.4.2 Tipos de fato e dimensão

- **Fato aditivo:** soma em **todas** as dimensões (ex.: vendas).
- **Fato semiaditivo:** **não** soma em todas as dimensões, tipicamente não soma no tempo (ex.: saldo, estoque).
- **Fato não aditivo:** razão/percentual, nunca soma.
- **Dimensão degenerada:** atributo de identificação (ex.: nº do pedido) que fica na tabela fato, sem dimensão própria.

#### ATENÇÃO — Como a FGV arma a pegadinha

A banca troca semiaditivo por aditivo — lembre que semiaditivo tipicamente **não** soma no eixo do tempo (saldo bancário de hoje não é a soma dos saldos anteriores).

### 7.4.3 Dimensões lentamente mutantes (SCD)

Atributo de dimensão muda com o tempo (o cliente troca de cidade, o produto troca de categoria). **SCD** (*Slowly Changing Dimension*) é a técnica que decide o que fazer com o histórico:

| Tipo          | O que faz                                                                                                                     | Efeito no histórico                                                                        |
|---------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Tipo 1</b> | sobrescreve o valor antigo                                                                                                    | histórico <b>perdido</b> ; os fatos passados passam a ser lidos com o valor novo           |
| <b>Tipo 2</b> | insere um novo registro versionado (nova chave <i>surrogate</i> , com <i>data_início/data_fim</i> ou <i>flag</i> de corrente) | <b>preserva o histórico completo</b> — cada fato continua ligado à versão vigente na época |
| <b>Tipo 3</b> | guarda o <b>valor anterior em outra coluna</b>                                                                                | preserva só <b>uma</b> mudança (o “valor anterior”), não a série toda                      |

É o **tipo 2** que responde ao cenário clássico: “as vendas antigas devem continuar associadas à cidade em que o cliente morava na época”. E é ele que justifica a **chave *surrogate*** (substituta) da dimensão: como a mesma chave natural passa a ter várias versões, o fato precisa apontar para uma chave artificial que identifique a **versão**, não a entidade.

#### ATENÇÃO — Como a FGV arma a pegadinha

Inversão de número é a pegadinha inteira aqui: oferecer o **tipo 1** (sobrescrever) para um cenário que pede histórico, ou o **tipo 2** para um cenário de correção de erro de digitação (aí o certo é sobrescrever — tipo 1). O tipo 3 aparece como distrator “quase certo”: preserva histórico, mas só o **valor imediatamente anterior**.

## 7.5 Sistemas de Suporte à Decisão (SSD/DSS)

Apoiam gestores em problemas **estruturados, semiestruturados e não estruturados** — oferecem flexibilidade para vários contextos. Não são “só para problemas estruturados” nem “só não estruturados”.

## 7.6 Mapeamento de fontes de dados (boas práticas)

- **Fazer:** entrevistar stakeholders, analisar sistemas existentes, avaliar **qualidade** e adequação do dado, documentar fontes (governança). Referência: **DAMA-DMBOK**, dados mestres e de referência.
- **NÃO fazer:** coletar **tudo sem discriminação**, incluindo dados inconsistentes/irrelevantes “para maximizar quantidade” — essa é a prática **não recomendada** que a FGV pede como resposta.

## 7.7 Data Mining e visualização

**Data Mining:** descobrir padrões, correlações e conhecimento não trivial em grandes volumes de dados.

| Tarefa               | Tem rótulo?                     | Cenário típico                                                                           |
|----------------------|---------------------------------|------------------------------------------------------------------------------------------|
| Classificação        | <b>sim</b> (supervisionada)     | prever se o cliente vai inadimplir (classes conhecidas)                                  |
| Regressão            | <b>sim</b> (supervisionada)     | prever um <b>valor numérico</b> (faturamento do mês)                                     |
| Clusterização        | <b>não</b> (não supervisionada) | <b>descobrir grupos</b> de clientes com comportamento parecido, <i>sem</i> rótulo prévio |
| Regras de associação | não                             | <b>o que é comprado junto</b> (cesta de compras)                                         |

**Regras de associação:** **suporte** = frequência do conjunto no total de transações; **confiança** = força da implicação  $A \rightarrow B$ . A FGV confunde os dois.

### ATENÇÃO — Como a FGV arma a pegadinha

O par que a banca inverte é **classificação**  $\times$  **clusterização**: se o enunciado diz “**sem rótulos prévios**” ou “descobrir grupos”, é clusterização (não supervisionada); se as classes já existem e o modelo aprende a atribuí-las, é classificação (supervisionada). “Produtos comprados juntos” é **regra de associação**, nunca clusterização.

### 7.7.1 CRISP-DM — o ciclo de um projeto de mineração

|   | Fase                           | O que acontece                                                                       |
|---|--------------------------------|--------------------------------------------------------------------------------------|
| 1 | Entendimento do <b>negócio</b> | objetivos e critérios de sucesso do negócio (é por aqui que começa, não pelos dados) |
| 2 | Entendimento dos <b>dados</b>  | coleta inicial, exploração, qualidade                                                |
| 3 | <b>Preparação</b> dos dados    | limpeza, seleção, integração, formatação                                             |
| 4 | <b>Modelagem</b>               | escolha da técnica, treino, ajuste de parâmetros                                     |
| 5 | <b>Avaliação</b>               | os resultados atendem aos <b>objetivos de negócio</b> ?                              |
| 6 | <b>Implantação</b>             | colocar o modelo em operação, monitorar                                              |

O ciclo é **iterativo**, não uma cascata: volta-se de qualquer fase à anterior, e a seta entre implantação e entendimento do negócio fecha o círculo.

**ATENÇÃO — Como a FGV arma a pegadinha**

Não confunda a **avaliação** (fase 5, contra os objetivos de **negócio**) com a validação técnica do modelo, que é parte da **modelagem** (fase 4). Depois da avaliação vem a **implantação** — se a alternativa disser que depois de avaliar volta-se sempre à preparação, é extrapolação. Outro distrator: começar o ciclo pelo entendimento **dos dados** — começa pelo **negócio**.

**Dashboards e relatórios interativos:** camada de visualização (Power BI — com recurso de *drillthrough* entre páginas —, Qlik, etc.).

**O que já caiu nas provas**

**Em prova real da FGV:** SSD para os três tipos de problema e a prática *não* recomendada no mapeamento de fontes (coletar tudo sem discriminar) — **Dataprev 2024**; Star × Snowflake em cenário de varejo/DW (contagem de *joins*), o que a tabela fato contém (chaves estrangeiras + medidas numéricas), Data Mart como recorte departamental e o que pertence à fase de **Transformação** do ETL — **ALERO 2026**; arquitetura **Lakehouse** e **dice** (filtro em várias dimensões) — **TJ-RJ**; **suporte × confiança** em regras de associação, relacional × multidimensional e *drillthrough* no Power BI — **MPU**. O par ETL × ELT caiu na **Dataprev 2024**, como item de Banco de Dados (Capítulo 6).

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): granularidade (o grão da fato); **SCD tipo 2**; as fases do **CRISP-DM**; fato semiaditivo; as quatro características de Inmon; clusterização × regras de associação; DW × Data Lake; drill-down × roll-up.

Rode `./quiz.py bi`.

## 7.8 Alta probabilidade / pesquisa extra

- **Kimball** (bottom-up, data marts) × **Inmon** (top-down, DW corporativo).
- **Self-service BI** e **storytelling com dados** (aparece no Perfil 1).
- **KPI × métrica:** KPI é indicador-chave atrelado a um objetivo.
- Absolutos a descartar: “ferramentas de BI são incompatíveis com tabelas normalizadas”, “3FN maximiza performance de agregação no DW” — falso; 3FN é para OLTP, não para o DW analítico.

**Como se sair melhor**

Decore o par Kimball: Star = 1 fato + dimensões planas (desnormalizadas) = menos joins = mais rápido para BI. Snowflake = dimensões normalizadas = mais joins = mais lento, porém economiza espaço. OLAP dice = filtro em MÚLTIPLAS dimensões; slice = filtro em UMA. Camadas do Lakehouse: Bronze (bruto) → Silver (limpo) → Gold (dimensional para BI). Gatilho de erro: “sempre”, “obrigatório”, “incompatível”, “3FN no DW” — desconfie.

## Capítulo 8

# Segurança da Informação

### Ficha do edital

Políticas e procedimentos de segurança; ISO/IEC 27001:2022 e 27002:2022; confidencialidade, integridade e disponibilidade; mecanismos de segurança (controle de acesso, OAuth2, SSO); gestão de riscos (ameaça, vulnerabilidade, impacto); SDL e OWASP Top 10; análise estática e dinâmica de código (SAST e DAST). Também: HTTPS, SSL/TLS.

Peso esperado: ALTO.

### 8.1 Tríade CIA (a base de tudo)

A tríade não é uma lista de três palavras bonitas: é o **critério de classificação** de todo o resto do capítulo. Cada controle que você vai estudar — cifra, hash, backup, RBAC, WAF — existe para sustentar um dos três pilares, e a pergunta útil diante de qualquer mecanismo é sempre a mesma: *qual pilar este controle protege?*

- **Confidencialidade:** só quem tem direito acessa/vê (cifra, controle de acesso).
- **Integridade:** dado **íntegro**, não é alterado indevidamente (hash, assinatura).
- **Disponibilidade:** **acessível** quando necessário (redundância, backup).
- Complementos: autenticidade, não repúdio, legalidade.

### Cada controle serve a um pilar — e às vezes contra outro

Amarrar mecanismo a pilar resolve metade das questões do bloco de uma vez:

|                          |                                                                                                            |
|--------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Confidencialidade</b> | cifra (simétrica e assimétrica), controle de acesso, mascaramento, classificação da informação, TDE        |
| <b>Integridade</b>       | hash, assinatura digital, HMAC, controle de versão, trilha de auditoria, restrição de integridade no banco |
| <b>Disponibilidade</b>   | backup, redundância, <i>cluster</i> , balanceador, plano de continuidade (RTO/RPO), proteção contra DDoS   |

E o detalhe que separa quem entendeu de quem decorou: os três **competem entre si**. Aumentar confidencialidade quase sempre custa disponibilidade — segundo fator de autenticação barra o invasor e também barra o usuário legítimo que perdeu o celular; cifrar o backup protege o dado roubado e o torna inútil se a chave se perder. Por isso questão de cenário raramente pede “o mais seguro”: pede o **equilíbrio adequado ao contexto**, e a alternativa que maximiza um pilar ignorando os outros costuma ser a plantada.

**ATENÇÃO — Como a FGV arma a pegadinha**

Mapeie o verbo do cenário: “restringir quem vê” é confidencialidade, não integridade; “ficar no ar” é disponibilidade; “alteração indevida” é integridade. A FGV troca a associação cenário → princípio.

## 8.2 Criptografia

**Assimétrica: a mesma matemática, dois usos opostos**

O par de chaves tem uma propriedade só, e ela é a origem de toda a confusão: **o que uma chave fecha, só a outra abre**. Como as duas direções funcionam, a mesma matemática serve a dois objetivos diferentes — e a **direção escolhida é que decide qual**.

|                   | Para SIGILO                              | Para ASSINATURA                                 |
|-------------------|------------------------------------------|-------------------------------------------------|
| Cifra com         | a <b>pública do destinatário</b>         | a <b>privada do emissor</b>                     |
| Abre com          | a <b>privada do destinatário</b>         | a <b>pública do emissor</b>                     |
| Quem consegue ler | só o destinatário — só ele tem a privada | <b>qualquer um</b> : a pública é pública        |
| Logo, garante     | <b>confidencialidade</b>                 | <b>autenticidade, integridade e não repúdio</b> |

A lógica de cada uma, em uma frase. **Sigilo**: eu quero que *só você* leia, então uso o cadeado que *só você* abre — sua chave pública. **Assinatura**: eu quero provar que *fui eu*, então uso a chave que *só eu* tenho — minha privada; que qualquer um consiga abrir não é defeito, é o ponto: é assim que qualquer um verifica.

Duas notas que a banca cobra. Primeira: assina-se o **hash** do documento, não o documento inteiro — assimétrica é lenta, e o hash tem tamanho fixo. Segunda: na prática o **sigilo** também não usa assimétrica para o volume de dados. O TLS faz o que se chama de *envelope*: usa a assimétrica **só para combinar** uma chave simétrica de sessão, e daí em diante cifra tudo com a simétrica, que é rápida. Por isso “a assimétrica substituiu a simétrica” é falso — elas se combinam.

|            | Simétrica                       | Assimétrica (chave pública) |
|------------|---------------------------------|-----------------------------|
| Chaves     | uma chave secreta compartilhada | par: pública + privada      |
| Velocidade | rápida                          | lenta                       |
| Uso        | cifrar volume de dados          | troca de chave, assinatura  |
| Exemplos   | AES, DES/3DES                   | RSA, ECC                    |

- **Hash** (MD5, SHA-256): resumo de tamanho fixo, **unidirecional**; garante **integridade**, não confidencialidade (não “descriptografa”). MD5 e SHA-1 são considerados fracos.
- **Assinatura digital**: hash do documento cifrado com a **chave privada** do emissor → garante autenticidade, integridade e não repúdio.



- **Certificado digital / ICP-Brasil:** vincula identidade a uma chave pública, emitido por uma AC (Autoridade Certificadora).
- **TDE (Transparent Data Encryption):** cifra os dados **em repouso** (arquivos do banco/disco) de forma transparente à aplicação — protege contra roubo do arquivo físico, não contra acesso lógico autorizado.
- **Esteganografia**  $\neq$  criptografia: **oculta a existência** da mensagem (esconde dado dentro de imagem/áudio); a criptografia oculta o **conteúdo**, não o fato de haver mensagem.

#### ATENÇÃO — Como a FGV arma a pegadinha

Hash não é cifra reversível; confidencialidade vem de cifra (não de hash); assina-se com a chave **privada**, verifica-se com a **pública**; esteganografia esconde a existência, cifra esconde o conteúdo. HMAC (chave simétrica) garante autenticidade e integridade — não confidencialidade.

#### HMAC e o cenário do *webhook*

**HMAC** (*Hash-based Message Authentication Code*) é hash **combinado a uma chave secreta compartilhada**. Como só quem tem a chave produz o código, ele entrega **integridade** (a mensagem não foi alterada) e **autenticidade** (veio de quem tem a chave). O que ele **não** entrega: **confidencialidade** — a mensagem viaja legível — nem **não repúdio**, porque a chave é *simétrica* e as duas pontas conseguem gerar o mesmo código.

**O cenário que a FGV monta:** um sistema externo envia um *webhook* (uma chamada HTTP de notificação) e é preciso garantir que a requisição veio mesmo do parceiro e não foi adulterada no caminho. A resposta é assinar o corpo com **HMAC** usando um segredo combinado entre os dois, e o receptor recalcula e compara. Hash puro não serve (qualquer um recalcula); cifrar o corpo resolveria confidencialidade, que não é o problema.

#### ATENÇÃO — Como a FGV arma a pegadinha

As duas armadilhas do HMAC: vendê-lo como garantia de **confidencialidade** (não é — não há cifra do conteúdo) e como garantia de **não repúdio** (não é — para isso é preciso **assinatura digital** com chave *privada*, que só o emissor possui). Guarde o corte: chave *simétrica*  $\rightarrow$  autenticidade + integridade; chave *privada*  $\rightarrow$  acrescenta o não repúdio.

## 8.3 HTTPS, SSL e TLS

**HTTPS = HTTP sobre TLS/SSL.** Garante confidencialidade + integridade + autenticação do servidor.

**TLS substitui o SSL:** corrige vulnerabilidades das versões antigas do SSL. SSL está obsoleto; TLS 1.2/1.3 é o atual.

#### ATENÇÃO — Como a FGV arma a pegadinha

Nunca “SSL é mais seguro/moderno que TLS” (é o contrário: TLS sucedeu e corrigiu o SSL) nem tratar os dois como intercambiáveis.

## 8.4 Controle de acesso

| Política             | Base da decisão                                                                              |
|----------------------|----------------------------------------------------------------------------------------------|
| DAC (discricionário) | dono do recurso concede acesso                                                               |
| MAC (mandatário)     | <b>rótulos de segurança</b> comparados com autorizações; regra central, o usuário não decide |
| RBAC (por papéis)    | acesso via papéis/funções                                                                    |
| ABAC (por atributos) | atributos de usuário/recurso/contexto                                                        |

**Menor privilégio:** só o acesso necessário (ex.: gerente de RH acessa só o que precisa).

**OAuth2:** protocolo de **autorização** delegada (tokens); **diferente** de autenticação. **OpenID Connect (OIDC)** (sobre OAuth2) é quem faz autenticação — claims do ID Token: **iat** (issued at, emissão), **exp** (expiração), **sub** (subject, identificador do usuário), **jti** (id do token).

**SSO (Single Sign-On):** um login para vários sistemas.

### ATENÇÃO — Como a FGV arma a pegadinha

MAC = rótulos e classificações impostos pelo sistema (o usuário *não* decide); DAC = o proprietário concede; RBAC = permissões por função/cargo. A FGV descreve rótulos comparados a autorizações (isso é MAC) e oferece “discricionário” como pegadinha. OAuth2 é autorização, **não** autenticação. Nos claims do OIDC, a banca pede um e oferece os outros — decore os quatro.

## 8.5 OWASP Top 10 — 2025 (vigente) e 2021 (leia os dois)

O edital aponta para o projeto OWASP sem fixar ano, e hoje existem duas numerações válidas. A **edição vigente é a 2025** — oitava da série, com *release candidate* apresentado em novembro de 2025 e versão final publicada em janeiro de 2026. A **2021** é a que a Dataprev 2024 cobrou e a que ancora as questões deste material. Saber as duas cobre qualquer recorte que a prova use; e, ao ler o enunciado, **verifique qual ano ele cita** antes de contar posição.

| #  | Top 10:2025 (vigente)                                                    | Top 10:2021 (Dataprev 2024)                     |
|----|--------------------------------------------------------------------------|-------------------------------------------------|
| 1  | Broken Access Control (absorveu o SSRF)                                  | Broken Access Control                           |
| 2  | Security Misconfiguration (subiu de 5º)                                  | Cryptographic Failures                          |
| 3  | Software Supply Chain Failures (novo; expande “componentes vulneráveis”) | Injection (SQLi; <b>XSS absorvido aqui</b> )    |
| 4  | Cryptographic Failures (caiu de 2º)                                      | Insecure Design (novo em 2021)                  |
| 5  | Injection (caiu de 3º)                                                   | Security Misconfiguration                       |
| 6  | Insecure Design                                                          | Vulnerable and Outdated Components              |
| 7  | Authentication Failures                                                  | Identification and Authentication Failures      |
| 8  | Software <i>or</i> Data Integrity Failures                               | Software <i>and</i> Data Integrity Failures     |
| 9  | Security Logging and <b>Alerting</b> Failures                            | Security Logging and <b>Monitoring</b> Failures |
| 10 | Mishandling of Exceptional Conditions (novo)                             | Server-Side Request Forgery (SSRF)              |

O que mudou de 2021 para 2025: **duas categorias novas** (A03 Software Supply Chain Failures e A10 Mishandling of Exceptional Conditions) e **uma consolidação** (o SSRF, que era A10 em 2021, foi absorvido pelo Broken Access Control). Três categorias só **mudaram de nome**: A07 perdeu o “Identification and”; A08 trocou o “and” pelo “or”; A09 trocou **Monitoring** por **Alerting**.

#### ATENÇÃO — Como a FGV arma a pegadinha

A Dataprev 2024 descreveu “injeção” citando SQL → categoria **Injection**; lembre que desde 2021 **XSS entrou em Injection** (era categoria própria até 2017). A banca mistura itens que **não** são da lista OWASP web — “proteção da cadeia de suprimentos”, “ambiente de engenharia”, “treinamento operacional” são de CI/CD e SSDF, não OWASP (atenção: em 2025 a cadeia de suprimentos *de software* virou categoria própria, a A03 — o que não torna “proteção da cadeia de suprimentos” item de 2021). As renomeações são material de pegadinha pronta: oferecer “Security Logging and **Monitoring**” num item que diz 2025, ou “Authentication Failures” num item que diz 2021.

## 8.6 Desenvolvimento seguro

- **SDL (Security Development Lifecycle)**: segurança em todo o ciclo.
- **SAST (estática)**: analisa o **código-fonte** sem executar (caixa-branca).
- **DAST (dinâmica)**: testa a **aplicação em execução** (caixa-preta).
- **IAST**: combina os dois em runtime instrumentado.
- **DevSecOps**: segurança automatizada no pipeline CI/CD.

**ATENÇÃO — Como a FGV arma a pegadinha**

**SAST** × **DAST**: código parado × app rodando. Não inverta.

## 8.7 X.800 — a arquitetura de segurança OSI

Recomendação da ITU-T que dá o **vocabulário** de segurança usado por normas e por bancas. Divide o assunto em três: *ataques*, *serviços* e *mecanismos*. O que a FGV cobra é a divisão dos **mecanismos**.

**Os cinco serviços de segurança:** autenticação, controle de acesso, confidencialidade, integridade e **não repúdio** (a disponibilidade aparece como categoria adicional).

**Mecanismos — e o corte que a banca pede:**

|                                                                        |                                                                                                                                                                                                                                |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Específicos</b><br>(ligados a uma camada e a um serviço)            | cifração ( <i>encipherment</i> ), assinatura digital, controle de acesso, integridade de dados, troca de autenticação, <b>preenchimento de tráfego</b> ( <i>traffic padding</i> ), controle de roteamento e <b>notarização</b> |
| <b>Disseminados</b><br>( <i>pervasive</i> , não específicos de camada) | funcionalidade confiável, <b>rótulos de segurança</b> , detecção de eventos, <b>trilha de auditoria</b> de segurança e recuperação de segurança                                                                                |

**ATENÇÃO — Como a FGV arma a pegadinha**

A troca é sempre entre as duas colunas: oferecer **trilha de auditoria** ou **rótulo de segurança** como mecanismo “específico” (são **disseminados**), ou **cifração/notarização** como “disseminado” (são **específicos**). O critério é: mecanismo específico implementa *um serviço* numa *camada*; disseminado é de gestão, vale para o sistema inteiro. Note que **preenchimento de tráfego** e **controle de roteamento** costumam surpreender — são específicos, e servem à confidencialidade do *fluxo*, não do conteúdo.

## 8.8 Gestão de riscos: ameaça, vulnerabilidade e impacto

O edital pede os três termos, e eles só fazem sentido juntos. Um risco existe quando **uma ameaça encontra uma vulnerabilidade** e disso resulta um **impacto**. Tirar qualquer um dos três e o risco desaparece — é essa composição que explica por que se trata risco de várias formas diferentes.

**Os três termos, e o que cada um permite fazer**

|                        |                                                                                                                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ameaça</b>          | o evento indesejado <i>em potencial</i> — incêndio, invasor, funcionário descuidado, <i>ransomware</i> . Vem de fora do seu controle: você <b>não elimina</b> ameaça.              |
| <b>Vulnerabilidade</b> | a fraqueza que permite a ameaça se concretizar — servidor sem <i>patch</i> , senha fraca, ausência de <i>backup</i> . É o <b>único dos três</b> sobre o qual você age diretamente. |
| <b>Impacto</b>         | o dano se acontecer. Você não impede o impacto, mas pode <b>reduzi-lo</b> (segmentar rede, cifrar o dado roubado) ou <b>transferi-lo</b> (seguro).                                 |

O **risco** é a combinação de **probabilidade**  $\times$  **impacto** — é isso que a matriz 5 $\times$ 5 desenha: probabilidade num eixo, impacto no outro, e a cor da célula diz a prioridade. Note a consequência: um evento catastrófico e improvável pode ficar *abaixo* de um evento pequeno e frequente. A banca gosta de cenários em que a intuição diz “o incêndio é pior” e a matriz diz outra coisa.

As quatro formas de tratar (ISO 27005/31000) — e o verbo do enunciado entrega qual é:

|                                  |                                                                                                                                                                                   |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mitigar</b> (reduzir)         | implanta controle e <b>diminui</b> probabilidade ou impacto. É o caso mais comum: aplicar patch, pôr WAF, treinar equipe.                                                         |
| <b>Transferir</b> (compartilhar) | passa a consequência financeira a um terceiro — <b>seguro</b> , terceirização com cláusula de responsabilidade. Repare: transferir <b>não reduz</b> a probabilidade de acontecer. |
| <b>Evitar</b>                    | <b>deixa de fazer</b> a atividade que gera o risco — descontinuar o sistema legado, não coletar o dado sensível.                                                                  |
| <b>Aceitar</b> (reter)           | decide conviver, <b>formalmente</b> e no nível certo da hierarquia, porque o controle custa mais que o dano.                                                                      |

**Risco residual** é o que **sobra depois** do tratamento. Ele nunca é zero, e a norma exige que seja **aceito formalmente** por quem tem alçada — aceitar risco residual é decisão de gestão, não do time técnico.

#### ATENÇÃO — Como a FGV arma a pegadinha

Três trocas prontas. (i) Dizer que **transferir o risco elimina o risco**: o seguro paga o prejuízo, mas o incidente acontece do mesmo jeito — quem reduz a chance é o *mitigar*. (ii) Confundir **aceitar** com **ignorar**: aceitar é ato formal, documentado e aprovado; deixar sem tratamento por omissão não é uma das quatro opções. (iii) Afirmar que o **risco residual deve ser zero** — se pudesse ser zerado, não se chamaria residual; o que a norma exige é que ele seja conhecido e aceito. Cuidado também com a inversão ameaça $\leftrightarrow$ vulnerabilidade: o *ransomware* é a ameaça, o servidor sem patch é a vulnerabilidade.

## 8.9 Continuidade de negócio: RTO $\times$ RPO

Os dois objetivos que o plano de continuidade fixa para cada serviço crítico. Guardar a distinção pela **unidade do que se perde**:

| Sigla      | Nome                            | O que limita                                                                  |
|------------|---------------------------------|-------------------------------------------------------------------------------|
| <b>RTO</b> | Recovery <b>Time</b> Objective  | <b>TEMPO</b> : quanto o serviço pode ficar indisponível até ser restabelecido |
| <b>RPO</b> | Recovery <b>Point</b> Objective | <b>DADO</b> : até que ponto no passado se aceita perder informação            |

É o **RPO** que determina a **frequência do backup**: aceitar perder no máximo 15 minutos de dado obriga a proteger os dados a cada 15 minutos. Já o RTO cobra da infraestrutura de recuperação (redundância, sítio alternativo, tempo de restauração).

As siglas vizinhas que a banca oferece junto:

- **MTBF** (Mean Time Between Failures): tempo **médio entre** falhas sucessivas — métrica de *confiabilidade*, não objetivo de plano.
- **MTTR** (Mean Time To Repair): tempo **médio de reparo**.
- **SLA** (Service Level Agreement): o **acordo** de nível de serviço — compromisso contratual, não a métrica em si.

#### ATENÇÃO — Como a FGV arma a pegadinha

O cenário dá dois números na ordem “pode ficar fora X, pode perder Y” e a alternativa **inverte as siglas** (RPO de 4 horas, RTO de 15 minutos). Ancore em uma letra: **T** de RTO é **Tempo**; **P** de RPO é **Ponto** no tempo (*dado*). Os distratores de MTBF/MTTR/SLA descrevem médias observadas e compromisso contratual — nenhum é objetivo definido por diretoria em plano de continuidade. Backup incremental × diferencial está no Cap. 14.

## 8.10 Detecção e resposta

**IDS** (detecção): **alerta** sobre intrusão, passivo. **IPS** (prevenção): **bloqueia** ativamente (inline). **SIEM**: correlaciona logs/eventos para detecção e resposta.

**Resposta a incidentes (NIST SP 800-61) — a ordem cai:**

1. **Preparação** (antes de tudo: time, ferramentas, políticas).
2. **Detecção e Análise** (identificar e entender o incidente).
3. **Contenção, Erradicação e Recuperação** (isolar, remover a causa, restaurar serviços/backups).
4. **Atividade pós-incidente / Lições Aprendidas** (documentar, prevenir recorrência — realimenta a Preparação).

#### ATENÇÃO — Como a FGV arma a pegadinha

Depois de conter e erradicar vem **recuperação + lições aprendidas** (a etapa que “fecha” o ciclo); detecção e análise vêm **antes** da contenção — a FGV embaralha essa ordem.

#### O que já caiu nas provas

**Em prova real da FGV:** 38 questões, e a lista abaixo é quase toda verdadeira. Cuidado com esse total: **17 das 38** saíram de uma prova só, a da ALERO para o perfil *Redes*, e puxam o bloco para o lado de rede (*malware*, DDoS, VPN, proxy, NGFW). Nas provas mais próximas de desenvolvimento, segurança fica em **19** — na Dataprev 2024 foram **4**. Controle de acesso **mandatório** (comparação de rótulos); **OWASP Top 10:2021**, identificar a categoria válida (gabarito: SSRF, o A10 — as demais alternativas descreviam *práticas*, não vulnerabilidades); **X.800** (mecanismo específico × disseminado); **SSL × TLS — Dataprev 2024**. Tríade **CIA** em associação de colunas; modelo de **responsabilidade compartilhada** (PaaS); **HMAC** em webhook; o *claim* sub do OIDC para rastreabilidade; aplicabilidade da LGPD a empresa privada — **TJ-RJ**. Anonimização como tratamento que rompe a associação ao titular; tipo de controle na ISO 27001; o *claim* iat do OIDC — **MPU**. **RTO × RPO** saindo de uma BIA; **IDS × IPS** (o que *age*, não só detecta); SSL/TLS; responsabilidade compartilhada em SaaS; **RBAC**; segregação de funções; proprietário do ativo na 27002; tratamento de risco e risco residual; matriz 5×5 com WAF; criptografia simétrica × assinatura digital sobre ICP; a família de *malware* inteira (*worm*, *ransomware*, *spyware* + *keylogger*, *phishing*); DDoS e SYN *flood*; *spoofing* de MAC/IP; NGFW, WAF, proxy e VPN; e as etapas de **resposta a incidentes** (contenção, depois recuperação e lições aprendidas) — **ALERO 2026**, que sozinha respondeu

por 17 questões de segurança.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): **SAST** × **DAST** — nenhuma das 432 questões reais cita qualquer um dos dois. Continua valendo o estudo: é par de edital e a FGV adora inverter estático com dinâmico.

Rode `./quiz.py segurança`.

## 8.11 Alta probabilidade / pesquisa extra

- **ISO/IEC 27002:2022: 93 controles** em **4 temas** — organizacionais (37), pessoas (8), físicos (14), tecnológicos (34). A 27001:2022 é o **SGSI** (requisitos, Anexo A); a 27002 é o **guia de controles**. Trabalho remoto = pessoas; mídia de armazenamento = física; criptografia = tecnológica; políticas = organizacional.
- **NIST Cybersecurity Framework 1.1**: funções Identify, Protect, Detect, Respond, Recover (o 2.0 acrescentou **Govern**).
- **Gestão de risco (ISO 27005/31000)**: risco =  $f(\text{ameaça}, \text{vulnerabilidade}, \text{impacto})$ ; tratamento: mitigar, transferir, aceitar, evitar.
- **STRIDE** (modelagem de ameaças): Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.

### Como se sair melhor

CIA: Confidencialidade = quem VÊ; Integridade = dado ÍNTEGRO; Disponibilidade = ACES-SÍVEL. Controle de acesso, decore a frase-chave: MAC = rótulos impostos pelo sistema; DAC = o proprietário concede; RBAC = permissões por função/cargo. OWASP Top 10:2021, âncoras seguras: A01 Broken Access Control, A03 Injection, A10 SSRF. Se a opção falar em “cadeia de suprimentos/pipeline/treinamento”, é distrator de outro framework. Gatilho geral: “sempre isento”, “SSL mais seguro”, claim trocado — releia com calma.



## Capítulo 9

# Programação

### Ficha do edital

Desenvolvimento em Java/JavaEE/JakartaEE/JPA/JavaScript; frameworks JUnit, Hibernate, JSF, Primefaces, Spring, Spring Cloud, Spring Boot; mobile (Android/iOS); low-code/no-code; análise estática (clean code, SonarQube); padrões XML, XSLT, UDDI, REST, JSON; DevOps; Git; codificação (transacional, analítico, mobile, API); reuso; blockchain. (Java em si tem capítulo próprio: Capítulo 10.)

Peso esperado: MÉDIO-ALTO.

### 9.1 Ecossistema Spring (cai muito)

Antes do Spring, uma classe de negócio construía as próprias dependências com **new**: o serviço de pedidos decidia, dentro do seu código, qual repositório usar. Isso amarrava três coisas de uma vez — trocar a implementação exigia editar e recompilar quem a usava, testar exigia o banco de verdade (não havia onde encaixar um dublê), e a configuração ficava espalhada por todas as classes. O ecossistema abaixo existe para desfazer esse nó, e entender o nó é o que faz a tabela parar de ser decoreba.

| Framework    | Papel                                                                                |
|--------------|--------------------------------------------------------------------------------------|
| Spring       | contêiner de inversão de controle (IoC) e injeção de dependência                     |
| Spring Boot  | Spring com autoconfiguração e servidor embutido; elimina config manual e boilerplate |
| Spring Cloud | ferramentas para sistemas distribuídos/microserviços (config, discovery, gateway)    |
| Hibernate    | ORM (mapeamento objeto-relacional), implementa JPA                                   |
| JUnit        | framework de testes de unidade                                                       |

### Inversão de controle e injeção de dependência — o que o contêiner realmente faz

**Inversão de controle (IoC)** é isto: quem decide *qual* implementação usar e *quando* construí-la deixa de ser a classe e passa a ser o contêiner. A classe apenas **declara** do que precisa; recebe pronto de fora. O controle foi invertido — daí o nome.

**Injeção de dependência (DI)** é a forma concreta de fazer IoC: entregar a dependência por **construtor** (preferida — a dependência é obrigatória e o objeto já nasce válido), por *setter* ou por campo anotado. Não são conceitos rivais: DI é o mecanismo, IoC é o princípio.

O ganho é duplo e é o que a banca cobra em cenário: **trocar a implementação** sem tocar em



quem a usa, e **testar** injetando um dublê no lugar do recurso real. É o *Dependency Inversion* — o **D** do SOLID (Capítulo 4) — executado por um contêiner.

**Detalhe que separa quem entendeu:** o objeto gerenciado pelo contêiner chama-se **bean**, e o escopo padrão do Spring é *singleton* — **uma instância por definição de bean, dentro daquele contêiner**. Isso *não* é o padrão Singleton do GoF, que faz a própria classe impedir a segunda instância. Mesmo nome, garantias diferentes.

#### ATENÇÃO — Como a FGV arma a pegadinha

Spring Boot **não exige** config manual de servidor (embute Tomcat) e não exige Tomcat/JBoss standalone; “Spring só serve monólito” é falso. Hibernate é **ORM/persistência**, não é framework de teste. JUnit é teste, **não** é ORM. Spring Cloud = distribuído; Spring Boot = app individual — a FGV troca os papéis do ecossistema numa frase só.

## 9.2 Formatos de dados: XML, JSON, XSLT

|           | XML                        | JSON                              |
|-----------|----------------------------|-----------------------------------|
| Estrutura | marcação com tags, verboso | pares chave-valor, leve           |
| Uso hoje  | legado, config, SOAP       | <b>APIs web</b> , transporte leve |

**XSLT:** linguagem que **transforma XML** em outro formato (HTML, XML, texto). **Não** transforma JSON. **REST** usa JSON tipicamente; **SOAP** usa XML.

#### ATENÇÃO — Como a FGV arma a pegadinha

“XSLT transforma JSON” = falso (é XML); “XML e JSON são equivalentes/idênticos” = falso; JSON não tem comentários nem tipo date nativo.

## 9.3 Mobile e low-code

| Abordagem       | O que é                                  | Ferramentas / linguagem                                                        |
|-----------------|------------------------------------------|--------------------------------------------------------------------------------|
| Nativo          | app específico para cada SO              | Android: <b>Kotlin/Java</b> ;<br>iOS: <b>Swift/Objective-C</b>                 |
| Multiplataforma | um código para vários SO                 | <b>Flutter (Dart)</b> , <b>React Native (JS)</b> , Xamarin (.NET), Ionic (web) |
| Híbrido         | web dentro de contêiner nativo (WebView) | Ionic, Cordova                                                                 |
| PWA             | web instalável, offline                  | ver Capítulo 11                                                                |

**Flutter** = **Dart** (a FGV troca por JS/Kotlin); **React Native** = **JavaScript** (renderiza componentes nativos, diferente do WebView do híbrido). Trade-off: nativo dá melhor desempenho/acesso

ao hardware; cross reduz custo e tempo (um código só).

**Low-code/no-code:** desenvolvimento visual, pouca ou nenhuma escrita de código; acelera entrega e permite “citizen developers”, mas limita customização e pode gerar *lock-in* na plataforma.

#### ATENÇÃO — Como a FGV arma a pegadinha

Pares mobile que a FGV inverte: Dart→Flutter, Kotlin→Android, Swift→iOS. Distrator de definição decorada: pede o framework Dart e oferece React Native (JS), Xamarin (C#) ou Ionic — nenhum é Dart.

## 9.4 Java EE / web server-side (JSF, Primefaces)

**JSF (JavaServer Faces):** framework de UI baseado em componentes, server-side, orientado a eventos; segue o padrão **MVC**. Ciclo de vida de requisição com fases (restore view, apply values, validation, update model, invoke application, render response).

**Primefaces:** biblioteca de componentes ricos para JSF (tabelas, gráficos, ajax pronto).

**JSP/Servlet:** camada web mais antiga do Java EE (Servlet processa a requisição; JSP gera a view). JSF é a evolução baseada em componentes.

#### ATENÇÃO — Como a FGV arma a pegadinha

JSF é **server-side baseado em componentes** (diferente de SPA client-side como React); Primefaces é biblioteca de componentes **de JSF**, não framework à parte.

## 9.5 Versionamento com Git e DevOps

**Git é distribuído:** cada clone tem o histórico completo (diferente do SVN, que é centralizado e lida mal com binários). GitLab CI usa variáveis e pipelines.

#### As três áreas — onde mora quase toda pegadinha de Git

diretório de trabalho  $\xrightarrow{\text{git add}}$  stage (index)  $\xrightarrow{\text{git commit}}$  repositório local  $\xrightarrow{\text{git push}}$  remoto

O commit grava **o que está no stage**: editar um arquivo já versionado e commitar **sem git add** não inclui a alteração — o commit sai vazio daquela mudança, sem erro nenhum.

| Comando                       | O que faz                                                                  |
|-------------------------------|----------------------------------------------------------------------------|
| <code>git clone</code>        | cria a cópia local inicial de um repositório remoto                        |
| <code>git add</code>          | leva a alteração para o stage                                              |
| <code>git commit</code>       | grava no repositório local o que está no stage                             |
| <code>git fetch</code>        | traz referências e objetos do remoto <b>sem</b> tocar na cópia de trabalho |
| <code>git pull</code>         | <b>fetch</b> + integração automática (merge ou rebase)                     |
| <code>git push</code>         | envia commits locais ao remoto                                             |
| <code>git merge</code>        | une duas linhas criando um <b>commit de junção</b>                         |
| <code>git rebase</code>       | <b>reaplica</b> os commits sobre outra base; histórico linear              |
| <code>git revert</code>       | cria um commit novo que <b>anula</b> outro (seguro em branch pública)      |
| <code>git reset --hard</code> | move a branch e <b>descarta</b> alterações locais                          |
| <code>git cherry-pick</code>  | copia <b>commits escolhidos</b> para a branch atual                        |

**Regra de ouro do rebase:** ele reescreve commits (novos identificadores). Não use em branch já compartilhada — é a razão de a banca chamá-lo de “perigoso”.

**Fluxos:** branch por funcionalidade + *pull request*; **Git Flow** (main, develop, feature/\*, release/\*, hotfix/\*); **trunk-based** (integrações curtas e frequentes no tronco, casa melhor com integração contínua).

### 9.5.1 CI, CD e CD — o trio que a FGV mais troca

| Prática                                           | Onde termina                                                                             |
|---------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Integração contínua</b> (CI)                   | integra e <b>testa</b> a cada push                                                       |
| <b>Entrega contínua</b> ( <i>delivery</i> )       | artefato <b>sempre pronto</b> para produção; a subida depende de <b>aprovação manual</b> |
| <b>Implantação contínua</b> ( <i>deployment</i> ) | o artefato aprovado vai a produção <b>automaticamente</b>                                |

Entrega contínua **pressupõe** integração contínua. **DevOps:** cultura de aproximar desenvolvimento e operação (automação, medição, responsabilidade compartilhada pelo que está em produção). **DevSecOps:** segurança deslocada para a **esquerda** (*shift left*), no pipeline desde o início — não como auditoria final.

### 9.5.2 Contêineres

**Docker** constrói (a partir do **Dockerfile**) e executa a **imagem**, que empacota aplicação + dependências. **Docker Compose** orquestra vários contêineres em **um host** (tipicamente desenvolvimento). **Kubernetes** orquestra em **cluster**: mantém réplicas, reinicia o que falha, distribui carga, faz *rolling update* — detalhes de arquitetura no Capítulo 5.

**ATENÇÃO — Como a FGV arma a pegadinha**

Inversão de par é o padrão do tema: `merge↔rebase`, `fetch↔pull`, `delivery↔deployment`, `Docker↔Kubernetes`. Se o enunciado disser “**sem alterar a cópia de trabalho**”, é `fetch`; se disser que a subida a produção **espera aprovação humana**, é `delivery`, não `deployment`. Outro desenho: a alternativa descreve corretamente o efeito de um comando, mas dá o **nome de outro** — leia a justificativa antes de aceitar o nome. Absoluto plantado: “o `pull` nunca altera arquivos”.

## 9.6 Qualidade de código

**Clean Code:** nomes claros, funções curtas, baixo acoplamento.

**SonarQube:** análise **estática** (sem executar) — code smells, bugs, vulnerabilidades, dívida técnica, cobertura.

**O que este bloco cobra e mora em outro capítulo**

O edital de Programação puxa conteúdo que está desenvolvido fora daqui — se você errou uma questão deste bloco sobre um destes temas, é para lá que deve ir:

- **SOLID** (SRP, OCP, Liskov, ISP, DIP) e **orientação a objetos** (herança, polimorfismo, sobrescrita): Capítulo 10.
- **Os 23 padrões GoF** (Singleton, Adapter, Observer, Strategy, Factory Method...) e o par **alta coesão × baixo acoplamento**: Capítulo 4.
- **REST na prática** — métodos HTTP (GET, POST, **PUT** substitui inteiro, **PATCH** parcial, DELETE), códigos de resposta (200, **201 Created**, 204, 400, 401, 403, 404, 500) e **idempotência** (GET, PUT e DELETE repetidos deixam o servidor no mesmo estado; POST, não): Capítulo 5.
- **Reuso, testes e ciclo de vida**: Capítulo 3.

## 9.7 Blockchain

**Cadeia de blocos** encadeados por **hash do bloco anterior** (imutável). Cada bloco: hash anterior, timestamp, dados/transações, nonce, raiz de Merkle. **O saldo das carteiras NÃO é armazenado no bloco** — é derivado do histórico de transações (UTXO no Bitcoin). Descentralizado, consenso (PoW/PoS), contratos inteligentes (Ethereum).

**ATENÇÃO — Como a FGV arma a pegadinha**

Pergunta clássica: “o que **NÃO** é armazenado no bloco” → resposta: **saldo das carteiras** (é calculado a partir do histórico, não guardado diretamente).

## 9.8 Python aplicado a dados (cobrado no MPU)

- **NumPy:** `view()` compartilha buffer (`.base` aponta para o original) × `copy()` cria cópia independente (`.base = None`). Alterar antes do `view()` reflete no view. Shape de vetor 1-D termina em vírgula: `(3,)`, não `(3,1)`.
- **matplotlib.pyplot:** parâmetro `explode` destaca fatia da pizza.

**Dicionário:** `max(dic)` compara as **chaves**; `max(dic, key=dic.get)` compara os **valores** e devolve a **chave** vencedora — nunca o valor. O `key=` de `max/min/sorted` diz por onde **comparar**, jamais o que devolver.

**Listas:** o fatiamento `lista[a:b]` inclui **a** e **exclui b** — `nums[1:4]` devolve **três** elementos (conte **b** – **a**). Índice negativo conta do fim (–1 é o último). Compreensão e expressão geradora com filtro: `sum(n for n in nums if n % 2 == 0)` soma só os que passam no filtro.

#### ATENÇÃO — Como a FGV arma a pegadinha

NumPy: a FGV inverte `view/copy` — diz que `copy` compartilha base ou que `view` tem base `None`; ou erra o shape do array 1-D. No `max` com `key=`, oferece a alternativa com a **chave certa e o valor errado** (ou devolve o valor máximo no lugar da chave). No fatiamento, monta a alternativa que **inclui** o índice final — é um elemento a mais, e passa despercebido.

### 9.8.1 PHP 8 — funções de sessão

Não é Python, mas cai no mesmo tipo de item (“qual função faz X”): `session_start`, `session_destroy`, `session_regenerate_id`.

## 9.9 Estruturas de dados

A FGV quase nunca pergunta “o que é uma pilha”. Ela descreve um **requisito** — “inserções e remoções frequentes no meio da coleção”, “busca por chave em tempo constante”, “listar em ordem e consultar por faixa” — e pergunta qual estrutura atende. Por isso decorar a definição não resolve: o que se cobra é o **custo de cada operação**, que é o que a escolha da estrutura fixa antes de a primeira linha ser escrita. E é uma escolha cara de desfazer, porque todo o código em volta se apoia nela.

#### Cada estrutura é um contrato de custo

| Estrutura              | O que é barato                                                       | O que é caro / o preço                                                    |
|------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------|
| Vetor ( <i>array</i> ) | acesso pela posição em $O(1)$                                        | inserir ou remover no meio é $O(n)$ : os demais elementos <b>deslocam</b> |
| Lista encadeada        | inserir/remover em $O(1)$ — <b>desde que você já esteja no ponto</b> | chegar ao $i$ -ésimo é $O(n)$ : percorre desde o início                   |
| Duplamente encadeada   | remover um nó conhecido em $O(1)$ e percorrer nos dois sentidos      | dois ponteiros por nó (memória a mais)                                    |
| Pilha (LIFO)           | empilhar/desempilhar no topo em $O(1)$                               | só o topo é acessível                                                     |
| Fila (FIFO)            | enfileirar/desenfileirar em $O(1)$                                   | só as pontas                                                              |
| Tabela hash            | busca/inserção/remoção em $O(1)$ <b>em média</b>                     | <b>não guarda ordem</b> ; pior caso $O(n)$                                |
| Árvore balanceada      | busca em $O(\log n)$ <b>mantendo a ordem</b>                         | mais lenta que o hash na busca pontual                                    |

Duas frases resolvem a maioria dos itens. A primeira: **vetor troca inserção por acesso, lista encadeada troca acesso por inserção** — e o  $O(1)$  da lista pressupõe que você já tem o ponteiro; se for preciso *achar* a posição, some o  $O(n)$  da busca. A segunda: **hash e árvore não competem pela mesma vaga** — se o enunciado pede *faixa* de valores ou *listagem ordenada*, a resposta é a árvore, mesmo que a tabela hash esteja entre as alternativas com o seu  $O(1)$  sedutor.

**Busca:** a binária é  $O(\log n)$ , mas **exige a coleção já ordenada** — sem essa condição ela não se aplica. Em arquivo desordenado e sem índice, a única saída é a **busca sequencial**,  $O(n)$ .

**Exemplo trabalhado — por que o hash degrada.** Uma tabela hash com  $m$  posições guardando  $n$  chaves tem **fator de carga**  $\alpha = n/m$ . Duas chaves diferentes podem cair na mesma posição (colisão), e há duas famílias de tratamento:

- **Encadeamento separado:** cada posição aponta para uma lista. As chaves moram **fora** do vetor, então  $\alpha$  **pode passar de 1**; o custo médio da busca cresce com o comprimento da lista.
- **Endereçamento aberto:** a chave mora **dentro** do próprio vetor, numa posição livre achada por *sondagem*. Como cada posição comporta uma chave,  $\alpha \leq 1$  **sempre** — é o que limita o fator de carga a 1.

Agora o caso concreto: endereçamento aberto com **sondagem linear** (“ocupado? tente a próxima posição”) sob ocupação alta,  $\alpha \approx 0,8$ . Só uma posição em cinco está livre, e as ocupadas se juntam em **blocos contíguos**. Uma chave que caía em *qualquer* ponto de um bloco tem de caminhar até o fim dele — e, ao se acomodar ali, **aumenta o bloco**, o que torna mais provável que a próxima chave também caia nele. Esse laço de realimentação é o **agrupamento primário** (*primary clustering*), e é ele que empurra a busca do  $O(1)$  médio para perto do  $O(n)$ . Remédios: sondagem quadrática ou hash duplo (espalham a sequência de tentativas) e, sobretudo, **redimensionar a tabela** quando  $\alpha$  passa do limiar — é o  $\alpha$ , não o número de chaves, que governa o desempenho.

Repare que a mesma lógica reaparece em Banco de Dados (Capítulo 6): o índice hash do SGBD encontra a linha por igualdade em tempo constante e é inútil para **BETWEEN** ou **ORDER BY** — exatamente porque hash não preserva ordem. Estrutura de dados e índice são a mesma discussão, em dois níveis.

#### ATENÇÃO — Como a FGV arma a pegadinha

A troca de custo é o desenho padrão: a alternativa afirma acesso  $O(1)$  ao  $i$ -ésimo elemento da **lista encadeada** (não existe — o  $O(1)$  é da inserção com o ponteiro em mãos) ou inserção  $O(1)$  no meio do **vetor** (esquece o deslocamento). No hash, o absoluto denuncia: “a tabela hash garante busca em tempo constante” sem o **em média** — e a correta é justamente a que traz a ressalva. Cenário de **consulta por faixa** com hash entre as opções: é árvore balanceada. Busca binária oferecida sem a coleção estar **ordenada**: não se aplica. E o par que a banca inverte no fim: encadeamento separado (o  $\alpha$  pode passar de 1)  $\times$  endereçamento aberto (o  $\alpha$  nunca passa de 1). Pilha  $\times$  fila costuma vir com **dois cenários no mesmo item** — responda um de cada vez, LIFO e FIFO, antes de olhar as alternativas.

## 9.10 Leitura ativa de código — técnica transversal

A FGV não para na terminologia: em Programação ela mostra um trecho pequeno de código (Python, SQL, JS, config) e pergunta a **saída** ou o **efeito**. Quem só decorou conceito trava aqui — o antídoto é **rodar o código mentalmente**, linha a linha, como um interpretador, anotando o estado das variáveis num rascunho.

### Protocolo de leitura ativa

1. **Não leia por cima.** Releia a linha inteira antes de assumir o que ela faz — a FGV muda *um* parâmetro, *um* operador ou *um* índice em relação ao que “pareceria óbvio”.
2. **Anote o estado.** Para cada variável relevante, escreva o valor depois de cada linha executada — não confie na memória depois da terceira linha.
3. **Desconfie de referência × cópia.** É a armadilha mais comum em qualquer linguagem: duas variáveis apontam para o **mesmo** objeto/array, e mudar uma “muda” a outra — ou o contrário, quando a API/linguagem faz cópia.
4. **Confirme contra a assinatura real da função/método,** não contra o que você acha que ela deveria fazer.

### Quatro fatos da linguagem que decidem a resposta antes do código

Simular a execução só funciona se você souber as regras do jogo. Estas quatro respondem à maior parte dos itens de leitura de código:

- **Compilada × interpretada × bytecode.** Compilada traduz para código de máquina *antes* de rodar — o erro de tipo aparece na compilação. Interpretada executa comando a comando — o mesmo erro só aparece **na linha em que a execução chega**. O **Java é o meio-termo** que a banca gosta de cobrar: compila para **bytecode** (`.class`), e a **JVM** é quem executa, compilando os trechos quentes para código nativo em tempo de execução (JIT).
- **Passagem por valor × por referência.** Em Java e em Python o que se passa é sempre o **valor da referência**. Consequência prática: **reatribuir** o parâmetro dentro da função *não* muda a variável de quem chamou; **mutar** o objeto apontado muda para os dois. Diante do código, a pergunta certa não é “é por valor ou por referência?”, e sim **esta linha reatribui ou muda?**
- **Imutável × mutável.** Em Python, `str`, `int` e `tuple` são imutáveis; `list`, `dict` e `set` são mutáveis. O que a banca cobra é a consequência: só objeto imutável (portanto *hashável*) serve de **chave de dicionário** ou elemento de `set` — **tupla pode, lista não**. Em Java a mesma ideia aparece na `String` imutável, razão de existir o `StringBuilder` (Capítulo 10).
- **Duck typing.** O que importa é o objeto *responder* ao método, não a que classe ele pertence — a compatibilidade é verificada no uso, não na declaração. É por isso que Python dispensa a interface explícita que o Java exige.

**Aviso de homônimo:** o *decorator* do Python (a sintaxe `@`) é uma função que recebe outra função e devolve uma versão embrulhada dela — metaprogramação da linguagem. **Não** é o padrão Decorator do GoF (Capítulo 4), que é composição de objetos. Mesmo nome, assuntos diferentes.

### Exemplo resolvido (NumPy — referência × cópia):

| Código                             | Execução mental                                                                            |
|------------------------------------|--------------------------------------------------------------------------------------------|
| <code>a = np.array([1,2,3])</code> | <code>a = [1,2,3]</code> (original)                                                        |
| <code>b = a.view()</code>          | <b>b compartilha</b> o buffer de <code>a</code> ;<br><code>b.base</code> is <code>a</code> |
| <code>a[0] = 99</code>             | muda o buffer compartilhado                                                                |
| <code>print(b[0])</code>           | imprime <b>99</b> (reflete, pois é view)                                                   |



Se a segunda linha fosse `b = a.copy()`, `b[0]` continuaria `1` depois de alterar `a[0]` — porque `copy()` rompe o vínculo com o buffer original. A pegadinha da FGV é exatamente trocar `view` por `copy` no enunciado e esperar que você não perceba a diferença de comportamento.

### Como treinar leitura ativa

Toda vez que uma questão do quiz trouxer código, resista ao impulso de ler a alternativa primeiro. Pegue papel, simule a execução, chegue a um valor, e só depois compare com as alternativas — isso elimina o efeito “parece certo”. Vale para Python, SQL, Java (Capítulo 10) e HTML/CSS/JS (Capítulo 11): é a mesma disciplina em qualquer linguagem.

### O que já caiu nas provas

**Em prova real da FGV:** papéis de Spring/Spring Cloud/Spring Boot/Hibernate/JUnit, XML/XSLT/JSON, Flutter (Dart), blockchain (o que *não* fica no bloco) e SPA × PWA — **Dataprev 2024**; Kotlin/Swift, NumPy `view` × `copy` e o `explode` do `matplotlib` — **MPU**; funções de sessão do PHP 8 — **TJ-RJ**. E, sobretudo, **estruturas de dados clássicas** — **pilha** × **fila** correlacionadas a LIFO/FIFO em dois cenários no mesmo item; **tabela hash** escolhida pelo requisito de busca  $O(1)$  em média; **lista duplamente encadeada** para inserção e remoção  $O(1)$  no meio; endereçamento aberto com **sondagem linear** sob ocupação alta ( $\alpha \approx 0,8$ ), cuja resposta é **agrupamento primário**; busca em arquivo desordenado e sem índice; **imutabilidade** e tupla como chave de dicionário; passagem por valor × por referência; interpretada × compilada; *decorators* e *duck typing* — **ALERO 2026**, que é a maior fonte do bloco no corpus (13 das 23 questões reais).

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): `max(dic, key=)` e fatiamento em Python; comandos do Git (`add/fetch/rebase`); entrega × implantação contínua; Docker × Kubernetes; estrutura do JSON; REST (201 Created, PUT idempotente); agrupamento primário como causa da degradação do hash; encadeamento separado × endereçamento aberto (onde mora o sinônimo, e qual dos dois limita o fator de carga a 1); árvore balanceada quando o requisito é consulta por faixa e listagem ordenada; pré-requisito de ordenação da busca binária; vetor × lista encadeada (deslocar elementos × reapontar ponteiros). Rode `./quiz.py programacao` e `./quiz.py git-devops`.

## 9.11 Alta probabilidade / pesquisa extra

- **Confluent Kafka** (mensageria/streaming): tópicos, produtores/consumidores — ver Capítulo 5.
- **API/Swagger (OpenAPI)**: documenta e contrata REST; Swagger UI.
- **RPA** (UiPath, Automation Anywhere): automação de tarefas de interface.
- **IA/LLM** entrou no edital — ver Capítulo 18 para os fundamentos.

### Como se sair melhor

Fixe uma frase por framework: Boot = configura e sobe sozinho (servidor embarcado); Cloud = distribuído/service discovery; Hibernate = objeto↔tabela; JUnit = testa; Spring = container/injeção. Nativo mobile: Kotlin/Android, Swift/iOS (Objective-C e Java são os antigos). Multiplataforma Dart = Flutter. Em Git/DevOps, pergunte **quem faz o quê**: quem cria commit, quem toca a cópia de trabalho, quem vai ao remoto, quem decide subir para produção — a resposta quase sempre se resolve nessa separação, sem precisar da sintaxe. Gatilhos: “exclusivamente”, “sempre”, “estritamente equivalentes”, “elimina a necessidade”. Em questão de



código, leia linha a linha — a FGV muda um parâmetro só.

## Capítulo 10

# Java

### Ficha do edital

Java (versão 6+), JavaEE (6+), JakartaEE, JPA (2+), frameworks JUnit, Hibernate, JSF, Primefaces, Spring/SpringCloud/SpringBoot. É a linguagem-base do perfil.

Peso esperado: **MÉDIO** — na amostra de provas, Java concentrou-se no TJ-RJ (Analista de Sistemas); caiu pouco na Dataprev 2024. Mas está no edital: estude os pares que a FGV adora inverter.

### 10.1 Exceções: checked × unchecked

|             | Checked (verificada)                    | Unchecked (não verificada)                                                     |
|-------------|-----------------------------------------|--------------------------------------------------------------------------------|
| Superclasse | Exception (não RuntimeException)        | RuntimeException                                                               |
| Compilador  | <b>exige</b> try-catch <b>ou</b> throws | não exige                                                                      |
| Exemplos    | IOException, SQLException               | NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException |

### ATENÇÃO — Como a FGV arma a pegadinha

Pegadinha nº 1 do bloco: trocar os exemplos. `NullPointerException` é **unchecked** (estende `RuntimeException`); `IOException` é **checked**. Absoluto a descartar: “unchecked exige throws obrigatório” — é o contrário.

### 10.2 Polimorfismo: overload × override

|            | Overload (sobrecarga)               | Override (sobrescrita) |
|------------|-------------------------------------|------------------------|
| Onde       | mesma classe                        | subclasse              |
| Assinatura | <b>muda</b> (parâmetros diferentes) | <b>igual</b> à do pai  |
| Resolução  | compile-time (estática)             | runtime (dinâmica)     |
| Anotação   | —                                   | @Override              |

A assinatura é o nome + a lista de parâmetros — o tipo de retorno não entra nela.

Logo, dois métodos no mesmo escopo que diferem **apenas no retorno** (`int calc(int x)` e `long calc(int x)`) **não compilam**: não é overload, é declaração duplicada.

### ATENÇÃO — Como a FGV arma a pegadinha

Overload = mesmo nome, assinatura diferente; override = mesma assinatura na subclasse. Sobrescrever **não** viola Liskov por si só. E o distrator recorrente é apresentar a mudança **só no tipo de retorno** como se fosse sobrecarga válida — é erro de compilação.

## 10.3 OO e SOLID

Pilares: abstração, encapsulamento, herança, polimorfismo.

### Os quatro pilares — o que cada um esconde de você

Os quatro têm o *mesmo* propósito: limitar o que o resto do sistema precisa saber. O que muda é o **que** está sendo escondido.

| Pilar          | Esconde...                                      | Como aparece no código                                                                                     |
|----------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Abstração      | os detalhes irrelevantes para quem usa          | a classe expõe <i>o que</i> faz ( <b>sacar</b> ), não <i>como</i>                                          |
| Encapsulamento | o <b>estado</b>                                 | atributo <b>private</b> + operações públicas que <b>protegem a regra</b>                                   |
| Herança        | a parte comum, escrita uma vez                  | <b>extends</b> : relação <b>é-um</b>                                                                       |
| Polimorfismo   | <b>qual</b> implementação vai atender a chamada | <code>conta.sacar()</code> executa o método da classe real do objeto, decidido em <b>tempo de execução</b> |

**Dois confusões que a banca explora.** A primeira: encapsulamento **não** é “gerar getter e setter para todo atributo” — um `setSaldo` público expõe o estado tão bem quanto o atributo público, e a regra (“não sacar além do saldo”) fica sem dono. Encapsular é expor *comportamento*, não campo. A segunda: o polimorfismo que a banca cobra é o de **sobrescrita**, resolvido em *runtime* pela classe real do objeto — a sobrecarga é escolhida em **compilação**, pelos tipos declarados, e por isso alguns autores nem a contam como polimorfismo de verdade. Herança é o pilar com **contraindicação**: ela acopla a subclasse à implementação do pai. Quando a relação for “**tem-um**” e não “é-um”, o certo é **composição** (Capítulo 4).

| Letra | Princípio                                                            |
|-------|----------------------------------------------------------------------|
| S     | Single Responsibility — uma responsabilidade por classe              |
| O     | Open/Closed — aberta a extensão, fechada a modificação               |
| L     | <b>Liskov</b> — subtipo substitui o tipo base sem quebrar o programa |
| I     | Interface Segregation — interfaces pequenas e específicas            |
| D     | Dependency Inversion — depender de abstração, não de implementação   |

**Exemplo trabalhado — a violação de Liskov que a FGV mais usa.** A herança compila; o problema aparece em quem *usa* o supertipo:

| O código                                                                                                                      | O que acontece                                                                                                                |
|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Retangulo {   setLargura; setAltura;   area() }</pre>                                                              | quem usa assume: mudar a largura não muda a altura                                                                            |
| <pre>class Quadrado extends Retangulo — e cada <i>setter</i> muda <b>os dois</b> lados, para manter o quadrado quadrado</pre> | a classe está coerente consigo mesma...                                                                                       |
| <pre>r.setLargura(5); r.setAltura(4); assert r.area() == 20;</pre>                                                            | ... mas com um Quadrado no lugar do Retangulo a área dá <b>16</b> . O código do cliente, que nunca soube do quadrado, quebrou |

Liskov não fala sobre a subclasse: fala sobre **quem consome o supertipo**. Daí o critério prático, que é o que resolve o item de prova: a subclasse não pode **exigir mais** do que o pai (fortalecer pré-condição), não pode **entregar menos** (enfraquecer pós-condição) e não pode quebrar uma **invariante** que o cliente já podia supor. O exemplo canônico do outro lado é o *Pinguim* que herda de *Ave* com o método *voar*: a hierarquia parece natural e o contrato não fecha.

**Nota que rende ponto:** o próprio compilador do Java já barra três violações de Liskov na sobrescrita — o método sobrescrito não pode **reduzir a visibilidade**, não pode declarar exceção **checked mais ampla** que a do pai, e o retorno tem de ser o mesmo ou um **subtipo** (covariante). O que ele *não* consegue barrar é a quebra de contrato semântico do exemplo acima — e é exatamente ali que a banca cobra.

#### ATENÇÃO — Como a FGV arma a pegadinha

Não confundir **ISP** (interface enxuta) com **SRP** (uma responsabilidade) — são os dois princípios que a FGV mais troca no SOLID. Liskov: a subclasse pode sobrescrever, desde que continue substituível.

## 10.4 Interface × classe abstrata

|                       | Classe abstrata                               | Interface                                                              |
|-----------------------|-----------------------------------------------|------------------------------------------------------------------------|
| Construtor            | <b>tem</b>                                    | <b>não tem</b>                                                         |
| Atributo de instância | tem (com estado)                              | só <code>public static final</code> (constante)                        |
| Herança múltipla      | <b>não</b> — só uma classe pode ser estendida | <b>sim</b> — várias podem ser implementadas                            |
| Método concreto       | sim, desde sempre                             | sim, a partir do Java 8 ( <code>default</code> e <code>static</code> ) |
| Palavra-chave         | <code>extends</code>                          | <code>implements</code>                                                |

Regra de bolso: uma classe **estende uma** classe (abstrata ou não) e **implementa várias** interfaces. Use classe abstrata quando há **estado e comportamento comuns** a compartilhar; interface quando o que importa é o **contrato**.

### ATENÇÃO — Como a FGV arma a pegadinha

Desde o Java 8 a interface tem métodos `default` com corpo — então “interface não pode ter implementação” virou **falso**, e a FGV usa essa desatualização como distrator. O que a interface continua **não** tendo é **construtor e atributo de instância**: é por aí que se separam as duas. Outro distrator: dizer que Java tem herança múltipla de **classes** (não tem — só de **tipo**, via interfaces).

## 10.5 Coleções (Collections)

O desenho do *framework* de coleções é uma aula de **contrato × implementação**, e é por isso que ele cai. `List`, `Set` e `Map` são **interfaces** — dizem que garantia você tem (ordem, unicidade, chave-valor) e nada sobre como. `ArrayList` e `LinkedList` são **implementações** da mesma interface, com os custos opostos das estruturas do Capítulo 9 (Seção 9.9). Por isso a boa prática é declarar pela interface (`List<String> lista = new ArrayList<>()`): trocar a implementação depois não mexe em quem usa.

| Interface         | Implementações                                                              | Característica               |
|-------------------|-----------------------------------------------------------------------------|------------------------------|
| <code>List</code> | <code>ArrayList</code> ,<br><code>LinkedList</code>                         | ordenada, permite duplicatas |
| <code>Set</code>  | <code>HashSet</code> , <code>TreeSet</code> ,<br><code>LinkedHashSet</code> | sem duplicatas               |
| <code>Map</code>  | <code>HashMap</code> , <code>TreeMap</code> ,<br><code>LinkedHashMap</code> | chave-valor                  |

**ArrayList** (array dinâmico, acesso  $O(1)$  por índice) × **LinkedList** (lista ligada, inserção/remoção  $O(1)$  no meio se já tem o nó). **HashMap** (sem ordem) × **LinkedHashMap** (ordem de inserção) × **TreeMap** (ordenado). `getOrDefault` retorna o **default** se a chave não existe — não lança exceção nem retorna erro HTTP. `==` compara **referência**; `.equals()` compara **conteúdo**.

### O contrato equals/hashCode — por que o objeto “desaparece” do HashSet

As coleções com **Hash** no nome não procuram o elemento comparando com todos: elas calculam o `hashCode()` para achar o **compartimento** e só ali usam `equals()` para confirmar. Isso impõe um contrato:

se `a.equals(b)` é `true`, então `a.hashCode() == b.hashCode()` **obrigatoriamente**

A recíproca não vale — dois objetos diferentes podem ter o mesmo *hash* (é colisão, e é normal). **A consequência que a banca cobra:** sobrescrever `equals` e **esquecer** `hashCode` produz um bug silencioso. Dois objetos “iguais” caem em compartimentos diferentes, então o `HashSet` aceita a duplicata e o `contains/get` do `HashMap` **não encontra** um objeto que está lá dentro. Nada estoura — só dá a resposta errada, que é o pior desenho possível para um item de “qual a saída”.

**Detalhe de “quase certa”:** `TreeSet` e `TreeMap` *não* usam `hashCode`; eles ordenam, e portanto dependem de `compareTo` (`Comparable`) ou de um `Comparator`. Uma classe sem nenhum dos dois estoura `ClassCastException` ao entrar num `TreeSet` — e continua funcionando perfeitamente num `HashSet`.

#### 10.5.1 String: imutabilidade e StringBuilder

**String é imutável:** toda “alteração” (`concat`, `+`, `replace`, `toUpperCase`) devolve um **objeto novo** e deixa o original intacto. Por isso concatenação dentro de laço cria um objeto descartado por iteração — em concatenação intensiva use **`StringBuilder`** (mutável, não sincronizado) ou **`StringBuffer`** (mutável e sincronizado, mais lento).

Literais iguais compartilham o ***string pool***, então `"a" == "a"` costuma dar `true`; já `new String("a") == "a"` dá `false`, porque o `new` força um objeto fora do pool. Comparação de conteúdo é sempre `.equals()`.

#### ATENÇÃO — Como a FGV arma a pegadinha

“**String** é mutável porque posso reatribuir a variável” — **falso:** o que muda é a **referência**, não o objeto. E cuidado com o par **`StringBuilder`** (não sincronizado, mais rápido) × **`StringBuffer`** (sincronizado, thread-safe): a FGV inverte os dois.

### 10.6 JVM, JPA e JavaEE

- **JVM:** executa bytecode; **heap** (objetos), **garbage collection** (libera memória sem referência); ferramentas de monitoramento (`jconsole`, `jps`, `jstack`).
- **JPA** (especificação de persistência) × **Hibernate** (implementação ORM). **EAGER** × **LAZY** (carregamento antecipado × sob demanda). Problema **N+1** (uma consulta por relação): resolve com **LAZY + JOIN FETCH** na consulta.
- **JavaEE/JakartaEE:** EJB, JPA, JMS, JSF; **Jakarta** é a continuação do JavaEE sob a Eclipse Foundation (namespace `jakarta.*` em vez de `javax.*`).
- **JSF/Primefaces:** framework de UI server-side baseado em componentes.

#### A JVM em duas ideias: bytecode e as duas áreas de memória

**Por que existe.** O compilador Java não gera código de máquina: gera **bytecode** (`.class`), uma instrução intermediária que nenhuma CPU executa. Quem executa é a **JVM**, e existe

uma JVM para cada sistema operacional — é daí, e só daí, que vem o *write once, run anywhere*. O `.class` é o mesmo em Windows e Linux; o que muda é o intérprete.

**Onde as coisas moram.** A separação explica metade das perguntas de memória:

| Área         | O que guarda                                                                                                                                                     |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Stack</b> | uma por <b>thread</b> : um quadro por chamada de método, com os parâmetros, as variáveis locais e as <b>referências</b> . O quadro morre quando o método retorna |
| <b>Heap</b>  | os <b>objetos</b> , compartilhados por todas as threads. Sobrevivem ao método que os criou, enquanto alguém os alcançar                                          |

A variável local mora na *stack*; o objeto que ela aponta mora no *heap*. É por isso que “passar o objeto” é passar uma referência copiada (Capítulo 9), e é por isso que a *stack* é uma pilha de verdade — a mesma estrutura LIFO da Seção 9.9, agora como mecanismo de execução: recursão sem fim estoura a pilha (`StackOverflowError`), objeto demais estoura o heap (`OutOfMemoryError`).

**Garbage collection.** O coletor libera o que **não é mais alcançável** a partir das raízes (variáveis vivas nas pilhas, campos estáticos) — não o que “não é mais usado”. Duas consequências que a banca cobra: Java **não tem destrutor determinístico** (você não sabe *quando* o objeto será coletado) e `System.gc()` é **sugestão**, não ordem. Recurso externo (arquivo, conexão) se fecha à mão, com `try-with-resources` — o GC cuida de memória, não de *handles*.

#### ATENÇÃO — Como a FGV arma a pegadinha

JPA N+1: o distrator oferece EAGER em tudo ou “deixar o JPA decidir” como solução — a resposta certa é **LAZY + JOIN FETCH** na consulta que precisa da relação.

### 10.6.1 Anotações que a FGV cobra pelo nome

O papel de cada framework do ecossistema Spring está no Capítulo 9; aqui ficam as anotações que aparecem em item de leitura de código.

| Anotação                                   | Papel                                                                                                                             |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <i>Spring — estereótipos de componente</i> |                                                                                                                                   |
| @Component                                 | bean genérico gerenciado pelo contêiner                                                                                           |
| @Service                                   | camada de regra de negócio                                                                                                        |
| @Repository                                | camada de <b>acesso a dados</b> ; acrescenta a <b>tradução automática de exceções</b> de persistência para a hierarquia do Spring |
| @Controller / @RestController              | camada web; o @RestController já embute @ResponseBody                                                                             |
| @Autowired                                 | injeta a dependência                                                                                                              |
| <i>JPA — mapeamento objeto-relacional</i>  |                                                                                                                                   |
| @Entity                                    | marca a classe como entidade persistente                                                                                          |
| @Id                                        | identifica a chave primária (com @GeneratedValue para geração automática)                                                         |
| @Table / @Column                           | renomeiam tabela e coluna (opcionais)                                                                                             |
| @ManyToOne                                 | <b>muitos</b> registros desta entidade para <b>um</b> da outra (é o lado dono, com a chave estrangeira)                           |
| @OneToMany                                 | o inverso; costuma vir com mappedBy                                                                                               |
| @OneToOne, @ManyToMany                     | um-para-um e muitos-para-muitos                                                                                                   |

O par mínimo para uma entidade JPA é @Entity + @Id — todo o resto é opcional.

#### ATENÇÃO — Como a FGV arma a pegadinha

A FGV troca o **estereótipo** pela camada errada (@Service no DAO, @Repository na regra de negócio) e inverte a **cardinalidade**: @ManyToOne lê-se do lado de cá para o lado de lá (**muitos** pedidos para **um** cliente). Distrator de anotação inventada também é comum (@Persistent, @PrimaryKey, @DataAccess) — decore o nome exato.

## 10.7 Recursos modernos (Java 17/21 LTS)

- **Sealed / non-sealed / final**: uma classe **sealed** exige subclasses declaradas (**permits**, ou no mesmo arquivo), e cada subtipo deve ser **final**, **sealed** ou **non-sealed**. **non-sealed** reabre a hierarquia.
- **Threads virtuais (Project Loom)** × threads de plataforma: milhares de **virtuais** compartilham uma **carrier thread** de plataforma (não é uma-para-uma). Brilham em cenários I/O-bound.
- **Streams e lambdas** (Java 8+): programação funcional em coleções.

#### Stream é um pipeline preguiçoso — e é aí que está o item de prova

Um *stream* não é uma coleção: é um **fluxo de processamento** sobre uma fonte. Ele tem três partes, e a terceira é a que importa:

fonte (lista.stream()) → operações **intermediárias** → operação **terminal**



- **Intermediárias** (`filter`, `map`, `sorted`, `distinct`, `limit`) devolvem *outro stream* e são **preguiçosas**: nenhuma executa na hora em que é escrita.
- **Terminais** (`collect`, `forEach`, `reduce`, `count`, `anyMatch`) **disparam** o pipeline e o encerram.

Daí os três fatos que a FGV transforma em alternativa: um pipeline **sem operação terminal não executa nada** (nem o `map` roda); o stream **não altera a fonte** — `map` devolve valores novos e a lista original continua intacta; e um stream é de **uso único** (reaproveitar o mesmo objeto dá `IllegalStateException`).

**Trio funcional, em uma linha cada:** `filter` escolhe *quais* passam (não muda o elemento); `map` transforma *cada* elemento, um para um; `reduce` combina todos em **um** resultado. A **lambda** é só a forma curta de escrever a implementação de uma interface de método único (`Predicate`, `Function`) no lugar de uma classe anônima — açúcar de sintaxe, não um recurso novo de tipo.

### 10.7.1 `var`, `record` e `switch` como expressão

**`var` (Java 10):** infere o tipo da variável **local** a partir do inicializador, em **tempo de compilação** — e o tipo **fica fixo** depois disso. Java continua estaticamente tipado. Exige inicializador (`var x;` não compila) e não vale para atributo de instância, parâmetro de método nem retorno.

**`record` (Java 16):** classe de dados imutável declarada pelo cabeçalho — `record Ponto(int x, int y) { }`. O compilador gera:

| Gerado                                      | Observação                                                            |
|---------------------------------------------|-----------------------------------------------------------------------|
| construtor canônico                         | pode ser sobrescrito (forma compacta) para validar                    |
| métodos de acesso                           | <code>p.x()</code> , <code>p.y()</code> — <b>sem</b> <code>get</code> |
| <code>equals</code> / <code>hashCode</code> | comparam componente a componente                                      |
| <code>toString</code>                       | formato <code>Ponto[x=1, y=2]</code>                                  |

Os campos são  **finais**. O `record` é implicitamente `final` e já estende `java.lang.Record`, logo **não pode estender outra classe** — mas **pode implementar interfaces**. Não aceita campo de instância adicional (só estático).

**`switch` como expressão (Java 14)** devolve valor e, na forma com seta (`->`), executa **só** o ramo correspondente: acabou o `break` e acabou o *fall-through* (use `yield` em bloco com chaves). **Blocos de texto (Java 15)**, delimitados por `"`, servem a conteúdo de várias linhas (JSON, SQL, HTML) sem concatenação nem `\n` manual.

#### ATENÇÃO — Como a FGV arma a pegadinha

“**`var`** torna a variável de tipo dinâmico” — **falso**: depois de inferido, atribuir valor incompatível é erro de compilação. “O **`record`** gera *getters* no padrão `getX()`” — falso, o acessor chama-se `x()`. E **`sealed`** **não** é sinônimo de `final`: o `final` proíbe herança, o `sealed` permite herança só pelos tipos autorizados.

**ATENÇÃO — Como a FGV arma a pegadinha**

Sealed: o distrator diz que `final` “não pode estender sealed” (pode) ou inventa erro de escopo; o erro real costuma ser uma sealed **sem nenhuma subclasse permitida**. Threads virtuais: o distrator diz que cada virtual equivale a uma de plataforma (consumo linear) ou que o número de threads de plataforma limita o de virtuais — é o contrário: muitas virtuais, poucas carriers.

## 10.8 Leitura ativa de código em Java

Java é a linguagem onde a FGV mais gosta de esconder o erro **dentro** de uma estrutura que parece correta à primeira vista — um `try-catch` que compila mas nunca captura a exceção certa, um escopo de variável que parece visível mas não é, um `synchronized` que quebra a promessa da thread virtual. Aplique o protocolo de leitura ativa do Capítulo 9 (Seção 9.10): releia linha a linha, sem assumir.

### 10.8.1 Checked/unchecked disfarçadas em try-catch

**O que a FGV esconde no bloco try-catch**

- **Ordem dos catch:** um `catch (Exception e)` antes de um `catch (IOException e)` **não compila** — o `catch` mais genérico precisa vir **por último**. A FGV mostra um trecho “quase certo” com a ordem trocada e pergunta o que acontece: é **erro de compilação**, não exceção não tratada em runtime.
- **Catch que não cobre o tipo lançado:** um método declara `throws SQLException` mas o bloco só tem `catch (RuntimeException e)` — isso **não compila**, porque `SQLException` é **checked** e exige tratamento explícito ou propagação com `throws` no método chamador.
- **finally que engole a exceção:** um `return` dentro do bloco `finally` **sobrescreve** qualquer exceção lançada no `try/catch` — a exceção original se perde silenciosamente. É pegadinha clássica de “qual valor o método retorna”.
- **Multi-catch:** `catch (IOException | SQLException e)` exige que as duas exceções **não tenham relação de herança entre si** (uma não pode ser subtipo da outra) — senão não compila.

**ATENÇÃO — Como a FGV arma a pegadinha**

O distrator típico apresenta um bloco `try-catch` com a ordem dos `catch` invertida (genérico antes do específico) e afirma que ele “compila e funciona normalmente” — **falso**, é erro de compilação. Outro distrator: dizer que uma exceção **checked** lançada dentro de um método `void` sem `throws` e sem `try-catch` “é ignorada em runtime” — também falso, isso **não compila**. Erro de compilação e exceção em runtime são coisas diferentes; a FGV testa se você distingue as duas.

### 10.8.2 Escopo de variável — a pegadinha silenciosa

- **Shadowing (sombreamento):** uma variável local com o **mesmo nome** de um atributo da classe **esconde** o atributo dentro do método — `this.atributo` continua acessível, mas `atributo` sozinho refere-se à variável local. Enunciado que usa o mesmo nome dos dois lados é sinal de alerta.
- **Variável efetivamente final em lambda/classe anônima:** uma variável local usada dentro

de uma lambda ou classe anônima precisa ser **final** ou **efetivamente final** (nunca reatribuída depois da inicialização) — código que reatribui essa variável depois de criar a lambda **não compila**.

- **Escopo de bloco em for:** uma variável declarada **dentro** do `for` (`for (int i = 0; ...)`) não existe fora dele — usá-la depois do laço é erro de compilação, não um valor residual do último loop.

#### ATENÇÃO — Como a FGV arma a pegadinha

A FGV declara uma variável dentro de um bloco (`if`, `for`, `try`) e depois faz o código “usá-la” fora do bloco na alternativa — isso é erro de compilação (variável fora de escopo), não um valor qualquer. Não confunda com JavaScript, onde `var` vaza escopo de função — em Java o escopo é sempre de **bloco**.

### 10.8.3 Armadilhas de threads virtuais

#### Pinning — quando a thread virtual não é tão leve assim

Uma thread virtual roda sobre uma **carrier thread** de plataforma. Se o código dentro de um bloco **synchronized** faz uma chamada **bloqueante** (I/O, banco de dados), a thread virtual pode ficar **presa (pinned)** à carrier em vez de liberá-la — isso derruba justamente a vantagem de escala das threads virtuais, porque agora uma operação bloqueante ocupa uma carrier inteira. Na prática, o cenário de risco é: **synchronized** + chamada bloqueante **dentro** do bloco sincronizado, sob alta concorrência.

#### ATENÇÃO — Como a FGV arma a pegadinha

O distrator diz que threads virtuais “eliminam totalmente qualquer preocupação com bloqueio ou concorrência” — **falso**: o cenário clássico de **synchronized** + I/O bloqueante ainda pode prender a thread virtual à sua carrier, anulando o ganho de escala. Outro distrator troca a causa: dizer que o problema é “excesso de memória” (é concorrência/pinning, não memória). Threads virtuais reduzem o **custo de criar** threads, não eliminam todo cuidado com seções críticas.

#### O que já caiu nas provas

**Em prova real da FGV:** classes **sealed** (o erro estava na subclasse **sealed** *sem permits*) — MPU; threads virtuais sobre a *carrier*, JPA N+1 (**FetchType.LAZY** + **JOIN FETCH**), leitura de REST controller (**@RestController**, **@GetMapping**, **@PathVariable**) com **getOrDefault**, Spring Cloud Eureka (atributo **lease-expiration**) e Hibernate Envers (auditoria via **Revision Listener**) — **TJ-RJ**. Liskov caiu na Dataprev 2024 e no MPU, mas como item de **Engenharia de Software** (Capítulo 3).

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): **checked** × **unchecked**; **overload** × **override**; **interface** × **classe abstrata**; coleções (**ArrayList**/**LinkedList**/**TreeMap**); **==** × **equals**; **String** imutável e **StringBuilder**; anotações de Spring e JPA; **record** e **var**; *pinning*; ordem de **catch** e exceção engolida por **finally**; escopo de variável em bloco.

Rode `./quiz.py java` e `./quiz.py java-moderno`.

### Como se sair melhor

Par para decorar: checked (`Exception`, verificada em compilação, trata ou declara `throws`) × unchecked (`RuntimeException`, não obriga tratamento); overload (mesmo nome, assinatura diferente, compile-time) × override (mesma assinatura na subclasse, runtime, `@Override`). N+1: LAZY para não carregar cedo + JOIN FETCH quando precisar da relação numa consulta só. Em leitura de código, execute mentalmente linha a linha e confie na semântica da API (`getOrDefault` → `default`). Gatilhos de distrator: “sempre”, “independentemente da carga”, “equivale a uma de plataforma”. Nomes de atributo/config (Eureka `lease-expiration`, Envers Revision Listener): a FGV troca por nomes plausíveis (`heartbeat-interval`, `interceptor`, filtro) — decore o nome exato.

## Capítulo 11

# Frontend

### Ficha do edital

Tecnologias e práticas frontend web — HTML, CSS, UX, Ajax, frameworks (VueJS, Angular, React); padrões de frontend; SPA e PWA; UX (sistemas de gestão de conteúdo, arquitetura de informação, portais, workflow, acessibilidade, usabilidade).

**Peso esperado: MÉDIO** — metade é distinguir tecnologias (SPA × PWA, frameworks, mobile) e metade é ler código HTML/CSS/JS e dizer a saída. Não dá para decorar só conceito; treine ler snippet.

### 11.1 HTML e CSS

**HTML:** estrutura/semântica (tags semânticas: `header`, `nav`, `main`, `article`, `section`, `footer`).

Vale entender por que a semântica é assunto e não decoração. Visualmente, `<div class="menu">` e `<nav>` podem ficar idênticos — a diferença é que só o segundo **informa o papel** daquele bloco. Quem lê esse papel são programas: o leitor de tela, que anuncia “navegação” e permite saltar direto para ela; o buscador, que pesa o conteúdo do `<main>` diferente do do `<footer>`; e o próprio navegador, que já entrega comportamento de teclado pronto em elementos nativos. É daí que sai a **regra número um do ARIA**, mais adiante neste capítulo: se existe tag nativa com a semântica certa, ela vence o atributo acrescentado depois. A separação clássica do bloco — **HTML estrutura, CSS apresenta, JS comporta** — é a mesma ideia de baixo acoplamento aplicada à página.

**CSS:** apresentação. **Box model:** content → **padding** (interno, dentro da borda) → **border** → **margin** (externo, fora da borda).

**Responsividade:** **media queries** (aplicam estilo conforme a característica do dispositivo, tipicamente a largura da tela), unidades relativas (% , em, rem, vw/vh), layouts flexíveis.

**Flexbox × Grid:** o **Flexbox** é **unidimensional** — distribui itens ao longo de **um** eixo por vez (linha *ou* coluna). O **Grid** é **bidimensional** — define linhas e colunas ao mesmo tempo. Um não substitui o outro: Grid para a estrutura da página, Flexbox para alinhar o conteúdo dentro de cada área.

**Especificidade dos seletores** (quem vence quando duas regras declaram a mesma propriedade): id (100) > classe / atributo / pseudoclas (10) > elemento (1). A ordem de declaração no arquivo só desempata **especificidades iguais** — não vence uma especificidade maior. (!important passa por cima de tudo, e por isso é desaconselhado.)

**@import × <link>:** os dois trazem uma folha de estilo externa, mas o **@import** é resolvido **dentro do CSS** e em série (bloqueia o carregamento em cascata), enquanto o **<link>** no HTML permite download em paralelo — por isso **<link>** é a prática recomendada. **@import não** é mecanismo de responsividade (é distrator recorrente em questão de media query, embora aceite condição de mídia).

**ATENÇÃO — Como a FGV arma a pegadinha**

**padding** × **margin** (interno × externo) é o par que a FGV mais inverte no bloco. Em CSS, some as regras com cuidado: **flex-direction: row-reverse** **inverte** a ordem visual; **::before** insere **antes**, **::after** **depois**; **content: attr(x)** imprime o **valor** do atributo **x**. Resolva no papel, elemento por elemento: aplique o pseudo-elemento, resolva o **attr()**, e só então aplique a direção do flex — a pegadinha está exatamente na soma dos três. Outras duas inversões do bloco: dizer que “a **última** regra declarada sempre vence” (só vence entre especificidades **iguais** — o seletor de **id** ganha do de classe mesmo declarado antes) e trocar **Flexbox** por **Grid** (uni × bidimensional).

**11.2 SPA × PWA (o coração deste bloco)**

|            | SPA (Single Page Application)                               | PWA (Progressive Web App)                          |
|------------|-------------------------------------------------------------|----------------------------------------------------|
| O que é    | app numa única página; troca conteúdo <b>sem recarregar</b> | site que se comporta como app nativo               |
| Chave      | roteamento no cliente, sem full reload                      | <b>Service Worker</b> , offline, <b>instalável</b> |
| Depende de | JS no navegador                                             | recursos web progressivos                          |

**SPA:** uma página, JavaScript atualiza o conteúdo sem recarregar (React, Angular, Vue).

**PWA:** usa **Service Worker** para **cache**, **offline** e **notificações**, e pode ser **instalada** no dispositivo como app nativo.

O **Service Worker** é um script que roda **em segundo plano**, separado da página, como **proxy** entre a aplicação e a rede — é daí que vêm o cache e o funcionamento offline. Não tem acesso direto ao DOM e **exige HTTPS** (só **localhost** é exceção), justamente porque interceptar requisições em conexão não cifrada seria um vetor de ataque. O **manifest** (**manifest.json**) é o outro pilar: nome, ícones e modo de exibição que permitem a **instalação**.

**Onde o HTML é montado: CSR, SSR, SSG e a *hydration***

A SPA resolveu a navegação e criou um problema novo: se o HTML é montado pelo JavaScript no navegador, o servidor entrega uma página **vazia** e o usuário espera o **bundle** baixar e executar antes de ver qualquer coisa. Daí as três estratégias, que se distinguem por **onde** e **quando** o HTML é gerado:

| Estratégia        | HTML é gerado...               | O que se ganha e o que se paga                                                                 |
|-------------------|--------------------------------|------------------------------------------------------------------------------------------------|
| CSR (client-side) | no navegador, a cada tela      | servidor leve e navegação instantânea depois; <b>primeira</b> pintura lenta e indexação frágil |
| SSR (server-side) | no servidor, a cada requisição | conteúdo visível e indexável de imediato; custo de CPU por requisição                          |
| SSG (estático)    | no build, uma vez              | o mais rápido e barato de servir; conteúdo só muda com nova publicação                         |

**Hydration** é o que costura SSR e SPA. O servidor manda HTML já pintado, mas HTML não tem comportamento: quando o JavaScript chega, ele “hidrata” aquela marcação — reassocia os componentes, o estado e os ouvintes de evento. A consequência é o detalhe que a banca gosta: existe uma janela em que a página **parece pronta e não responde ao clique**, porque foi pintada mas ainda não hidratada.

**Não confunda com a PWA.** SSR/CSR decidem *quem monta o HTML*; Service Worker e cache offline decidem *o que acontece quando não há rede*. São eixos independentes — uma SPA renderizada no cliente pode ser PWA, e uma página com SSR pode não ser.

#### ATENÇÃO — Como a FGV arma a pegadinha

**Service Worker é da PWA**, não requisito de SPA — a alternativa que dá Service Worker à SPA é falsa (caiu exatamente assim na Dataprev 2024). “SPA instala no SO como nativo” é característica de **PWA**. Nenhuma das duas “depende exclusivamente” de framework — dá para fazer com JS puro.

## 11.3 Frameworks

|           | React                   | Angular                     | Vue                   |
|-----------|-------------------------|-----------------------------|-----------------------|
| Tipo      | biblioteca de UI (Meta) | framework completo (Google) | framework progressivo |
| Linguagem | JS/JSX                  | TypeScript                  | JS                    |

#### ATENÇÃO — Como a FGV arma a pegadinha

A FGV troca React↔Angular (biblioteca × framework completo) e atribui TypeScript ao React. Em questão de **React**, cuidado com nomes inventados/parecidos (**preRender**, **preloadModule**) no lugar de **createPortal** (a função certa para renderizar fora da hierarquia normal do DOM).

## 11.4 Ajax e comunicação

**Ajax:** requisições **assíncronas** ao servidor sem recarregar a página (base do comportamento SPA); hoje via `fetch/XHR`, dados em JSON.

## 11.5 UX, acessibilidade e usabilidade

- **UX (experiência do usuário) × UI (interface):** UX é a experiência toda; UI é a camada visual.
- **Usabilidade** (Nielsen): eficiência, eficácia, satisfação, facilidade de aprendizado, prevenção de erros.
- **Acessibilidade: WCAG (W3C)** — quatro princípios **POUR**: perceptível, operável, compreensível, robusto. Cada critério de sucesso tem um **nível de conformidade: A** (mínimo), **AA** (o exigido na maioria das normas e contratos públicos) e **AAA** (máximo, nem sempre alcançável no site inteiro). No Brasil, **eMAG** para governo.
- **ARIA** (*Accessible Rich Internet Applications*): conjunto de atributos — `role`, `aria-label`, `aria-hidden`, `aria-live` — que acrescenta **semântica** a elementos **dinâmicos** ou construídos sem tag nativa, para que leitores de tela anunciem função, nome e estado. **Regra número um do ARIA: não usar ARIA.** Se existe elemento HTML nativo com a semântica certa (`<button>`, `<nav>`, `<label>`), use o nativo — o ARIA é complemento para o que o HTML não cobre, e ARIA mal aplicado piora a acessibilidade em vez de melhorar.
- **Arquitetura de informação:** organização e navegação do conteúdo.
- **CMS** (sistema de gestão de conteúdo): cria/gerencia conteúdo sem código (WordPress, portais corporativos, workflow editorial).

### UX × usabilidade × acessibilidade — três recortes, não sinônimos

Os três aparecem juntos no edital e a FGV cobra exatamente a fronteira entre eles:

| Recorte        | Pergunta que responde                                                  | Como se mede                                                             |
|----------------|------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Usabilidade    | o usuário <i>típico</i> consegue cumprir a tarefa com facilidade?      | eficácia, eficiência e satisfação <b>em um contexto de uso</b> declarado |
| Acessibilidade | <i>pessoas com deficiência</i> conseguem usar?                         | conformidade com critérios objetivos (WCAG A/AA/AAA)                     |
| UX             | como é a <i>experiência inteira</i> , inclusive antes e depois do uso? | percepção, expectativa, confiança — mais ampla e menos mensurável        |

Duas relações que resolvem o item. Primeira: **UX contém** usabilidade, que é um de seus atributos — não são a mesma coisa, e a **UI** é apenas a camada visual pela qual a experiência acontece. Segunda, a que a banca prefere: **acessibilidade não é um caso particular de usabilidade**. Um sistema pode ser cômodo para o usuário sem deficiência e *impossível* para quem navega por leitor de tela; e um sistema formalmente acessível pode ser péssimo de usar. Elas se cruzam, não se contêm.

A diferença de **natureza** é o resto: usabilidade é objetivo de qualidade, e acessibilidade é **requisito com base normativa** — WCAG (W3C) como critério técnico, **eMAG** como modelo



do governo brasileiro derivado dele, e a Lei Brasileira de Inclusão como obrigação legal em serviço público. Por isso “deixamos a acessibilidade para a próxima *sprint*, quando houver demanda” é sempre a alternativa errada.

### WCAG na prática

Princípio **operável**: preferir rótulos/labels bem associados aos campos e várias formas de entrada além do teclado; fugir de “submissão automática ao focar” (viola previsibilidade) — é o tipo de alternativa que a FGV usa como distrator de acessibilidade.

## 11.6 Mobile multiplataforma × nativo

- **Flutter** = Dart; **React Native** = JS; **Ionic** = web; **Xamarin** = C#.
- **Mobile nativo**: Android = **Kotlin** (antes Java); iOS = **Swift** (antes Objective-C). A FGV troca os pares.

## 11.7 Leitura ativa de código — CSS e React

Metade das questões de frontend não pede definição: pede para você **prever a renderização** de um trecho de HTML/CSS/JS. Aplique o mesmo protocolo de leitura ativa do Capítulo 9 — resolva no papel, propriedade por propriedade, antes de olhar as alternativas.

### 11.7.1 Pegadinhas visuais em CSS

#### Some as regras nesta ordem

1. **Box model primeiro.** O tamanho final visível de um elemento é `content + padding + border` (+ `margin` só conta para o espaço **ao redor**, não para o tamanho do próprio elemento). Se o enunciado usa `box-sizing: border-box`, `width` já **inclui** padding e border — se usa o padrão (`content-box`), `width` é só o conteúdo e padding/border **somam por fora**. A FGV monta a pegadinha exatamente nessa diferença de `box-sizing`.
2. **Pseudo-elementos.** `::before` insere conteúdo **antes** do conteúdo real do elemento (mas ainda **dentro** dele); `::after`, **depois**. Sem `content: ...` declarado, o pseudo-elemento **não aparece**, mesmo com outras propriedades definidas.
3. **content: attr(x).** Imprime o **valor** do atributo `x` do elemento HTML como texto — se o elemento não tiver esse atributo, o pseudo-elemento aparece **vazio**, não gera erro.
4. **Flex/Grid por último.** `flex-direction: row-reverse` inverte a **ordem visual** dos itens sem alterar a ordem no DOM/HTML; `justify-content` e `align-items` agem nos eixos principal e transversal — **trocar os dois** é o distrator mais comum de layout.

**Exemplo resolvido:** um `<span data-nivel="urgente">` com CSS `span::before { content: attr(data-nivel) "; font-weight: bold; }` → antes do texto do `span`, aparece em negrito o literal **“urgente: ”** — se a alternativa disser que aparece o texto `"data-nivel"` (o nome do atributo, não o valor), é o distrator clássico de `attr()`.

**ATENÇÃO — Como a FGV arma a pegadinha**

A FGV soma 2–3 regras na mesma questão (ex.: `box-sizing` + pseudo- elemento + `flex-direction`) e o erro está na **combinação**, não numa regra isolada. Resolva cada camada separadamente antes de somar o resultado — não tente “visualizar direto”.

**11.7.2 Escopo e closures em JavaScript****O item clássico do laço com var**

`var` tem escopo de **função** (não de bloco) e sofre *hoisting*: é içada para o topo da função. Num laço, isso significa que existe **uma única** variável, compartilhada por todas as funções criadas dentro dele — quando elas forem executadas, todas leem o valor **final**.

| Código                                                                        | Saída                                                            |
|-------------------------------------------------------------------------------|------------------------------------------------------------------|
| <pre>for (var i = 0; i &lt; 3; i++)   fs.push(() =&gt; console.log(i));</pre> | <b>3, 3, 3</b> — um <code>i</code> só, que vale 3 ao fim do laço |
| <pre>for (let i = 0; i &lt; 3; i++)   fs.push(() =&gt; console.log(i));</pre> | <b>0, 1, 2</b> — <code>let</code> cria uma variável por iteração |

`let` e `const` (ES6) têm escopo de **bloco**; `const` impede a **reatribuição** da variável, mas **não** congela o objeto — `const obj = {}; obj.x = 1;` é válido. `E ==` faz **coerção de tipo** (`"1" == 1` é `true`); `===` compara valor e tipo.

**ATENÇÃO — Como a FGV arma a pegadinha**

A alternativa que diz que o laço com `var` imprime **0, 1, 2** é o distrator número um — ela descreve o comportamento do `let`. O inverso também aparece: atribuir a `let` o vazamento de escopo que é do `var`. E não confunda com Java, onde o escopo é **sempre** de bloco (Capítulo 10).

**11.7.3 Renderização no React**

- **key em listas:** React usa a `key` para identificar qual item de uma lista mudou, foi adicionado ou removido entre renderizações. Usar o **índice do array** como `key` em lista que pode reordenar/inserir/remover item **quebra** esse rastreamento (o React pode reaproveitar o componente errado para o item errado) — a prática recomendada é usar um **id estável** dos dados, nunca o índice.
- **useState é assíncrono e em lote (batched):** chamar o *setter* de estado não atualiza a variável **na mesma linha** — o valor só reflete a mudança na **próxima renderização**. Um trecho que faz `setContador(contador + 1)` duas vezes seguidas **não** soma 2 automaticamente, porque as duas chamadas leem o mesmo `contador` “antigo” dentro do mesmo evento — para acumular corretamente usa-se a forma funcional `setContador(c => c + 1)`.
- **Reconciliação (Virtual DOM):** React compara a árvore virtual nova com a anterior e aplica só as diferenças no DOM real — não recria a página inteira a cada render. Criar uma

**função anônima nova a cada renderização** (ex.: `onClick={() => ...}` sem memoização) não quebra a reconciliação, mas pode gerar **re-renders desnecessários** em componentes filhos otimizados com `memo`.

#### ATENÇÃO — Como a FGV arma a pegadinha

Alternativa que diz que “usar o índice do array como **key** nunca traz problema” — falso quando a lista pode reordenar ou ter itens inseridos/removidos no meio. Alternativa que diz que `setState/useState` atualiza **imediatamente e de forma síncrona** — falso: é assíncrono e em lote dentro do mesmo handler de evento. Confundir **Virtual DOM** (estrutura em memória que o React usa para calcular a diferença) com o **DOM real** do navegador é outro distrator recorrente.

#### O que já caiu nas provas

**Em prova real da FGV:** leitura de HTML+CSS prevendo a renderização (pseudo-elementos, `content`, `attr()`, `flex-direction`), a função certa do React (`createPortal`, para renderizar fora da hierarquia normal do DOM) e WCAG na prática (associar rótulos aos campos, princípio **operável**) — MPU; SPA × PWA, com Service Worker/offline/instalável — **Dataprev 2024**. **No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): box model (`padding` × `margin`, `box-sizing`); especificidade de seletores; **Flexbox** × **Grid**; media queries; escopo de `var` × `let` em laço; frameworks (React × Angular); **key** em listas e `useState` assíncrono/*batched*; ARIA; UX × UI; Ajax; CMS.  
Rode `./quiz.py frontend` e `./quiz.py leitura-codigo`.

## 11.8 Alta probabilidade / pesquisa extra

- **HTTPS, SSL/TLS** estão no edital do frontend também — ver Capítulo 8.
- **SSR** × **CSR** (renderização no servidor × no cliente); **hydration**.
- **Web Components, micro-frontends** (tendência arquitetural).
- **Core Web Vitals** (performance percebida) e acessibilidade como requisito legal em serviços públicos (Lei Brasileira de Inclusão).

#### Como se sair melhor

Decore a tabela SPA × PWA: Service Worker, offline, instalável e “parece nativo” são atributos da PWA. Fixe pares mobile: Dart→Flutter, Kotlin→Android, Swift→iOS.

## Capítulo 12

# Gestão e Governança de TI

### Ficha do edital

Gerenciamento de projetos (conceitos, áreas, projetos/programas/portfólio; abordagens tradicional, híbrida e ágil — Scrum, Lean, Kanban; Guia Scrum); processos e grupos de processos; gestão de riscos; ITIL v4; COBIT 2019; gestão e modelagem de processos com BPMN.

**Peso esperado: MÉDIO** — ITIL 4, COBIT 2019 e PMBOK 7ª são o tripé. A FGV monta um mini-cenário (“a analista Patrícia...”, “a empresa Y...”) e pede qual prática, domínio ou princípio se aplica. Quem decorou definição solta erra; quem sabe a que domínio/fase cada coisa pertence acerta.

## 12.1 Gerenciamento de projetos (PMBOK)

- **Projeto × programa × portfólio:** projeto = esforço temporário com resultado único; **programa** = grupo de projetos relacionados; **portfólio** = conjunto de programas/projetos alinhados à estratégia.
- **PMBOK 6:** 5 **grupos de processos** (Iniciação, Planejamento, Execução, Monitoramento e Controle, Encerramento) × 10 **áreas de conhecimento** (integração, escopo, cronograma, custo, qualidade, recursos, comunicação, riscos, aquisições, partes interessadas).
- **PMBOK 7:** mudou para **princípios e domínios de desempenho** (foco em valor e resultados, menos prescritivo). Os **12 princípios** incluem foco em valor, pensamento sistêmico, ser um administrador zeloso (*steward*), liderança, qualidade, adaptação (*tailoring*) e **adaptabilidade e resiliência**.
- **Tripla restrição** (“triângulo de ferro”): **escopo, tempo e custo** — os três se condicionam, e mexer em um força ajuste nos outros. A **qualidade** é o que sofre o impacto direto quando se aperta os três (em algumas formulações ela entra como quarto vértice, junto com risco e recursos).
- **Abordagens:** **tradicional/preditiva** (escopo fixo, cascata), **ágil/adaptativa**, e **híbrida** (combina as duas conforme a necessidade).

**Por que a 6ª virou 7ª.** O PMBOK 6 descrevia **o que fazer**: 49 processos, com entradas, ferramentas e saídas, organizados na matriz de 5 grupos × 10 áreas. Funciona bem em projeto previsível e mal em projeto que descobre o escopo andando — e o guia passou a ser lido como receita a cumprir. A 7ª edição inverteu o eixo: saiu do *processo* e foi para o **princípio** (12) mais os **domínios de desempenho** (8), que são áreas de resultado a acompanhar, não etapas a executar. Junto veio o **tailoring**: adaptar a abordagem ao contexto passou a ser princípio, não exceção. As duas edições convivem em prova, e a distinção é justamente o que a banca cobra — **processo é PMBOK 6; princípio é PMBOK 7**.

**ATENÇÃO — Como a FGV arma a pegadinha**

Híbrido = **combinar** ágil + tradicional (nunca “só ágil” nem “só cascata”); não confundir grupo de processos (5, do PMBOK 6) com área de conhecimento (10) nem com princípio (12, do PMBOK 7 — que **não** é baseado em processos). No PMBOK 7, a banca oferece um princípio plausível mas não o do cenário: mudança de escopo por feedback = **foco em valor**; sistema que evolui e se integra ao ambiente = **pensamento sistêmico**; cuidado ético com o produto = **administrador zeloso**.

## 12.2 Scrum, Kanban, Lean (ágil)

**Scrum:** framework com Sprints time-boxed; papéis, eventos, artefatos (ver Capítulo 3). O Guia Scrum é citado no edital.

Os time-boxes que a FGV cobra pelo número (Sprint de um mês):

| Evento               | Duração máx. | Quem conduz / para quê                                                                                       |
|----------------------|--------------|--------------------------------------------------------------------------------------------------------------|
| Daily Scrum          | 15 min       | os <b>Developers</b> , todo dia, para inspecionar o progresso rumo à <b>Meta da Sprint</b> e adaptar o plano |
| Sprint Planning      | 8 h          | o time todo, no início da Sprint                                                                             |
| Sprint Review        | 4 h          | time + stakeholders, sobre o incremento                                                                      |
| Sprint Retrospective | 3 h          | o time, sobre o <i>processo</i>                                                                              |

Os 15 min da Daily são **fixos**, independentemente do tamanho da Sprint; os demais são proporcionais (os valores acima valem para Sprint de um mês e encolhem em Sprints menores).

**Kanban:** fluxo contínuo, limita WIP, sem sprints.

### Lead time × cycle time — o par de métricas do Kanban

| Métrica           | O que mede                                                                                                        |
|-------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Lead time</b>  | da <b>solicitação</b> (entrada no sistema, sob a ótica do cliente) até a <b>entrega</b> . Inclui o tempo em fila. |
| <b>Cycle time</b> | do <b>início do trabalho</b> até a entrega. É um <b>trecho</b> do lead time.                                      |
| <b>Throughput</b> | <b>quantidade</b> de itens entregues por período (não é tempo).                                                   |

Logo, **lead time** ≥ **cycle time** sempre — a diferença entre os dois é exatamente o tempo que o item passou **esperando** para ser puxado. Reduzir o WIP é o que encurta a fila e, com ela, o lead time (Lei de Little).

**Lean:** eliminar desperdício, maximizar valor.

**ATENÇÃO — Como a FGV arma a pegadinha**

Inversão de par clássica: dar ao **lead time** a definição do **cycle time** (“do começo do trabalho até a entrega”). A âncora é a palavra **solicitação**: se o enunciado conta o tempo a partir do **pedido do cliente**, é lead time. Outro distrator troca tempo por quantidade — “itens entregues por semana” é **throughput**, não lead time. E “o Kanban limita o WIP” × “o Scrum organiza em Sprints de duração fixa” é a distinção que a banca pede quando o cenário combina os dois métodos.

**Como se sair melhor**

Ágil: mudança tardia é bem-vinda e se prioriza no backlog — nunca “suspender o projeto” ou “cobrar taxa sem discutir” (distratores clássicos da FGV para esse cenário).

## 12.3 ITIL 4 (gerenciamento de serviços)

Foco em **cocriação de valor** por meio de serviços.

Essa frase é a chave da versão 4, e vale desmontá-la. **Serviço**, na ITIL, é o meio de permitir que o cliente alcance um resultado que ele quer, sem que precise gerenciar todos os custos e riscos envolvidos. E **cocriação** significa que o valor não é “entregue” de um lado para o outro: ele só existe quando o cliente *usa* o serviço — provedor e consumidor produzem o resultado juntos. É por isso que a v4 abandonou o vocabulário de **processos** (uma sequência que o provedor executa) e passou a falar em **práticas** (um conjunto de recursos organizacionais aplicados a um propósito). Trocar “prática” por “processo” num item de ITIL 4 é o distrator de vocabulário mais frequente do tema.

**Incidente × problema × requisição × mudança**

São quatro coisas diferentes que chegam pelo mesmo balcão (a **central de serviços**, que é ponto de contato, não solucionadora), e a FGV vive trocando as quatro:

| O que chegou                 | O que é                                                     | O objetivo do tratamento                                                      |
|------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Incidente</b>             | interrupção não planejada ou queda de qualidade do serviço  | <b>restaurar o serviço</b> o mais rápido possível — solução de contorno serve |
| <b>Problema</b>              | a <b>causa</b> de um ou mais incidentes                     | achar a <b>causa-raiz</b> e eliminá-la; não tem pressa de minutos             |
| <b>Requisição de serviço</b> | pedido <b>normal e previsto</b> (acesso, informação, senha) | atender com fluxo padronizado — <b>não houve falha</b>                        |
| <b>Mudança</b>               | alteração em algo que pode afetar serviços                  | maximizar mudanças bem-sucedidas, avaliando risco antes                       |

A **âncora**: incidente é medido em *tempo de restauração*; problema, em *causa encontrada*. Um incidente resolvido por contorno **fecha** mesmo com o problema aberto — e é isso que a alternativa “quase certa” nega, dizendo que não se fecha incidente sem eliminar a causa. Requisição de serviço é a que mais aparece disfarçada: se ninguém está fora do ar, não é incidente.

- **Sistema de Valor de Serviço (SVS):** como os componentes trabalham juntos para gerar valor. Inclui: princípios orientadores, governança, cadeia de valor de serviço, práticas e melhoria contínua.
- **7 princípios orientadores:** focar no valor; começar de onde se está; progredir iterativamente com feedback; colaborar e promover visibilidade; pensar e trabalhar de forma holística; manter simples e prático; otimizar e automatizar.
- **4 dimensões:** (1) organizações e pessoas; (2) informação e tecnologia; (3) parceiros e fornecedores; (4) fluxos de valor e processos.
- **Cadeia de valor de serviço (6 atividades):** Planejar, Melhorar, Engajar, Desenhar e transicionar, Obter/construir, Entregar e suportar.
- **34 práticas** em 3 grupos: **gerais** (ex.: gestão de risco, gestão de projeto), **de serviço** (ex.: gerenciamento de incidentes, gestão de mudança/*change enablement*, gerenciamento de problemas, central de serviços), **técnicas** (ex.: desenvolvimento de software, gestão de implantação).

#### ATENÇÃO — Como a FGV arma a pegadinha

Incidente (**restaurar serviço**) × problema (**causa-raiz**) × requisição de serviço — a FGV troca esses três conceitos. ITIL 4 fala em **práticas** (não “processos” como o ITIL v3). A banca também troca o propósito de práticas parecidas: gerência de infraestrutura e plataforma (monitorar/prover as soluções tecnológicas) × gerência de liberação (mover para produção) × central de serviços (ponto de contato).

## 12.4 COBIT 2019 (governança de TI)

Distingue **governança** (avaliar, dirigir, monitorar — responsabilidade do conselho) de **gestão** (planejar, construir, executar, monitorar — responsabilidade da diretoria).

#### Governança × gestão — a linha que organiza o capítulo inteiro

Essa distinção não é vocabulário: é a divisão de **quem decide o quê**, e é dela que sai o gabarito da maioria dos cenários.

|               | Governança                                                                | Gestão                                                               |
|---------------|---------------------------------------------------------------------------|----------------------------------------------------------------------|
| Pergunta      | “o <b>quê</b> e por <b>quê</b> ?” — que direção seguir, que risco aceitar | “ <b>como</b> e quando?” — executar dentro daquela direção           |
| Verbos        | <b>avaliar, dirigir, monitorar</b> (EDM)                                  | <b>planejar, construir, executar, monitorar</b> (APO, BAI, DSS, MEA) |
| Quem responde | o <b>conselho</b> /alta administração                                     | a <b>diretoria</b> executiva de TI                                   |

O **teste prático** para o item de prova: se o cenário fala em *definir appetite a risco*, *aprovar direção*, *cobrar resultado*, é governança (EDM). Se fala em *levantar requisito*, *construir*, *implantar*, *atender usuário*, *medir desempenho*, é gestão. Repare que **monitorar aparece nos dois lados** — e é aí que a banca planta a dúvida: a governança monitora *se a direção está sendo seguida* (EDM), a gestão monitora *o desempenho do que está rodando* (MEA).

Por isso o COBIT insiste que o sistema de governança é feito **sob medida** (os *fatores de desenho*): não existe conjunto de objetivos que sirva a toda organização — tamanho, setor,



apetite a risco e estratégia mudam a prioridade. “Implantar o COBIT inteiro como vem no guia” é distrator.

- **40 objetivos de governança e gestão em 5 domínios:**
  - **EDM** — Evaluate, Direct and Monitor (**governança**).
  - **APO** — Align, Plan and Organize (gestão).
  - **BAI** — Build, Acquire and Implement (gestão).
  - **DSS** — Deliver, Service and Support (gestão).
  - **MEA** — Monitor, Evaluate and Assess (gestão).
- **Fatores de desenho (design factors) e componentes do sistema de governança.**  
Princípios: sistema de governança feito sob medida; holístico; dinâmico; distingue governança de gestão; etc.

#### ATENÇÃO — Como a FGV arma a pegadinha

**EDM é governança**; APO/BAI/DSS/MEA são **gestão** — a FGV troca o domínio ou atribui um objetivo ao domínio errado. No MPU, “especificar e priorizar requisitos com base na experiência do usuário” era **BAI**, não APO nem DSS. Lembre também que o COBIT define **componentes** para construir/sustentar um sistema de governança (gabarito no TJ-RJ) — não uma “estratégia de TI” nem um “inventário de ativos”.

## 12.5 BPMN (modelagem de processos)

Notação para processos de negócio. Elementos-chave:

- **Eventos** (círculos): início (borda fina), intermediário (borda dupla), fim (borda grossa).
- **Atividades** (retângulos arredondados): tarefas e subprocessos.
- **Gateways** (losangos): decisões/desvios (exclusivo XOR, paralelo AND, inclusivo OR).
- **Raias (pools/lanes)**: quem executa (papéis/organizações).
- **Fluxos**: de sequência (sólido) × de mensagem (tracejado).

#### Os três gateways — e o que cada um faz na volta

O gateway é o único elemento do BPMN com **duas** semânticas: uma quando divide o fluxo e outra quando o junta. A banca cobra a segunda, que quase ninguém estuda.

| Gateway                                       | Na divisão                                                                | Na junção                                                                   |
|-----------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Exclusivo</b><br>(XOR, losango ou com “X”) | segue por <b>exatamente um</b> caminho, o da primeira condição verdadeira | apenas <b>repassa</b> o que chegar — não espera nada                        |
| <b>Paralelo</b> (AND, losango com “+”)        | dispara <b>todos</b> os caminhos ao mesmo tempo, sem avaliar condição     | <b>sincroniza</b> : espera <b>todos</b> os caminhos chegarem para continuar |
| <b>Inclusivo</b> (OR, losango com “O”)        | segue por <b>um ou mais</b> , todos os que tiverem condição verdadeira    | espera só os caminhos que <b>foram efetivamente ativados</b>                |



**O gatilho no enunciado:** “as duas análises ocorrem simultaneamente e o processo só avança quando **ambas** terminarem” é paralelo (AND) — tanto na abertura quanto no fechamento. “Ou aprova, ou recusa” é exclusivo. “Podem ser necessárias uma, duas ou as três verificações” é inclusivo, que é o único que decide na junção quantos esperar. Trocar o AND de fechamento pelo XOR é o erro de modelagem clássico: o processo segue adiante com metade do trabalho feito.

#### ATENÇÃO — Como a FGV arma a pegadinha

Troca do símbolo pelo espessura da borda (início↔fim) ou gateway × atividade. No CBOK/BPMN, cuidado com *swim lanes/handoffs* e gateway exclusivo × paralelo.

#### O que já caiu nas provas

**Em prova real da FGV:** o domínio **BAI** do COBIT 2019 (“especificar e priorizar requisitos com base na experiência do usuário”) — **MPU**; a aplicação do COBIT 2019 como framework para **definir os componentes** de um sistema de governança, e o **propósito** da prática ITIL de gerenciamento de infraestrutura e plataforma (monitorar as soluções tecnológicas) — **TJ-RJ**; os princípios do **PMBOK 7** aplicados a cenário — *adaptação e resiliência* e *pensamento sistêmico* — **TJ-RJ**.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): ágil híbrida (combinar); eventos do Scrum e o time-box de **15 min** da Daily; Kanban (fluxo contínuo, limite de WIP); **lead time** × cycle time; tripla restrição; ITIL 4 (4 dimensões, SVS, 6 atividades da cadeia de valor, 7 princípios, incidente × problema); os 5 domínios do COBIT; símbolos do BPMN; os 5 grupos de processos do PMBOK 6; projeto × programa × portfólio.

Rode `./quiz.py governanca`.

## 12.6 Alta probabilidade / pesquisa extra

- **ITIL 4** ainda é o vigente; fique atento ao vocabulário novo (práticas, SVS, cocriação de valor).
- **COBIT 2019** usa “objetivos de governança e gestão” (não “processos” do COBIT 5).
- **Gestão de riscos (ISO 31000)**: identificar, analisar, avaliar, tratar, monitorar.

#### Como se sair melhor

COBIT: fixe o mapa dos 5 domínios e a linha divisória governança (EDM) × gestão (APO/BAI/DSS/MEA). ITIL 4: memorize o “propósito” de cada prática numa frase — a FGV cobra exatamente essa frase. PMBOK 7: memorize os 12 princípios pelo nome; associe cada cenário a UM princípio, não a uma fase. Trocas de quantidade a decorar: 5 domínios COBIT, 4 dimensões ITIL, 5 grupos de processos PMBOK 6, 12 princípios PMBOK 7.

## Capítulo 13

# Redes de Computadores

**ATENÇÃO — Fora do edital do Perfil 3, mas cai — não pule este capítulo**

Redes de Computadores **não está** no conteúdo do Perfil 3 (está nos Perfis 2 e 5). O edital do Perfil 3 só cita HTTPS e SSL/TLS. Mesmo assim, a **Dataprev 2024 trouxe ~3 questões de redes** para Desenvolvimento de Software (modelo OSI, protocolo, segurança de rede). Ignorar isso custou 7,5 pontos. Estude o essencial abaixo — é ponto barato e seguro.

### 13.1 Modelos de referência

**OSI (7 camadas)** — de baixo para cima:

| # | Camada       | Função                     | Exemplos         |
|---|--------------|----------------------------|------------------|
| 1 | Física       | bits no meio               | cabos, sinais    |
| 2 | Enlace       | quadros, MAC               | switch, Ethernet |
| 3 | Rede         | pacotes, IP, roteamento    | roteador, IP     |
| 4 | Transporte   | fim a fim, portas          | TCP, UDP         |
| 5 | Sessão       | diálogo, checkpoints       | —                |
| 6 | Apresentação | formato, cifra, compressão | TLS (discutível) |
| 7 | Aplicação    | serviços ao usuário        | HTTP, DNS, SMTP  |

**TCP/IP (4 camadas):** Acesso à Rede, Internet (IP), Transporte (TCP/UDP), Aplicação.

**Encapsulamento** — a ideia única que faz o modelo em camadas funcionar

Camada não é gaveta de classificação: é **divisão de trabalho**. Cada camada resolve *um* problema, entrega o resultado à de baixo e **acrescenta o próprio cabeçalho** — na volta, cada uma retira o seu. Por isso o nome da unidade de dados denuncia a camada, e isso responde item de prova direto:

| Camada     | Unidade (PDU)                                 | Problema que ela resolve                                         |
|------------|-----------------------------------------------|------------------------------------------------------------------|
| Aplicação  | mensagem/dados                                | o que os dois programas querem dizer                             |
| Transporte | <b>segmento</b> (TCP)<br>/<br>datagrama (UDP) | entregar ao <b>processo</b> certo (porta), na ordem e sem perda  |
| Rede       | <b>pacote</b>                                 | achar o caminho <b>entre redes</b> diferentes (IP, roteamento)   |
| Enlace     | <b>quadro</b>                                 | entregar ao vizinho <b>dentro do mesmo segmento</b> físico (MAC) |
| Física     | bits                                          | pôr o sinal no meio                                              |

**O critério de decisão**, que é como a FGV cobra (por função, nunca pelo número): o problema é *entre duas máquinas da mesma rede local?* Enlace. *Entre redes?* Rede. *Chegou à máquina certa mas não ao programa certo*, ou faltou ordem/retransmissão? Transporte. *Diálogo, retomada, checkpoint?* Sessão. *Formato, cifra, compressão?* Apresentação.

Os dois modelos não competem: o OSI de 7 camadas é o **referencial de ensino**, e a pilha realmente implementada é a **TCP/IP** de 4 — que junta OSI 5–7 em “Aplicação” e OSI 1–2 em “Acesso à Rede”. Quando o enunciado cita sessão ou apresentação, está falando de OSI.

#### ATENÇÃO — Como a FGV arma a pegadinha

Modelo OSI é cobrado por **função**, não por número: o cenário descreve o comportamento e pede a camada. Ex.: “checkpoints no fluxo + controle de diálogo (simplex/half/full-duplex)” = camada de **Sessão** — não Transporte, Enlace ou Apresentação, que são os distratores comuns. Troca clássica de elemento por camada: switch = 2 (enlace), roteador = 3 (rede).

## 13.2 Equipamentos

**Hub** (camada 1, burro, repete para todos) × **Switch** (camada 2, usa MAC, comuta para a porta certa) × **Roteador** (camada 3, usa IP, interliga redes). Existe switch L3 (roteia), mas o par clássico cobrado é switch=2, roteador=3.

## 13.3 TCP × UDP

|                | TCP                                    | UDP                  |
|----------------|----------------------------------------|----------------------|
| Conexão        | orientado a conexão (handshake 3 vias) | sem conexão          |
| Confiabilidade | garante entrega e ordem                | não garante          |
| Uso            | web, e-mail, transferência             | streaming, DNS, VoIP |

**ATENÇÃO — Como a FGV arma a pegadinha**

O **three-way handshake** do TCP **sincroniza números de sequência** (SYN / SYN-ACK / ACK). Os distratores dizem que ele “autentica dispositivos”, “negocia segurança” ou “controla fluxo” — nenhum dos três é o papel do handshake. Handshake existe **só no TCP**.

## 13.4 Protocolos e portas

| Protocolo   | Porta         | Função               |
|-------------|---------------|----------------------|
| HTTP        | 80            | web                  |
| HTTPS       | 443           | web seguro (TLS)     |
| FTP         | 21 (20 dados) | transferência        |
| SSH         | 22            | acesso remoto seguro |
| SMTP        | 25            | envio de e-mail      |
| DNS         | 53            | nomes → IP           |
| POP3 / IMAP | 110 / 143     | leitura de e-mail    |
| DHCP        | 67/68         | IP automático        |

*Observação:* porta específica não apareceu na amostra de provas reais analisadas; a FGV preferiu raciocínio de rota, camada e ACL. Está no edital, mas priorize os padrões desta seção.

## 13.5 Endereçamento e roteamento IP

**IPv4:** 32 bits. Faixas privadas (RFC 1918): 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. **IPv6:** 128 bits; prefixo de sub-rede típico /64.

**Roteamento:** rota estática × dinâmica (OSPF, RIP, BGP). **Distância administrativa** (Cisco) desempata a fonte da rota: conectada 0, estática 1, OSPF 110, RIP 120 — **menor** vence. Com duas rotas default, ganha a estática (1) sobre OSPF (110).

**ATENÇÃO — Como a FGV arma a pegadinha**

Absolutos em roteamento denunciam o distrator: “OSPF SEMPRE tem prioridade sobre rota estática”, “roteador faz load balancing AUTOMÁTICO”, “descarta por rotas conflitantes”. A regra real é determinística: menor distância administrativa vence, sempre.

### 13.5.1 Sub-redes: máscara × hosts utilizáveis

A conta é sempre a mesma: com  $h$  bits sobrando para host, **hosts utilizáveis** =  $2^h - 2$  (descontam-se o endereço de **rede** e o de **broadcast**). O prefixo  $/n$  deixa  $h = 32 - n$ .

| Prefixo | Máscara         | Bits de host | Hosts utilizáveis |
|---------|-----------------|--------------|-------------------|
| /24     | 255.255.255.0   | 8            | 254               |
| /25     | 255.255.255.128 | 7            | 126               |
| /26     | 255.255.255.192 | 6            | 62                |
| /27     | 255.255.255.224 | 5            | <b>30</b>         |
| /28     | 255.255.255.240 | 4            | 14                |

O enunciado costuma dar a necessidade (“no máximo 30 hosts”) e pedir a máscara: procure a **menor** sub-rede que **comporta** o número pedido — 30 hosts cabem em /27 exatamente, então /26 (62) desperdiça e /28 (14) não cabe.

**IPv6 — notação simplificada.** Duas regras, nesta ordem: (1) apague os **zeros à esquerda** de cada grupo (0DB8 → DB8, 0001 → 1); (2) substitua **uma única** sequência de grupos inteiramente nulos por :: — só uma vez no endereço, senão fica ambíguo. Assim 2001:0DB8:0000:0000:0000:0000:FE00:0001 vira 2001:DB8::FE00:1.

### 13.5.2 ACL Cisco e wildcard mask

Wildcard mask = inverso da máscara; bit **0** = “tem que bater”, bit **1** = “ignora”. Para casar só endereços ímpares, o bit menos significativo do octeto precisa estar fixo em 1 → wildcard 0.0.0.254 (deixa livres os demais bits, fixa o último). Confira montando os bits, não de cor.

### 13.5.3 MPLS, VLAN, NAT

- **MPLS (Multiprotocol Label Switching):** encaminha por **rótulos (labels)** em vez de olhar o IP de destino a cada salto — por isso é chamado de “**camada 2.5**” (entre enlace/L2 e rede/L3). Cria caminhos (LSP) para engenharia de tráfego e QoS.
- **VLAN:** segmenta logicamente a camada 2 (broadcast domains separados no mesmo switch físico).
- **NAT:** traduz endereços privados ↔ público (economiza IPv4, oculta a rede interna).

#### ATENÇÃO — Como a FGV arma a pegadinha

MPLS é **rótulo** (“2.5”), não roteamento IP puro; VLAN segmenta **L2**. Não confundir os três termos entre si.

## 13.6 Tipos de rede corporativa

Internet (pública global) × intranet (interna) × extranet (acesso controlado a parceiros/clientes externos) × portal (ponto único de acesso) — ver também Capítulo 5.

#### ATENÇÃO — Como a FGV arma a pegadinha

Inverter intranet/extranet/Internet: chamar intranet de “rede pública global”, extranet de “só interna”, Internet de “rede restrita” — os três papéis trocados são o distrator clássico.

## 13.7 Segurança de rede (interface com Segurança da Informação)

**Firewall** (filtra tráfego), **proxy**, **VPN** (IPsec, SSL VPN). **IDS** (detecta/alerta) × **IPS** (bloqueia) — ver Capítulo 8. **SSL** × **TLS** × **HTTPS** e **X.800** (mecanismos de segurança OSI) também são fronteira com segurança.

### 13.7.1 Firewall stateful × stateless

|                | Stateless (filtro de pacotes)                                            | Stateful (com estado)                                                    |
|----------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------|
| O que olha     | cada pacote <b>isoladamente</b> : IP de origem/destino, porta, protocolo | o pacote <b>no contexto da sessão</b>                                    |
| Como decide    | só pela regra estática                                                   | mantém uma <b>tabela de estado</b> das conexões ativas                   |
| Efeito prático | para liberar a resposta é preciso escrever a regra de volta à mão        | a resposta de uma conexão iniciada de dentro já é aceita automaticamente |

Os dois atuam nas camadas 3 e 4. Quem sobe para a camada 7 (inspeciona o conteúdo da aplicação) é o **firewall de próxima geração (NGFW)** ou o **WAF**, que são outra coisa.

### 13.7.2 SSH × Telnet e o handshake do TLS

**Telnet** (porta **23**) trafega **tudo em texto claro**, inclusive a senha do administrador — quem estiver no caminho lê a sessão inteira. O **SSH** (porta **22**) faz o mesmo trabalho de administração remota **cifrando toda a comunicação, inclusive a autenticação**. É o exemplo canônico de protocolo inseguro aposentado por um seguro, e a razão da substituição é **criptografia**, não velocidade.

No **HTTPS**, o TLS usa criptografia **híbrida**, e a divisão de trabalho é o que a banca cobra:

- **Assimétrica** — **só no handshake**. O certificado digital do servidor autentica quem ele diz ser, e o par de chaves serve para as duas pontas **acordarem com segurança uma chave simétrica de sessão**.
- **Simétrica** — **no resto**. Todo o volume de dados da sessão é cifrado com essa chave simétrica, que é **muito mais rápida**.

A assimétrica é lenta demais para cifrar o tráfego inteiro; a simétrica não resolveria sozinha o problema de *como* combinar a chave por um canal inseguro. Daí o híbrido.

#### ATENÇÃO — Como a FGV arma a pegadinha

“SSL mais seguro/moderno que TLS” está **invertido** — TLS sucedeu e corrigiu o SSL. No handshake TLS, o distrator diz que a assimétrica “cifra todo o tráfego da sessão” (não: ela só **troca a chave**), que ela “autentica o usuário final por login e senha” (não: autentica o **servidor**, por certificado) ou que “dispensa os certificados” (é justamente o contrário). Em firewall, o distrator descreve o **stateless** e chama de stateful (“analisa apenas os cabeçalhos de cada pacote isoladamente”) — inversão de par pura. Outro: dizer que o stateful “atua exclusivamente na camada de aplicação” — absoluto e errado, isso é NGFW/WAF. Em SSH, o distrator inverte o motivo: “é mais rápido por não usar criptografia” ou “transmite as credenciais em texto claro” — essa é a descrição do **Telnet**.

### O que já caiu nas provas

**Em prova real da FGV:** camada OSI **por função** — sessão e *checkpoints* na retomada de uma transferência interrompida —, **rota estática** × **OSPF** pela distância administrativa e **ACL Cisco com wildcard mask**; ainda volume anômalo de handshakes TCP (SYN flood) — **TJ-RJ**. Camada OSI para **diagnosticar** falha entre servidor e switch; **switch** × **roteador** × **hub/repetidor**; **TCP** × **UDP** (entrega ordenada e garantida); **RFC 1918** e **NAT**; **IPv6** (abreviação do endereço); **cálculo de sub-rede** a partir de um /24; **MPLS**; meio físico, topologia e Wi-Fi; **SSH substituindo o Telnet** — **ALERO 2026**. **Internet** × **intranet** × **extranet** × **portal** e **X.800** (mecanismos de segurança do modelo OSI) — **Dataprev 2024**. **No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): **número de porta específico** (80/443, DNS 53, DHCP) — veja a observação da Seção de protocolos, a FGV preferiu raciocínio de rota, camada e ACL; o **handshake híbrido do TLS**; **firewall stateful** × **stateless**; e **VLAN** como segmentação de domínio de broadcast. Rode `./quiz.py redes`.

### Como se sair melhor

Não vale estudar redes com a profundidade dos Perfis 2/5. Cubra: OSI/TCP-IP, equipamentos por camada, TCP×UDP, portas comuns, IP público×privado, IDS×IPS. Isso resolve o padrão histórico da FGV para o perfil de desenvolvimento com pouco tempo investido. OSI de cima pra baixo, para decorar rápido: Aplicação, Apresentação, Sessão, Transporte, Rede, Enlace, Física.

## Capítulo 14

# Órfãos — Administração de BD e Temas Coringa

### O que é este capítulo

Bloco “coringa” do quiz: o que não encaixa nos outros capítulos. Na prática, a maior parte das questões daqui é **administração de banco de dados (DBA)** e **administração de dados** — conteúdo do edital (“noções de administração de dados e de banco de dados; backup, restauração; otimização de performance”) que o capítulo de Banco de Dados, focado em modelagem/SQL, não detalha. O resto são temas avulsos: arquitetura de computadores, informática, sistemas de informação, IA/ML e blockchain. Nas provas recentes a FGV vem carregando em IA/ML e cloud — trate como tema quente.

## 14.1 Administração de Dados × Administração de Banco de Dados

|       | Administração de Dados (AD)                                      | Administração de BD (DBA)                              |
|-------|------------------------------------------------------------------|--------------------------------------------------------|
| Foco  | <b>negócio/lógico:</b> o dado como recurso corporativo           | <b>técnico/físico:</b> o SGBD funcionando              |
| Faz   | modelagem conceitual, políticas, governança, dicionário de dados | instalação, tuning, backup, segurança, disponibilidade |
| Papel | estratégico, independente de SGBD                                | operacional, ligado ao produto (Oracle, etc.)          |

### ATENÇÃO — Como a FGV arma a pegadinha

AD é conceitual/negócio; DBA é físico/operacional. A FGV troca os dois papéis entre si.

## 14.2 Recuperação e transações (ARIES)

**ARIES** (protocolo de recuperação da maioria dos SGBDs) usa **WAL** (Write-Ahead Logging — o log vai para disco **antes** do dado) e três fases na recuperação após falha, **nesta ordem**:

1. **Analysis** (Análise): reconstrói o estado a partir do último checkpoint.
2. **Redo** (Refazer): reaplica **todas** as operações registradas (até as de transações que não commitaram) — “repeating history”.
3. **Undo** (Desfazer): desfaz as transações que não commitaram.

WAL garante a durabilidade (D do ACID) sem gravar o dado imediatamente.



**ATENÇÃO — Como a FGV arma a pegadinha**

Ordem Analysis → Redo → Undo — a FGV inverte Redo/Undo.

### 14.3 Armazenamento físico e desempenho

- **Tablespace** (Oracle) / **filegroup**: unidade lógica de armazenamento que agrupa objetos em arquivos físicos. Separa dados, índices, temporário.
- **Índices** (B+ Tree, bitmap, hash): aceleram busca ao custo de escrita. Bitmap é bom para **baixa cardinalidade** (poucos valores distintos); B+ Tree para alta cardinalidade e faixas.
- **Otimizador de consultas (query optimizer)**: escolhe o **plano de execução** (ordem de joins, uso de índice). Planos ruins geralmente vêm de **estatísticas desatualizadas** — atualizar estatísticas costuma resolver.
- **Particionamento** (horizontal/sharding, vertical): divide tabela grande para escalar e melhorar desempenho.

### 14.4 Backup e recuperação

| Tipo            | O que copia                                                    |
|-----------------|----------------------------------------------------------------|
| Completo (full) | tudo                                                           |
| Incremental     | só o que mudou <b>desde o último backup (de qualquer tipo)</b> |
| Diferencial     | tudo que mudou <b>desde o último backup completo</b>           |

**RPO** (Recovery Point Objective): quanto de dado se aceita perder (janela). **RTO** (Recovery Time Objective): em quanto tempo o serviço volta. Backup **quente** (online, banco no ar) × **frio** (offline).

**ATENÇÃO — Como a FGV arma a pegadinha**

Incremental (desde o último **qualquer**) × diferencial (desde o último **completo**) — a diferencial cresce até o próximo full. A FGV troca essa definição com frequência.

### 14.5 Segurança e acesso no SGBD

**GRANT/REVOKE** (DCL), **roles/papéis** (RBAC), **views** para restringir colunas, *row-level security*, auditoria. Princípio do **menor privilégio**: por exemplo, o gerente de RH acessa só o que precisa.

### 14.6 Temas coringa genuínos

- **Arquitetura de computadores**: CPU (Unidade de Controle × ULA), ciclo de instrução (busca-decodifica-executa), pipeline, memória (registrador → cache → RAM → disco, mais rápida e cara no topo), sistemas de numeração (binário, octal, hexadecimal — conversão dígito a dígito).

- **Noções de informática:** pacote office, sistemas de arquivos, atalhos.
- **Sistemas de informação:** SIG, SPT (transacional), SAD/SSD (decisão), ERP, CRM, SCM — o nível gerencial que cada um atende.

## 14.7 Onde cada tema coringa está detalhado

Este é o capítulo mais transversal do material: como o quiz aponta o capítulo pela **tag** da questão, tudo que é **orfaos** cai aqui — mesmo quando o conteúdo mora, com razão, em outro capítulo. Use este mapa em vez de procurar de página em página.

| Tema                                                                                         | Onde está                                  |
|----------------------------------------------------------------------------------------------|--------------------------------------------|
| Blockchain (hash do bloco anterior, raiz de Merkle, <i>nonce</i> , imutabilidade enca-deada) | Capítulo 9                                 |
| Servidor web × servidor de aplicação                                                         | Capítulo 5, primeira seção                 |
| 2PC (two-phase commit) e o contraste com Raft                                                | Capítulo 5                                 |
| Virtualização: hipervisor Tipo 1 × Tipo 2 × con-têiner                                       | Capítulo 5                                 |
| Sistema operacional: pa-ginação/memória virtual, E/S bloqueante, troca de contexto           | Capítulo 5                                 |
| Low-code / no-code                                                                           | Capítulo 9                                 |
| RPA (Robotic Process Au-tomation)                                                            | Capítulos 3 e 9                            |
| Os Vs do Big Data                                                                            | Capítulo 6                                 |
| Internet × intranet × ex-tranet × portal                                                     | Capítulo 13 (e a pegadinha neste capítulo) |
| IA generativa, LLM, RAG, ética e regulação                                                   | Capítulo 18                                |

Três desses valem uma frase aqui, porque são exatamente o que a questão cobra e o candidato costuma resolver no reflexo errado:

- **2PC:** um **coordenador** pergunta a todos os participantes se podem confirmar (fase *prepare*) e só ordena o *commit* com **unanimidade** — um único “não” aborta a transação inteira. Não é maioria simples (isso é consenso, tipo Raft), nem confirmação independente de cada nó.
- **RPA:** software que **imita o usuário** (clique, digitação, leitura de tela) em tarefas digitais repetitivas e **baseadas em regras**. Não é IA, não é robô físico, não substitui banco de dados.
- **Low-code:** desenvolvimento por **modelagem visual e configuração**, com o **mínimo** de código manual — *no-code* elimina o código de vez. Nenhum dos dois “dispensa desenvolvedor profissional”: esse absoluto é o distrator.

## 14.8 IA/ML (tema quente neste bloco)

- **Supervisionado × não supervisionado × semisupervisionado.** Supervisionado usa dados **rotulados** (classificação/regressão); não supervisionado acha estrutura **sem rótulo** (clusterização/associação). A FGV inverte “rotulado” entre os dois.
- **Overfitting** (decora o treino, vai mal em dados novos) × **underfitting** (modelo simples demais). Regularização combate **overfitting**.
- **Regularização: L1 (Lasso)** zera coeficientes e faz seleção de atributos; **L2 (Ridge)** encolhe sem zerar. A banca troca os dois.
- **Concept drift** (muda a relação entrada→saída, o próprio conceito alvo) × **data drift** (muda a distribuição da **entrada**). Fácil de trocar.
- **Métricas:** MAE = erro médio absoluto (regressão); F1 = média harmônica de precisão e recall; ROC/AUC = capacidade de separar classes variando o limiar.
- **Redes neurais** (ativação sigmoide) e CNN; **LLM/IA generativa:** RAG, agentes, engenharia de prompt, ética e explicabilidade — ver também Capítulo 18.
- Em cenário de IA em produção, “monitorar drift” e deploy blue-green/canário são as respostas típicas de **MLOps**. Não confundir IA generativa (cria conteúdo) com IA discriminativa (classifica/prediz).

### ATENÇÃO — Como a FGV arma a pegadinha

Inverter hipervisor: **Tipo 1** (bare-metal) roda direto no hardware; **Tipo 2** roda sobre um SO hospedeiro; **contêiner** compartilha o kernel do host (não virtualiza hardware). Troca internet/intranet/extranet: intranet = interna; extranet = estende a intranet a parceiros externos.

### O que já caiu nas provas

**Em prova real da FGV: 21 questões, e nenhuma delas é conteúdo do Perfil 3.** Depois que a classificação foi consertada, este bloco virou o que o nome sempre prometeu — o que não tem casa — e ele se divide em duas famílias, as duas fora do edital:

- **Arquitetura de computadores e sistema operacional (11).** Gargalo de **Von Neumann**, ULA e unidade de controle, *overflow × carry*, complemento de dois, conversão de base, ROM/firmware, periférico, ciclo busca-decodificação-execução, **MMU/TLB** e *thrashing*, tabela de páginas, **DMA** — **ALERO 2026** e **TJ-RJ**. O edital do Perfil 3 fala em arquitetura *de software* (Capítulo 5).
- **Administração e direito público (10).** Avaliação de desempenho de gestores, priorização de melhoria de processo, liderança de equipe, departamentalização, Resolução CNMP, redistribuição de cargo, compromisso da LINDB, critério de desempate em licitação, responsabilidade da concessionária e processo disciplinar — **MPU**, tudo do Módulo I daquele concurso.

**O que saiu daqui.** Até o fechamento do ITEM 7 este bloco declarava 68 questões e “maioria esmagadora DBA”. Era sintoma de um defeito do importador: o rótulo de sub-bloco dos mapas da ALERO usava o *slug* (**banco-dados, eng-software**), que a tabela de tags não reconhecia, e 47 questões caíam aqui por engano. Foram para onde sempre pertenceram — **34 para banco de dados** (Capítulo 6) e **13 para engenharia de software** (Capítulo 3). O *conteúdo* de administração física (ARIES, B+ Tree, tablespaces, otimizador, backup) continua sendo ensinado neste capítulo, e o Capítulo 6 remete para cá.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): blockchain pelo

encadeamento de hash; RPA; low-code; os Vs do Big Data; supervisionado × não supervisionado. Dois itens que já estiveram nesta lista **caíram** de fato, só que sob outra tag na **Dataprev 2024: servidor web** × **servidor de aplicação** (Capítulo 5) e **internet/intranet/extranet/portal** (item de redes, Capítulo 13).

Regra prática: o que este capítulo *ensina* continua valendo muito — administração física de banco de dados é matéria de prova, e as questões dela agora estão sob **banco-dados**. O que ficou com a etiqueta de órfã é material de outro perfil: reconheça e siga em frente. Memorize os pares: AD × DBA, incremental × diferencial, RPO × RTO, ordem do ARIES, bitmap × B+ Tree, rotulado × não rotulado, data × concept drift, L1 × L2, Tipo 1 × Tipo 2, over × underfitting.

Rode `./quiz.py orfaos`.

### Como se sair melhor

Monte flashcards dos pares que a FGV inverte neste bloco (listados acima) e revise-os isolados dos outros capítulos — é o jeito mais rápido de fixar um bloco que mistura tantos assuntos diferentes.

## Parte II

### Módulo I — Conhecimentos Gerais (peso 1, 40 questões)

## Capítulo 15

# Língua Portuguesa

### Ficha do edital

Compreensão e interpretação de textos; tipos e gêneros textuais; ortografia; coesão (referenciação, conectores, tempos/modos verbais); estrutura morfosintática (classes de palavras, coordenação, subordinação, pontuação, concordância verbal e nominal, regência, crase, colocação pronominal); reescrita e significação.

**Peso esperado: 12 questões, peso 1 — mesmo peso do Inglês. Interpretação é o eixo. Na Dataprev 2024 foram as questões Q1–Q12; o carro-chefe é gramática aplicada e interpretação de frase curta, não redação nem texto longo.**

### 15.1 Onde a prova se concentra

A FGV mistura **interpretação** com **gramática aplicada** (não gramática decorada e solta). Prioridades:

1. **Interpretação e reescrita** (o maior naipe): sentido do texto, do trecho, de conectivos; substituição de palavras/trechos mantendo o sentido.
2. **Sintaxe do período**: coordenação × subordinação, função dos termos (sujeito, adjunto adnominal × complemento nominal), orações reduzidas.
3. **Concordância e regência**: verbal e nominal; crase.
4. **Coesão**: referenciação (pronomes, elipse), conectores e o que eles expressam (causa, consequência, concessão, condição).
5. **Pontuação**: justificar o uso da vírgula (aposto, adjunto deslocado, oração subordinada, enumeração).

### 15.2 Pares e conceitos que a FGV cobra

- **Adjunto adnominal** (liga-se a substantivo, indica posse/qualidade) × **complemento nominal** (completa o sentido de nome, é alvo da ação).
- **Oração subordinada substantiva** (função de substantivo: sujeito, objeto) × **adjetiva** (função de adjetivo, restritiva × explicativa) × **adverbial**.
- **Oração reduzida** (verbo no infinitivo/gerúndio/particípio, sem conectivo) — desenvolvê-la revela a função.
- **Regência**: “obedecer **a**”, “assistir **a**” (ver), “lembrar-se **de**”, “residir **na/à**” — verbos que a FGV adora com regência trocada.
- **Concordância**: “anexo/anexa” concorda; “é proibido/proibida a entrada” (varia com o determinante); “meio” advérbio é invariável (“meio confusa”).
- **Discurso direto × indireto**: o direto reproduz a fala (aspas, travessão, verbo de elocução); não “demarca a soberania do narrador”.

### 15.3 Concordância verbal — os verbos impessoais

A concordância **nominal** (anexo, proibido, meio) é a mais lembrada, mas a **verbal** cai tanto quanto, e quase sempre pelo mesmo grupo de verbos **impessoais** — os que **não têm sujeito** e por isso ficam travados na 3ª pessoa do singular:

| Verbo                   | Quando é impessoal                   | Certo (e o erro planejado)                                                |
|-------------------------|--------------------------------------|---------------------------------------------------------------------------|
| <b>fazer</b>            | tempo decorrido                      | <i>Faz</i> dois anos — nunca “ <i>Fazem</i> dois anos”                    |
| <b>haver</b>            | no sentido de “existir” ou tempo     | <i>Havia</i> muitas falhas — nunca “ <i>Haviam</i> muitas falhas”         |
| <b>haver</b> + auxiliar | o auxiliar também trava              | <i>Deve haver</i> relatórios — nunca “ <i>Devem haver</i> ”               |
| <b>existir</b>          | <b>não</b> é impessoal — tem sujeito | <i>Existem</i> muitos processos — nunca “ <i>Existe</i> muitos processos” |

O corte é esse: **haver** (existir) não tem sujeito e não varia; **existir** tem sujeito e varia. É a inversão que a banca monta.

### 15.4 Por que, porque, por quê, porquê

| Forma          | Quando                                                                  | Exemplo                                                                 |
|----------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>por que</b> | pergunta (direta ou indireta) = “por qual motivo/razão”                 | <i>Por que</i> o sistema caiu? / Não sei <i>por que</i> o sistema caiu. |
| <b>porque</b>  | resposta, causa = “pois”                                                | Caiu <i>porque</i> faltou energia.                                      |
| <b>por quê</b> | mesma coisa que “por que”, mas <b>no fim</b> da frase ou antes de pausa | O sistema caiu <i>por quê</i> ?                                         |
| <b>porquê</b>  | substantivo — vem com artigo/determinante                               | Ele explicou o <i>porquê</i> da falha.                                  |

Regra prática de três passos: (1) dá para trocar por “por qual motivo”? → **por que** separado; (2) está no **fim** da frase? → ganha acento: **por quê**; (3) tem **artigo** antes (o, um, esse)? → junto e com acento: **porquê**. Sobrou? é o **porque** de causa.

Atenção à armadilha mais comum: em **interrogativa indireta** (“Não sei por que...”, “Pergunto por que...”) não há ponto de interrogação, e por isso muita gente escreve “porque” — mas continua sendo pergunta, logo **por que**, separado e sem acento.

#### ATENÇÃO — Como a FGV arma a pegadinha

O enunciado é quase sempre no **NEGATIVO**: “assinale a opção **INCORRETA**”, “em que o termo **NÃO** funciona como...”. Metade dos erros é ler correto onde está incorreto — circule a

palavra-chave antes de olhar as alternativas.

Em interpretação, o distrator é a leitura que **EXTRAPOLA** ou **INVERTE** a tese. Ex.: num verso de Pessoa, “visão negativa do fazer poético” ou “torna o poeta imune à dor” — o texto não diz nada disso; o correto é o mais literal (“o fingimento é matéria de poesia”).

Distrator com termo **absoluto**: “É impossível ser feliz e viver”, “Viver pressupõe felicidade”. A frase original só descrevia um paradoxo — desconfie de qualquer opção que radicaliza.

Em gramática, a alternativa errada costuma ter **um erro pontual plantado** (regência trocada, concordância de “meio/anexo/proibido”, crase indevida). O resto da frase parece natural de propósito.

### Como se sair melhor

Volte ao texto e teste cada alternativa contra o que está **ESCRITO**, não contra o que “faz sentido no mundo”. Se a opção precisa de informação que o trecho não dá, ela extrapola: elimine. Elimine primeiro os absolutos (sempre, nunca, impossível, todos) e as leituras que agregam juízo de valor ausente. Revise as regras-armadilha clássicas: concordância de “anexo/proibido/meio” (variam ou não conforme o caso), regência de obedecer/simpatizar/residir/lembrar, e o uso da vírgula (aposto, adjunto deslocado, oração intercalada). Em oração subordinada, saiba nomear rapidamente: substantiva (função de substantivo — sujeito, objeto), adjetiva (delimita/explica um nome), reduzida (verbo no infinitivo/gerúndio/particípio, sem conectivo).

### O que já caiu nas provas

**Em prova real da FGV:** análise sintática de frase curta (“É preciso estar atento e forte”); adjunto adnominal; regência verbal; concordância nominal; interpretação de verso (Fernando Pessoa, Luiz Gonzaga); justificativa da vírgula; discurso direto — **Dataprev 2024** (Q1–Q12). Pleonismo em reescrita; reescrever *mudando* ou *mantendo* o sentido; processos de coesão para evitar repetição; referência de termo de ligação; oração adjetiva trocada por termo equivalente; apassivação; **colocação pronominal** (“Dê-me” × “Me dá”); **regência de *assistir*** — MPU. Português é o maior bloco de todas as provas do corpus (67 questões reais entre Dataprev, MPU e TJ-RJ) e é sempre **interpretação + gramática aplicada**.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): crase; concordância *verbal* dos impessoais; *por que/porque/por quê/porquê*; conjunção concessiva substituível (“embora” → “ainda que”).

Rode `./quiz.py portugues`.

## 15.5 Pontos de atenção / pesquisa extra

- **Crase:** casos obrigatórios (locuções femininas, “à medida que”), proibidos (antes de verbo, masculino, pronomes) e facultativos.
- **Colocação pronominal:** próclise (palavra atrativa: negação, pronome relativo, advérbio), ênclise, mesóclise.



## Capítulo 16

# Língua Inglesa

### Ficha do edital

Compreensão de textos em inglês e itens gramaticais relevantes para o entendimento dos sentidos dos textos.

**Peso esperado: 12 questões, peso 1 — mesmo peso do Português. É quase toda interpretação de texto. Para quem lê documentação técnica, é a disciplina de melhor retorno por hora estudada de toda a prova.**

## 16.1 O que a FGV cobra

Foram 12 questões na Dataprev 2024 (Q13–Q24), todas ancoradas em **textos reais** em inglês (anúncio de e-book, resenha de produto, abstract acadêmico). Um texto serve a 2–6 questões. Os itens que mais aparecem:

1. **Ideia principal / propósito do texto** (“what information is in the text”, “the main purpose is”). Você tem que captar a ideia central e o gênero (é anúncio? resenha? artigo acadêmico?).
2. **Referência de pronome:** a quem/quê o *it, this, they, her* se refere — volte à frase anterior.
3. **Conectivos / discourse markers:** o que eles sinalizam.
4. **Vocabulário em contexto:** sentido de uma palavra no texto; palavra que funciona como verbo × substantivo (ex.: *browse, rate*).
5. **Verdadeiro/falso sobre o texto** (“only 1 and 4 are true”).

## 16.2 Conectivos que caem muito

| Marcador                                 | Sentido                         |
|------------------------------------------|---------------------------------|
| thus, therefore, hence, so               | consequência/efeito             |
| however, whereas, but, although, despite | contraste/concessão             |
| moreover, furthermore, and, besides      | adição                          |
| because, since, as                       | causa                           |
| unless                                   | a menos que (condição negativa) |
| meanwhile                                | enquanto isso (tempo)           |

Exemplo que já caiu: “*Thus*” introduz efeito/consequência; “*and*” como adição  $\approx$  *moreover*.

## 16.3 Modais e tempos (para o sentido)

**should** = sugestão/conselho; **must** = obrigação; **can/could** = possibilidade/permissão; **may/might** = possibilidade. O foco não é decorar regra, mas entender o **sentido** que o item dá ao texto.

### ATENÇÃO — Como a FGV arma a pegadinha

Formato “julgue as afirmativas 1–4” e escolha o conjunto verdadeiro/falso. O distrator inverte um detalhe: “It is a printed book” quando o texto diz “eBook Kindle only”, ou “tips only for fresh graduates” quando o texto atende também a quem tem 20 anos de experiência.

A alternativa de compreensão global mais **longa e bem escrita** costuma ser a certa, mas cuidado: a errada adiciona uma palavra que o texto não sustenta (“abroad”, “internship”, “social status”, “web developers”) — extrapolação clássica, igual à do Português.

No pronome, o distrator oferece o substantivo mais **próximo** ou mais “óbvio”, não o correto. “her” pode se referir a “mom”, não a students/professors só porque estão perto no texto.

No conector, a banca confunde causa com contraste. “Thus” = **effect** (consequência), não example nem condition. Decore os pares: thus/hence/so (consequência), whereas/while (contraste), moreover/furthermore (adição), unless (condição negativa).

### Como se sair melhor

É prova de **LEITURA**, não de conversação. Não precisa entender cada palavra; localize a informação que a questão pede e releia **só** aquela frase. Para pronome/vocabulário, volte à linha exata do trecho citado — a resposta está no texto, nunca no seu palpite. Elimine alternativas que trazem informação **não mencionada** (mesmo que plausível no mundo real); se o texto não afirma, é falsa — mesma técnica do português. Monte uma minitabela mental de conectivos e de modais antes da prova; são pontos quase gratuitos e caem sempre. Nas questões de V/F com afirmativas numeradas, resolva **uma afirmativa por vez** e vá cortando as combinações de resposta.

### O que já caiu nas provas

**Em prova real da FGV:** ideia principal do texto (guia de carreira em TI); referência do pronome *her*; conector *thus* (efeito); *and*  $\approx$  *moreover*; *should* (sugestão); *browse/rate* como verbo  $\times$  substantivo; referência de *it* (impressora) — **Dataprev 2024** (Q13–Q24). Vale notar: a Dataprev 2024 é a **única prova do corpus com Língua Inglesa** — MPU, TJ-RJ e ALERO 2026 não têm o bloco. Toda a calibração de estilo do capítulo vem, portanto, dessas 12 questões. **No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): textos técnicos sobre nuvem, IA na programação, trabalho remoto, *phishing* e IA no setor público, com o mesmo desenho — ideia principal, referência, conectivo e vocabulário em contexto. Os dois lotes mais recentes partem de **texto real**, no mesmo perfil de fonte que a Dataprev 2024 usou: um *abstract* do **arXiv** sobre o efeito medido da IA na produtividade de desenvolvedores (*forecast*  $\times$  *estimate*, *Surprisingly* como contraexpectativa, *This slowdown* como anáfora) e um post do **Cisco Talos** sobre uso de ferramenta de IA em campanha de *phishing* contra a administração pública (*Notably* como realce, *This incident* como anáfora, *lower the barrier to entry* como vocabulário em contexto). A fonte completa de cada texto está no **why** da primeira questão do bloco.

Rode `./quiz.py ingles`.

## 16.4 Pontos de atenção / pesquisa extra

- **Vocabulário técnico de TI** ajuda muito (deploy, framework, request, query) — você já tem vantagem por estudar a área.
- **Skimming** (ideia geral) e **scanning** (achar informação específica) são as técnicas de leitura para ganhar tempo.

## Capítulo 17

# Raciocínio Lógico Matemático

### Ficha do edital

Estruturas lógicas; lógica de argumentação (analogias, inferências, deduções, conclusões); lógica sentencial (proposições, tabelas-verdade, equivalências, diagramas); lógica de primeira ordem; raciocínio lógico envolvendo problemas aritméticos, geométricos e matriciais.

Peso esperado: 5 questões, peso 1.

### ATENÇÃO — Duas frentes: matemática E lógica formal — não leia só uma

Cuidado com uma leitura só. A **Dataprev 2024** foi quase toda **conta** — por isso a fama de “RLM da FGV é matemática”. Mas o **edital 2026** lista a lógica formal com peso próprio: estruturas lógicas, lógica de argumentação, lógica sentencial e **lógica de primeira ordem** (itens 1–4), e só no item 5 os “problemas aritméticos, geométricos e matriciais”. São só 5 questões, então não dá para prever o mix — **não pule a lógica formal apostando no padrão de 2024**. Cubra as duas frentes.

| Assunto                              | Prioridade         | O que revisar                                             |
|--------------------------------------|--------------------|-----------------------------------------------------------|
| Porcentagem / aumentos sucessivos    | altíssima          | fator multiplicativo (1,30 × 1,10), desconto, lucro       |
| Razão e proporção / regra de três    | alta               | direta e inversa, divisão proporcional                    |
| Análise combinatória                 | alta               | arranjo, combinação, permutação, princípio multiplicativo |
| Médias e estatística                 | alta               | média simples, ponderada, mediana, moda                   |
| Lógica: equivalências e argumentação | alta (edital 2026) | condicional, contrapositiva, De Morgan, validade          |
| PA / PG                              | média              | termo geral, soma                                         |
| Geometria plana                      | média              | área, perímetro, Pitágoras                                |
| Matrizes                             | média              | operações, determinante                                   |
| Diagramas lógicos / quantificadores  | média              | todo/algun/nenhum, primeira ordem                         |

## 17.1 Ferramentas mínimas de matemática

- **Aumentos sucessivos:** multiplique os fatores.  $+30\%$  depois  $+10\% \rightarrow 1,30 \times 1,10 = 1,43 \rightarrow$  aumento total **43%** (não 40%). Taxa média mensal: raiz,  $\approx$  **19,6%** (entre 19% e 20%).
- **Divisão proporcional:** reparte na razão das partes (ex.: prejuízo proporcional ao capital investido).
- **Média ponderada:**  $\Sigma(\text{nota} \times \text{peso}) / \Sigma(\text{pesos})$ . Colocar a menor nota no maior peso derruba a média — cuidado com “necessariamente”.
- **Combinação**  $C(n, p) = n! / [p!(n - p)!]$  (ordem não importa)  $\times$  **arranjo**  $A(n, p)$  (ordem importa). Grafo completo de  $n$  vértices tem  $C(n, 2)$  arestas.
- **Juros simples  $\times$  compostos.** Simples: o juro incide **sempre sobre o capital inicial**,  $M = C(1 + i \cdot n)$ . Compostos: o juro incide **sobre o montante anterior** (juro sobre juro),  $M = C(1 + i)^n$ . Em 2 períodos a diferença é exatamente o *juro do juro do primeiro período*. Ex.: R\$ 10.000 a 10% a.a. por 2 anos  $\rightarrow$  simples R\$ 12.000, composto  $10.000 \times 1,1^2 =$  R\$ 12.100; diferença R\$ 100.
- **Progressão aritmética (PA):** termo geral  $a_n = a_1 + (n - 1)r$ ; soma  $S_n = (a_1 + a_n) n / 2$ . **PG:**  $a_n = a_1 \cdot q^{n-1}$ . O distrator clássico usa  $n$  no lugar de  $n - 1$  (com  $a_1 = 7$  e  $r = 4$ , o 10º termo é  $7 + 9 \cdot 4 = 43$ , não 47).
- **Matrizes:** o produto  $A \cdot B$  só existe se o número de **colunas de A** for igual ao número de **linhas de B**; o resultado tem as **linhas de A** e as **colunas de B**. Logo  $(2 \times 3) \cdot (3 \times 4) = (2 \times 4)$ . O produto **não é comutativo**.
- **Conjuntos — inclusão-exclusão:**  $|A \cup B| = |A| + |B| - |A \cap B|$ . Quem está “fora dos dois” é o total menos a união. Ex.: 60 candidatos, 35 Java, 28 Python, 12 ambos  $\rightarrow$  união  $= 35 + 28 - 12 = 51$ ; fora  $= 60 - 51 = 9$ . Esquecer de subtrair a interseção é o erro plantado.
- **Medidas de posição:** **média** = soma/quantidade; **mediana** = valor central do rol **ordenado** (com  $n$  par, média dos dois centrais); **moda** = o que mais se repete (pode não haver, ou haver mais de uma). Ordenar antes de pegar a mediana é o passo que a banca conta que você pule.
- **Desvio padrão** mede a **dispersão em torno da média**, não a posição da média. Duas séries com a *mesma* média e desvios diferentes: a de **menor** desvio é a mais **regular** (concentrada). Desvio padrão não é amplitude — não diz que os valores variaram “de 0 até o desvio”.

## 17.2 Lógica formal (não subestime)

Conectivos e tabela-verdade:

| Conectivo                       | Símbolo               | Falso quando                                    |
|---------------------------------|-----------------------|-------------------------------------------------|
| Conjunção “e”                   | $p \wedge q$          | ao menos um é falso                             |
| Disjunção “ou”                  | $p \vee q$            | ambos falsos                                    |
| Condicional “se...então”        | $p \rightarrow q$     | <b>V</b> $\rightarrow$ <b>F</b> (só nesse caso) |
| Bicondicional “se e somente se” | $p \leftrightarrow q$ | valores diferentes                              |
| Disjunção exclusiva “ou...ou”   | $p \oplus q$          | valores iguais                                  |

Equivalências que a FGV cobra:

- **Contrapositiva:**  $p \rightarrow q \equiv \neg q \rightarrow \neg p$ . “Se é fã então assiste”  $\equiv$  “se **não** assiste então **não** é fã”. (A recíproca  $q \rightarrow p$  **não** é equivalente.)

- **Condicional como disjunção:**  $p \rightarrow q \equiv \neg p \vee q$ .
- **Negação da condicional:**  $\neg(p \rightarrow q) \equiv p \wedge \neg q$ .
- **De Morgan:**  $\neg(p \wedge q) \equiv \neg p \vee \neg q$ ;  $\neg(p \vee q) \equiv \neg p \wedge \neg q$ .

**Argumentação (validade):** um argumento é **válido** se, sempre que as premissas forem verdadeiras, a conclusão também for.

- **Modus ponens:**  $p \rightarrow q, p \vdash q$  (válido).
- **Modus tollens:**  $p \rightarrow q, \neg q \vdash \neg p$  (válido).
- **Falácias:** afirmar o consequente ( $p \rightarrow q, q \vdash p$ ) e negar o antecedente ( $p \rightarrow q, \neg p \vdash \neg q$ ) são **inválidas** — a FGV usa como pegadinha.
- **Silogismo hipotético:**  $p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$ .

**Quantificadores / primeira ordem e diagramas:** “Todo A é B”, “Algum A é B”, “Nenhum A é B”.

- **Negações:** negar “todo A é B”  $\rightarrow$  “**algum A não é B**”; negar “algum A é B”  $\rightarrow$  “**nenhum A é B**”. A FGV adora a negação errada do quantificador.
- **Diagramas lógicos** (círculos de Euler/Venn): resolvem “todo/algum/nenhum” desenhando os conjuntos e testando o que **necessariamente** decorre — não o que “pode ser”.

#### ATENÇÃO — Pegadinha recorrente de lógica

Confundir **equivalência** com **implicação**, marcar a **recíproca** como equivalente, ou negar quantificador trocando “todo” por “nenhum” (o certo é “algum não”).

#### ATENÇÃO — Como a FGV arma a pegadinha

**Nas de matemática:** cada alternativa errada é o resultado de um erro de conta específico — esqueceu de somar o peso, inverteu numerador/denominador, contou o caso duplicado, somou as taxas em vez de multiplicar, usou  $n$  em vez de  $n - 1$  na PA, não subtraiu a interseção na união de conjuntos, dividiu um grupo pelo outro em vez de pelo total na probabilidade. Refaça com calma; o distrator “quase certo” vem de um passo omitido.

**Nas de estatística:** o distrator confunde **dispersão** com **posição** — “desvio padrão menor indica média menor” é falso (a média pode ser idêntica) — ou trata o desvio como **amplitude** (“variou entre 0 e 45 ms”). Na mediana, oferece o resultado de quem **não ordenou** a série.

**Nas de lógica:** troca equivalência por implicação, marca a recíproca como equivalente, nega o quantificador errado (“todo”  $\rightarrow$  “nenhum” em vez de “algum não”), ou apresenta uma falácia (afirmar o consequente) como argumento válido.

#### O que já caiu nas provas

**Em prova real da FGV:** divisão proporcional (repartir prejuízo na razão do capital investido); média ponderada com pesos por bimestre; sistema com soma e soma dos quadrados (produtos notáveis); **equivalência da condicional** (contrapositiva); grafo/estradas resolvido por combinação  $C(n, 2)$ ; aumentos sucessivos e taxa média — **Dataprev 2024** (Q25–Q30). Porcentagem e lucro; contagem em fila; **lógica proposicional encadeada** e argumentação a partir de declarações; **permutação com restrição** (probabilidade); **soma de PA**; **geometria plana** (perímetro  $\times$  comprimento de um retângulo); sistemas de equações e razão — **ALERO 2026** (11 questões).

Repare: a ALERO 2026 desmente a leitura de que “RLM da FGV é só conta” — **lógica formal**

e **argumentação apareceram lá**, e o edital 2026 da Dataprev as coloca em quatro dos cinco itens. É a razão do alerta que abre este capítulo.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): De Morgan; negação de quantificador; PA pelo termo geral; juros simples  $\times$  compostos; produto de matrizes; inclusão-exclusão; mediana/moda; desvio padrão; Pitágoras.

Rode `./quiz.py rlm`.

### Como se sair melhor

Chegue com as fórmulas na ponta:  $C(n, 2) = n(n - 1)/2$ ; média ponderada  $= \Sigma(\text{nota} \times \text{peso}) / \Sigma \text{pesos}$ ;  $(a + b)^2 = a^2 + 2ab + b^2$ ; taxa acumulada = produto dos fatores; PA  $a_n = a_1 + (n - 1)r$ ; juros compostos  $M = C(1 + i)^n$  contra simples  $M = C(1 + i \cdot n)$ ;  $|A \cup B| = |A| + |B| - |A \cap B|$ ; produto de matrizes  $(m \times n) \cdot (n \times p) = (m \times p)$ . Em porcentagem sucessiva, **sempre multiplique** fatores; nunca some percentuais. Estime a ordem de grandeza antes de marcar: isso elimina o distrator “redondo” plantado pela banca. Não gaste tempo demais numa questão de conta pesada — são só **5** no total (foram 6 na Dataprev 2024); garanta as de porcentagem, média e combinatória, que são as mais recorrentes.

## 17.3 Alta probabilidade / pesquisa extra

- **Probabilidade** básica: casos favoráveis / casos **possíveis** (o total, não o outro grupo). Urna com 5 azuis e 3 verdes  $\rightarrow P(\text{azul}) = 5/8$ , não  $5/3$  (nem probabilidade seria, passa de 1) nem  $5/3$  invertido em  $3/5$ .
- **Diagramas lógicos** (todo/algum/nenhum) para argumentação.
- Treine **velocidade**: são 5 questões valendo 1 ponto cada — não gaste tempo demais numa; os específicos valem  $2,5\times$ .

## Capítulo 18

# Atualidades e Inteligência Artificial

### Ficha do edital

(1) Tópicos atuais de segurança, transportes, política, economia, sociedade, educação, saúde, cultura, tecnologia, energia, relações internacionais, desenvolvimento sustentável e ecologia; (2) Inteligência Artificial: fundamentos e aplicações — conceitos de IA, aprendizado de máquina, modelos generativos e de linguagem (LLMs), ética, governança e privacidade em IA.

**Peso esperado: 6 questões, peso 1 — NOVO em 2026: a disciplina ganhou o bloco de IA. IA saiu do específico e entrou nas gerais.**

### Leia antes: incerteza de FORMATO, não de valor

Este é o **único capítulo da apostila sem calibração em provas anteriores da FGV**. Inteligência Artificial entrou no edital da Dataprev só em 2026 e não há questão passada da banca para ancorar o estilo. Tudo que segue sobre *formato* — se cai como julgamento de afirmativas, como cenário aplicado, quantas questões são conceituais — é **projeção**, não histórico observado como nos demais capítulos.

O que isso muda na preparação: a incerteza é de **formato, não de valor**. O conteúdo aqui continua sendo o que a área cobra. E justamente por ser tema novo, a probabilidade maior é de item **introdutório/conceitual** (definições, tipos de aprendizado, o que é LLM, viés, alucinação) do que de pegadinha sofisticada — essas a banca só monta depois de anos calibrando o assunto. Por isso a recomendação é **dominar bem os fundamentos** desta parte em vez de tentar antecipar o desenho da questão: fundamento sólido responde qualquer formato; decorar um formato hipotético, não.

## 18.1 Parte 1 — Atualidades (viés socioambiental)

Na Dataprev 2024 foram 5 questões (Q31–Q35) e o viés é claríssimo: **SOCIOAMBIENTAL**. Sustentabilidade, meio ambiente, ESG, desigualdade, proteção de dados aparecem quase sempre. A FGV costuma cobrar no formato **julgar afirmativas** (V/F, “está correto o que se afirma em I, II, III”).

### ESG — as três dimensões

**ESG** vem do inglês *Environmental, Social and Governance* — em português, **Ambiental, Social e Governança**. É o conjunto de critérios usado para avaliar o desempenho **não financeiro** de uma organização, hoje presente em relatórios corporativos e em critérios de investimento:

- **E — Ambiental:** emissões, energia, água, resíduos, biodiversidade, mudança climática.
- **S — Social:** trabalhadores, diversidade e inclusão, direitos humanos, comunidades, cadeia de fornecedores.



- **G — Governança:** composição e independência do conselho, ética, transparência, anticorrupção, prestação de contas.

O “G” é o mais trocado: a banca oferece “Gestão”, “Global” ou “Growth” no lugar de **Governança**, e às vezes troca o “S” de Social por “Sustentável” — sustentabilidade é o guarda-chuva, não a letra.

Temas concretos vistos nas provas FGV do edital:

- Fóruns/cúpulas internacionais (G20 no Brasil, presidência rotativa, “Mundo Justo e Planeta Sustentável”, combate às desigualdades).
- Impacto ambiental da tecnologia (consumo de energia de data centers, pegada de carbono do digital — já caiu na Dataprev 2024, ~2% da eletricidade mundial no texto-base).
- Art. 225 da Constituição (direito ao meio ambiente equilibrado), racismo ambiental, justiça socioambiental.
- Proteção de dados / LGPD aplicada a caso real (uso indevido de dados de consumidores, publicidade direcionada).
- No MPU: A3P, Política Nacional sobre Mudança do Clima (Lei 12.187/2009), Protocolo de Quioto, mudanças climáticas (enchentes RS, incêndios) — mesma pegada verde.

#### ATENÇÃO — Como a FGV arma a pegadinha

Item com **absoluto** ou termo radical é quase sempre falso: “todos os avanços tecnológicos representam INVARIavelmente benefícios para o meio ambiente” — o “invariavelmente” já entrega que é falso. Item que mantém “a lógica de consumo do atual modelo econômico” e ainda promete “justiça social para todos” é contradição plantada; desenvolvimento sustentável pede mudar o modelo. Item com dado numérico inflado/inventado (“data centers = metade do consumo dos ecossistemas digitais” quando o texto disse ~2% da eletricidade mundial) — confira o número contra o texto-base. Item “bonzinho” mas falso: destino de dados “para pesquisas que democratizam remédios baratos” soa nobre, mas não é o que a denúncia trata (uso indevido/comercialização) — a banca usa a alternativa moralmente simpática como isca. O item verdadeiro costuma ser o mais moderado e alinhado ao senso socioambiental.

#### Como se sair melhor

Trate como prova de interpretação: a maioria das respostas está no **próprio texto-base**. Releia antes de julgar cada item. Marque como falso todo item com “sempre, nunca, apenas, invariavelmente, exclusivamente, todos”. Desconfie de item que contradiz o consenso ESG (sustentável = mudar consumo, incluir os vulneráveis, reduzir emissões). Julgue item por item e só então escolha a combinação; se dois itens você tem certeza, as combinações de resposta já eliminam metade das opções. Acompanhe: cúpulas do Brasil (G20, COP/clima), LGPD/ANPD em casos noticiados, energia e IA/data centers, desigualdade.

## 18.2 Parte 2 — Fundamentos de IA (novo, alto retorno)

### 18.2.1 Conceitos

- **IA:** sistemas que executam tarefas que exigiriam inteligência humana.
- **Machine Learning (ML):** subcampo em que o sistema **aprende de dados** e melhora com a experiência, sem ser explicitamente programado.
- **Deep Learning:** ML com **redes neurais profundas** (muitas camadas).

- Relação:  $IA \supset ML \supset Deep\ Learning$ .

### 18.2.2 Tipos de aprendizado

| Tipo               | Dados                | Objetivo        | Exemplos                              |
|--------------------|----------------------|-----------------|---------------------------------------|
| Supervisionado     | rotulados            | prever rótulo   | classificação, regressão              |
| Não supervisionado | sem rótulo           | achar estrutura | clustering (K-Means), associação, PCA |
| Por reforço        | recompensa / punição | política ótima  | agentes, jogos                        |

#### ATENÇÃO — Como a FGV arma a pegadinha

Supervisionado usa dados **rotulados**; não supervisionado, **sem rótulo**. A FGV troca os dois. “Aprende com os dados e melhora ao longo do tempo” = **redes neurais/ML**, não lógica booleana nem programação linear.

### 18.2.3 Ajuste do modelo e métricas de avaliação

**Viés (bias) × variância (variance)** é o trade-off central do aprendizado supervisionado:

|         | Alto viés                              | Alta variância                                       |
|---------|----------------------------------------|------------------------------------------------------|
| Modelo  | <b>simples demais</b>                  | <b>complexo demais</b>                               |
| Sintoma | erra no treino <i>e</i> no teste       | acerta o treino, erra o teste                        |
| Nome    | <b>underfitting</b>                    | <b>overfitting</b>                                   |
| Combate | mais atributos, modelo mais expressivo | regularização (L1/L2), validação cruzada, mais dados |

Reduzir um tende a aumentar o outro — por isso é *trade-off*, não uma escolha em que dá para zerar os dois.

**Métricas de classificação.** A partir da matriz de confusão (verdadeiro/falso positivo e negativo):

- **Acurácia** = acertos / total. **Enganosa em base desbalanceada:** num conjunto com 99% de e-mails legítimos, o modelo que chuta “legítimo” sempre tem 99% de acurácia e não detecta **nenhum** spam.
- **Precisão** = dos que o modelo apontou como positivos, quantos eram mesmo. Sobe quando o custo do **falso positivo** é alto (barrar e-mail legítimo).
- **Recall** (revocação/sensibilidade) = dos positivos que existiam, quantos o modelo pegou. Sobe quando o custo do **falso negativo** é alto (deixar passar fraude).
- **F1-score** = **média harmônica** de precisão e recall — harmônica, não aritmética, justamente para punir o desequilíbrio entre as duas.
- **ROC/AUC** = capacidade de separar as classes variando o limiar de decisão.

Em **regressão** (prever um número, não uma classe): **MAE** (erro médio absoluto), **RMSE** e **R<sup>2</sup>**. Métrica de classificação em problema de regressão — e vice-versa — é distrator comum.

#### ATENÇÃO — Como a FGV arma a pegadinha

Inversão de par: dizer que **alto viés** causa overfitting ou que **alta variância** é modelo simples demais. O ancoradouro: viés alto = o modelo é *burro* (underfit); variância alta = o modelo *decorou* (overfit). E a regularização combate a **variância/overfitting**.

Em métricas: chamar F1 de “média aritmética” de precisão e recall (é **harmônica**); trocar precisão por recall na definição; e tratar acurácia alta como prova de bom modelo sem olhar o balanceamento da base — esse é o cenário que a banca monta com spam, fraude e doença rara.

### 18.2.4 Modelos generativos e LLMs

- **Modelos generativos:** geram conteúdo novo (texto, imagem) aprendido da distribuição dos dados.
- **LLM (Large Language Model):** modelo de linguagem treinado em texto massivo (arquitetura **Transformer**, mecanismo de **atenção**); gera/entende linguagem. Ex.: família GPT, Claude, Gemini.
- **RAG (Retrieval-Augmented Generation):** combina o LLM com busca em base externa para respostas mais atuais e fundamentadas.
- **Alucinação:** o modelo gera resposta plausível porém incorreta.
- **Prompt, fine-tuning, embeddings** são conceitos recorrentes.

### 18.2.5 Ética, governança e privacidade em IA

- **Viés algorítmico** (dados enviesados → decisão injusta), transparência/explicabilidade, responsabilização.
- **Privacidade:** IA e LGPD (dados pessoais em treino e inferência).
- **Regulação:** discussões de marco legal da IA (Brasil PL 2338/2023; **AI Act** europeu, baseado em risco).

### 18.2.6 IA aplicada a negócios e políticas públicas — como a FGV cobra

O edital não pede só a teoria (redes neurais, tipos de aprendizado): ele pede **fundamentos e aplicações**. Isso significa que a FGV pode enquadrar a mesma pergunta técnica dentro de um **cenário** — um órgão público, um banco, uma prefeitura — e cobrar se o candidato reconhece o conceito por trás da situação descrita. É o mesmo estilo de pegadinha por cenário que já aparece em Governança (Capítulo 12) e em Segurança (Capítulo 8), agora aplicado a IA.

#### Padrões de cenário que a FGV usa

- **Triagem/priorização de atendimento** (fila de posto de saúde, concessão de benefício social, análise de currículos): pede se o modelo é **classificação supervisionada** (rótulo conhecido: aprovado/negado, prioritário/não) e se há **direito a explicação/revisão humana** da decisão automatizada.
- **Deteção de fraude ou anomalia** (glosa de conta pública, fraude em benefício, transação bancária suspeita): em geral **não supervisionado** ou semisupervisionado, porque o padrão fraudulento muda com o tempo (concept drift) e casos rotulados são raros.
- **Chatbot/atendimento ao cidadão** com LLM: cenário testa se você sabe que a resposta

pode **alucinar** e que, por isso, decisão que afeta direito do cidadão exige **validação humana**, não a resposta do modelo sozinho.

- **Concessão de crédito, seguro ou benefício por score algorítmico:** cenário clássico de **viés algorítmico** — se o dado de treino reflete desigualdade histórica (ex.: bairro, gênero, raça correlacionados a inadimplência passada), o modelo **reproduz e amplifica** essa desigualdade, mesmo sem usar o atributo sensível diretamente (proxy discriminatório).

**Base legal que ancora esses cenários (LGPD, art. 20):** o titular tem direito a **solicitar revisão** de decisões tomadas unicamente com base em tratamento automatizado que afetem seus interesses, incluindo decisões destinadas a definir o seu perfil pessoal, profissional, de consumo e de crédito. A FGV cruza esse artigo com IA quando o cenário é score de crédito, RH automatizado ou benefício público.

#### ATENÇÃO — Como a FGV arma a pegadinha em cenários de IA aplicada

- O cenário descreve um sistema que **decide sozinho, sem qualquer revisão humana**, e a alternativa correta chama isso de boa prática — **descarte:** LGPD garante revisão humana de decisão automatizada relevante.
- “O modelo de IA é neutro porque não usa o atributo raça/gênero diretamente” — **falso:** variáveis **proxy** (CEP, escola, histórico de consumo) podem recriar o viés indiretamente. Ignorar isso é o distrator clássico de “neutralidade tecnológica”.
- Confundir **explicabilidade** (o modelo consegue justificar a decisão de forma compreensível a humanos) com **acurácia** (o modelo acerta muito) — um modelo pode ser muito acurado e pouco explicável (caixa-preta), e o edital cobra exatamente essa tensão.
- Tratar chatbot/LLM em atendimento público como fonte **definitiva e infalível** de informação — ignora o risco de alucinação, que é justamente o ponto que a banca quer que você saiba mitigar (revisão humana, RAG com fonte oficial, escopo restrito).

### 18.2.7 Interseção IA × ESG: energia, governança e regulação

A FGV gosta de cruzar o bloco de IA com o viés socioambiental da Parte 1 deste capítulo — é a interseção mais provável do bloco “Atualidades e IA” em 2026, porque une os dois assuntos que a banca mais repete.

#### O que sustenta esse cenário

- **Consumo de energia dos data centers:** a maior fatia da energia de um data center vai para a **infraestrutura de TI** (processamento, ~metade do consumo) e para o **resfriamento/ar-condicionado** (a segunda maior fatia) — refrigeração é, depois do próprio processamento, o maior custo energético de treinar/rodar modelos de IA em escala.
- **PUE (Power Usage Effectiveness):** métrica-padrão de eficiência energética de data center (energia total consumida ÷ energia usada só pela TI). Regulações recentes (ex.: diretiva europeia) já fixam metas numéricas de PUE para novos data centers — quanto mais próximo de 1,0, mais eficiente.
- **Matriz energética:** boa parte da energia elétrica que alimenta data centers no mundo ainda vem de fontes fósseis — por isso a expansão acelerada de IA pressiona diretamente as metas de descarbonização (o mesmo tema de mudança climática que já caiu em Atualidades).
- **Governança:** reguladores discutem exigir que empresas **divulguem publicamente** o

impacto ambiental (energia, água, emissões) de produtos e serviços baseados em IA — transparência como instrumento de política pública, não só como boa vontade corporativa.

### ATENÇÃO — Como a FGV arma a pegadinha

O texto-base cita um número (ex.: “X% da energia global”, “Y milhões de toneladas de CO<sub>2</sub>”) e a alternativa **infla ou distorce esse número** — releia o texto-base antes de julgar, nunca confie no número de memória. Também é comum a alternativa afirmar que “a infraestrutura de IA usa MAJORITARIAMENTE energia limpa/renovável” — trate como distrator otimista, salvo se o próprio texto-base afirmar isso explicitamente. E cuidado com a armadilha inversa: dizer que IA é “incompatível com sustentabilidade” também é absoluto demais — o texto costuma apresentar o dilema (benefício tecnológico e custo ambiental), não um veredito fechado.

### O que já caiu nas provas

**Em prova real da FGV: supervisionado × não supervisionado** (duas questões, uma delas de associação tipo-técnica); **viés × variância com regularização Lasso/L1** e over/underfitting; **concept drift × data drift** num modelo de *churn* degradando em produção; **MLOps** num *pipeline* ponta a ponta de detecção de fraude; **MAE** para prever tempo de tramitação (regressão); **CNN** e camadas de convolução; plataforma baseada em **LLM** para análise documental — **TJ-RJ** (9 questões de IA). **Cúpula do G20 no Brasil** (“Mundo Justo e Planeta Sustentável”); **impacto ambiental do digital** e consumo de data centers; **art. 225 da Constituição** e racismo ambiental; **LGPD em caso real** (tratamento indevido de dados de consumidores); **cidade-esponja** — **Dataprev 2024** (Q31–Q35). No MPU, a mesma pegada verde em outro formato (A3P, Política Nacional sobre Mudança do Clima, Protocolo de Quioto).

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): a hierarquia IA  $\supset$  ML  $\supset$  Deep Learning; LLM e Transformer; RAG; alucinação; **viés algorítmico** em decisão automatizada (o “viés” que caiu no TJ-RJ é o *estatístico* do trade-off, conceito diferente); **direito à revisão humana** da LGPD, art. 20 — a explicabilidade do art. 20 caiu, mas no bloco de **Legislação** do TJ-RJ, não aqui; a abordagem baseada em risco do AI Act e o PL 2338/2023; ESG por extenso; PUE e matriz energética de data center; métricas de classificação. Rode `./quiz.py atualidades`.

## 18.3 Alta probabilidade / pesquisa extra

- **Regularização L1 (Lasso, zera coeficientes) × L2 (Ridge, encolhe sem zerar)**, **concept drift × data drift** e **MLOps** caíram nas provas de IA do TJ-RJ e estão detalhados no Capítulo 14, seção de IA/ML — que também é o capítulo para onde o quiz manda quem erra questões de **orfaos**.
- Acompanhe notícias de IA e regulação de 2026 — é conteúdo vivo, e esta disciplina recompensa quem está atualizado na véspera, não quem estudou em julho.

## Capítulo 19

# Legislação — Segurança da Informação e Proteção de Dados

### Ficha do edital

Lei 12.527/2011 (LAI) caps. I–V + Decretos 7.724 e 7.845; Lei 12.737/2012 (Delitos Informáticos) art. 2º; Lei 12.965/2014 (Marco Civil) cap. II Seção I e cap. III Seções I e II; Lei 13.709/2018 (LGPD) caps. I, II, III, IV, VII, VIII, IX.

**Peso esperado: 5 questões, peso 1.** No nosso recorte de edital a amostra de provas cobrou classificação na LAI, invasão de dispositivo (art. 154-A), sanções do Marco Civil, sanções da LGPD e ANPD × CNPD (Dataprev 2024); princípio da adequação, explicabilidade da decisão automatizada (art. 20) e IA generativa × LGPD (TJ-RJ). Ela cita o número e a data da lei no enunciado — não é decoreba de número, é entender competências, prazos e papéis.

### ATENÇÃO — Atualização crítica — Marco Civil, art. 19, pós-STF jun/2025

Em **26/06/2025** o STF mudou a regra de responsabilidade dos provedores por conteúdo de terceiros — é a atualização legislativa mais provável de cair em 2026 e tem seção própria mais adiante neste capítulo (ver Seção 19.2). Não decore a redação antiga do art. 19 como se ainda vigesse: hoje a regra geral é **notice-and-takedown**.

## 19.1 LGPD (Lei 13.709/2018)

- **Dado pessoal** (art. 5º, I) é informação relacionada a pessoa natural **identificada ou identificável** — nome, endereço, telefone, e-mail, placa, CPF. **Dado pessoal sensível** (art. 5º, II) é um rol **fechado e menor**: origem racial ou étnica, convicção religiosa, opinião política, filiação a sindicato ou a organização de caráter religioso, filosófico ou político, dado referente à **saúde** ou à **vida sexual**, dado **genético** ou **biométrico** vinculado a uma pessoa natural. Sensível tem regime de tratamento **mais restrito** (art. 11), não proibição.
- **Controlador** decide sobre o tratamento (finalidade e meios); **operador** trata em nome do controlador. **Encarregado (DPO)** é o canal com titulares e ANPD.
- **Bases legais (art. 7º)**: consentimento **não** é a única; há **10** bases (cumprimento de obrigação legal, execução de contrato, legítimo interesse, proteção da vida, tutela da saúde, políticas públicas, etc.).
- **Direitos do titular (art. 18)**: confirmação, acesso, correção, anonimização/bloqueio/eliminação, portabilidade, informação sobre compartilhamento, revogação do consentimento.
- **ANPD** — hoje **Agência** Nacional de Proteção de Dados, autarquia de natureza especial **vinculada ao MJSP** (Lei 15.352/2026) — **fiscaliza e sanciona**; o **CNPD** (Conselho) é **consultivo** (propõe diretrizes, sugere ações à ANPD). Não confunda os papéis.



- **Sanções (art. 52):** são nove em vigor (incisos I–VI e X–XII; VII–IX vetados) — advertência; multa simples de até 2% do faturamento no Brasil no último exercício, excluídos tributos, limitada a R\$ 50 milhões por infração; multa diária; publicização; bloqueio; eliminação; suspensão parcial do banco de dados; suspensão da atividade de tratamento; e proibição parcial ou total — estas três últimas só depois de já imposta outra sanção no mesmo caso (§6º). Exige processo administrativo.
- **LGPD aplicada a IA:** explicabilidade em decisão automatizada (art. 20), anonimização, *scraping* — os três já cobrados no TJ-RJ.

### 19.1.1 Agentes de tratamento — o papel vem do poder de decisão

A LGPD não distribui deveres por quem *toca* no dado, e sim por quem **decide** sobre ele. É por isso que a empresa que hospeda o banco inteiro, roda o processamento e emprega os programadores pode ter menos obrigações do que o órgão que nunca abriu o sistema: o critério é a decisão, não o acesso.

#### Controlador, operador e encarregado (art. 5º, VI a IX)

**Controlador** é aquele a quem competem **as decisões referentes ao tratamento** — define a finalidade (*para quê*) e os meios (*como*). **Operador** é quem **realiza o tratamento em nome do controlador**: executa, não escolhe. **Encarregado** é a pessoa indicada para atuar como **canal de comunicação** entre o controlador, os titulares e a ANPD — não decide nem executa, *intermedeia*.

A trava que a FGV explora: o art. 5º, IX define **agentes de tratamento** como **o controlador e o operador** — só os dois. **O encarregado não é agente de tratamento**. Quem responde perante a ANPD são os agentes; o encarregado é o telefone, não o responsável.

Exemplo. Um ministério contrata uma estatal de TI para processar a folha de pagamento. O ministério definiu por que os dados serão tratados e o que será feito com eles: é o **controlador**. A estatal executa o processamento segundo essas instruções: é a **operadora** — ainda que sejam dela o banco, os servidores e a equipe. Agora mude um detalhe: a estatal resolve, por conta própria, usar aquela mesma base para treinar um modelo interno de detecção de fraude. Naquele tratamento novo foi ela quem decidiu a finalidade — e, para ele, **virou controladora**. O papel não é um crachá permanente da empresa; é uma posição *por tratamento*.

Repare como isso amarra com os princípios da próxima subseção: quem responde pela *finalidade* e pela *adequação* é necessariamente quem decidiu a finalidade. Os *deveres* detalhados dos agentes e a responsabilidade civil solidária estão no Capítulo VI da lei, que o edital não pede — o que se cobra aqui é o papel, definido no Capítulo I.

### 19.1.2 Bases legais — o art. 7º e o art. 11 são listas diferentes

Todo tratamento de dado pessoal precisa de uma **base legal**, escolhida **antes** de tratar. Não é justificativa que se procura depois: sem base, o tratamento é ilícito desde o primeiro registro. O erro de origem é achar que a base é sempre o consentimento — ele é apenas uma das dez, e a mais frágil de todas, porque o titular pode revogá-lo.

#### Duas listas, não uma (arts. 7º e 11)

Para **dado pessoal comum**, o art. 7º traz **dez hipóteses**: consentimento; cumprimento de obrigação legal ou regulatória; execução de políticas públicas; realização de estudos por órgão de pesquisa; execução de contrato; exercício regular de direitos; proteção da vida; tutela da saúde; **legítimo interesse**; e **proteção do crédito**.

Para **dado pessoal sensível**, vale o art. 11 — uma lista **própria e menor**: consentimento **específico e destacado** para finalidades específicas, ou, sem consentimento, sete hipóteses em que o tratamento seja **indispensável** (obrigação legal; políticas públicas; estudos por órgão de pesquisa; exercício regular de direitos; proteção da vida; tutela da saúde; e prevenção à fraude e segurança do titular nos processos de identificação e autenticação).  
O que **não** está no art. 11 é justamente o que a banca oferece: **legítimo interesse** e **proteção do crédito não** autorizam tratar dado sensível.

Exemplo. Guardar o CPF de um usuário para executar um contrato entra pelo art. 7º, V. Guardar a **digital** do mesmo usuário para liberar a catraca é dado **biométrico**, portanto sensível: não adianta invocar legítimo interesse — a base é o art. 11, II, “g” (prevenção à fraude e segurança do titular nos processos de identificação e autenticação). Trocar o artigo troca a lista inteira de hipóteses disponíveis, e é essa troca que o distrator esconde.

A ligação com a subseção anterior é direta: quem escolhe a base é o controlador, porque escolher a base é decidir a finalidade.

### 19.1.3 Princípios (art. 6º) — o trio que a FGV confunde

São **dez** princípios: finalidade, adequação, necessidade, livre acesso, qualidade dos dados, transparência, segurança, prevenção, não discriminação, e responsabilização e prestação de contas. Três deles são o alvo preferido da banca, porque parecem sinônimos e não são:

| Princípio          | O que exige                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Finalidade</b>  | propósitos <b>legítimos, específicos e explícitos, informados ao titular</b> , sem tratamento posterior incompatível   |
| <b>Adequação</b>   | <b>compatibilidade</b> do tratamento com as finalidades informadas, <b>conforme o contexto</b>                         |
| <b>Necessidade</b> | tratamento <b>limitado ao mínimo</b> indispensável para atingir a finalidade (pertinente, proporcional, não excessivo) |

A âncora é a palavra-chave: **finalidade** = *para quê* (e avisou); **adequação** = *combina com* o que foi avisado; **necessidade** = *o mínimo*.

#### ATENÇÃO — Como a FGV arma a pegadinha

No TJ-RJ a FGV descreveu literalmente “o tratamento seja **compatível com os fins informados ao titular, de acordo com o contexto**” e ofereceu *finalidade, prevenção, adequação, necessidade* e *transparência* — gabarito **adequação**. Quem tinha decorado só “finalidade” errou, porque a palavra “fins” está no enunciado de propósito. Leia até o fim: quem fala em *compatibilidade* e *contexto* é a adequação; quem fala em *informar o propósito* é a finalidade; quem fala em *mínimo* é a necessidade.

### 19.1.4 Decisão automatizada e explicabilidade (art. 20)

Sistemas que decidem sozinhos — score de crédito, triagem de currículo, ordem de uma fila de atendimento — desembocam num artigo só, e é dele que a FGV pendura todo cenário de IA aplicada.



### O direito à revisão e o dever de explicar (art. 20)

O titular tem direito a **solicitar a revisão** de decisões tomadas **unicamente** com base em tratamento automatizado que afetem seus interesses, incluídas as destinadas a definir perfil pessoal, profissional, de consumo e de crédito. O **controlador** deve fornecer, sempre que solicitadas, **informações claras e adequadas sobre os critérios e os procedimentos** utilizados na decisão, **observados os segredos comercial e industrial** (§1º). Se ele se recusar invocando esse segredo, a **ANPD pode realizar auditoria** para verificar **aspectos discriminatórios** (§2º).

**Cuidado com a redação antiga:** o texto original de 2018 dizia revisão “**por pessoa natural**”. A **Lei 13.853/2019** retirou essa expressão, e o parágrafo que a reporia (§3º) foi **vetado**. Hoje a lei **não exige** revisor humano — material desatualizado ensina o contrário, e a alternativa que exige “revisão por pessoa natural” está errada.

Exemplo. Um sistema recusa automaticamente um pedido de benefício. O titular pode pedir revisão e pode saber *quais critérios* pesaram — não o código-fonte nem os pesos do modelo, que o segredo industrial protege, mas os fatores considerados e o procedimento seguido. É essa distinção que o TJ-RJ cobrou ao definir **explicabilidade**: a propriedade de um sistema de IA que permite que **os fatores importantes que influenciam uma decisão sejam expressos de maneira que humanos entendam**. Não é “o modelo é simples”, não é “o humano tende a confiar na decisão” (isso é viés de automação) — *é o fator sai em linguagem humana*.

Daí decorre a outra ponta cobrada no TJ-RJ: conteúdo publicamente acessível na web **não** é conteúdo livre de LGPD. Raspar a web para treinar modelo continua sendo tratamento de dados pessoais sempre que houver dado pessoal na coleta — e a conformidade é exigível tanto de quem disponibiliza a plataforma quanto de quem faz a raspagem.

#### 19.1.5 Segurança e boas práticas (Capítulo VII) — a LGPD que o desenvolvedor implementa

Os capítulos anteriores dizem *o que* pode ser tratado e *por quem*. O Capítulo VII diz *como* — e é a parte da lei que vira requisito de sistema, não política de gaveta. Está no recorte do edital e é a mais próxima do trabalho de quem desenvolve.

### Os quatro deveres operacionais (arts. 46 a 50)

**Segurança desde a concepção (art. 46).** Os agentes devem adotar medidas técnicas e administrativas aptas a proteger os dados de acessos não autorizados e de destruição, perda, alteração ou tratamento ilícito — e o §2º exige que essas medidas sejam observadas **desde a fase de concepção do produto ou do serviço até a sua execução**. É o *privacy by design* com texto de lei: segurança é requisito de projeto, não etapa final.

**Sigilo que sobrevive ao fim (art. 47).** A obrigação de garantir a segurança alcança todos os que intervêm em qualquer fase do tratamento **mesmo após o seu término**.

**Comunicação de incidente (art. 48).** O **controlador** deve comunicar **à ANPD e ao titular** o incidente de segurança que **possa acarretar risco ou dano relevante**, em **prazo razoável** definido pela autoridade, mencionando no mínimo a natureza dos dados afetados, os titulares envolvidos, as medidas de proteção utilizadas, os riscos do incidente, o motivo da demora e as providências de mitigação.

**Governança em privacidade (art. 50).** Controladores e operadores podem formular regras de boas práticas e implementar programa de governança em privacidade, adaptado à estrutura, à escala e ao volume das operações e à sensibilidade dos dados tratados.

Exemplo. Um vazamento expõe e-mails e CPFs de mil titulares. Três leituras erradas aparecem

como distrator. A primeira é que basta avisar a ANPD — a lei manda avisar **também o titular**. A segunda é que só se comunica vazamento de dado sensível — o critério é **risco ou dano relevante**, não a categoria do dado. A terceira é que a lei fixa um prazo em horas — ela diz **prazo razoável**, remetendo à regulamentação da ANPD. E há um detalhe que costuma passar: quem comunica é o **controlador**, porque o art. 48 põe o dever nele, ainda que o incidente tenha ocorrido na infraestrutura da operadora.

Note a amarração com a subseção seguinte: a adoção demonstrada desses mecanismos e da política de governança é um dos parâmetros que a ANPD pesa ao dosar a sanção. Investir em prevenção não serve só para evitar o incidente — conta a favor depois dele.

### 19.1.6 Sanções administrativas (art. 52) — é um regime, não uma tabela de multa

O erro de leitura mais comum aqui é tratar o art. 52 como tabela de preços. Ele é um **regime**: define quem aplica, sob qual rito, com quais critérios de dosagem e em qual ordem. Quase todos os distratores da FGV atacam o rito ou o destino do dinheiro, não o percentual.

#### O regime sancionatório da LGPD

**Quem aplica:** a ANPD, sobre os **agentes de tratamento**.

**Rito (§1º):** as sanções são aplicadas **após procedimento administrativo que possibilite a ampla defesa**, de forma **gradativa, isolada ou cumulativa**. Não existe sanção direta, nem para infrator contumaz.

**Dosimetria (§1º):** a lei lista **onze parâmetros** — gravidade e natureza da infração, boa-fé do infrator, vantagem auferida, condição econômica, reincidência, grau do dano, cooperação, adoção reiterada de mecanismos de mitigação, política de boas práticas e governança, pronta adoção de medidas corretivas e proporcionalidade. **Nacionalidade do infrator não é parâmetro.**

**As sanções:** advertência com prazo para medidas corretivas; multa simples; multa diária; publicização da infração; bloqueio dos dados; eliminação dos dados; suspensão parcial do banco de dados; suspensão da atividade de tratamento; e proibição parcial ou total da atividade. As três últimas só podem ser aplicadas **depois** de já imposta ao menos uma das anteriores no mesmo caso concreto (§6º).

**Multa simples:** até **2%** do faturamento da pessoa jurídica de direito privado, grupo ou conglomerado **no Brasil**, no **último exercício**, **excluídos os tributos**, limitada a **R\$ 50 milhões por infração**.

**Órgão público não leva multa (§3º):** a entidades e órgãos públicos aplicam-se apenas advertência, publicização, bloqueio, eliminação, suspensão e proibição — **multa simples e multa diária ficam de fora**.

**Destino da multa (§5º):** vai para o **Fundo de Defesa de Direitos Difusos**, **não** para o titular lesado.

**Vazamento individual (§7º):** admite **conciliação direta entre controlador e titular**; não havendo acordo, aplicam-se as penalidades do capítulo.

Repare no desenho: a reparação do titular **não** sai da sanção administrativa. A multa financia um fundo coletivo; o dinheiro do titular vem por via civil, em ação de reparação. Confundir as duas vias é o distrator preferido — “o produto da arrecadação será destinado às pessoas naturais cujo direito foi violado” soa justo e é falso.

E há a ligação com o Capítulo VII da subseção anterior: dois dos onze parâmetros de dosimetria (adoção reiterada de mecanismos de mitigação e política de boas práticas e governança) só existem para quem implementou o que aquele capítulo pede. A mesma lei que pune dá desconto a quem se preparou.

### 19.1.7 ANPD e CNPD — dois colegiados, e um deles mudou de nome em 2026

O par ANPD × CNPD é o item mais previsível do bloco: já caiu na Dataprev 2024 e a própria estrutura da lei convida à inversão.

#### Quem é quem no Capítulo IX

A **ANPD** é a autoridade: **fiscaliza, regulamenta e aplica sanção**. O **CNPD** é **consultivo: propõe** diretrizes estratégicas, **elabora** relatórios e estudos, **sugere ações** à ANPD e **dissemina** conhecimento sobre proteção de dados. Nenhum verbo do CNPD é verbo de decisão. A inversão estrutural que a banca explora: o CNPD **integra a estrutura da ANPD** (art. 55-C, II) — e não o contrário. Dizer que “a ANPD é uma das integrantes do CNPD” inverte a relação.

E não confunda os **dois colegiados**: o **Conselho Diretor** é o **órgão máximo de direção da própria ANPD**, com **cinco diretores** e mandato de **quatro anos**; o **CNPD** é o conselho consultivo, com **vinte e três representantes** de órgãos públicos, sociedade civil, instituições científicas e setor empresarial. Atribuir ao Conselho Diretor a composição multiinstitucional do CNPD foi exatamente o distrator da Dataprev 2024.

#### Atualização — Lei 15.352, de 25/02/2026

A **Lei 15.352/2026**, conversão da Medida Provisória 1.317/2025, deu nova redação ao art. 55-A. A sigla continua **ANPD**, mas o nome passou a ser **Agência Nacional de Proteção de Dados: autarquia de natureza especial vinculada ao Ministério da Justiça e Segurança Pública**, dotada de autonomia **funcional, técnica, decisória, administrativa e financeira**, com patrimônio próprio e sede e foro no Distrito Federal, **nos termos da Lei 13.848/2019** — a lei das agências reguladoras.

Guarde a trajetória, porque é dela que a banca tira os distratores: a ANPD nasceu como **órgão** da administração pública federal integrante da **Presidência da República** (redação de 2018–2019, de natureza jurídica declaradamente transitória); tornou-se **autarquia de natureza especial** pela **Lei 14.460/2022**; e em **2026** passou a **agência**, vinculada ao **Ministério da Justiça e Segurança Pública**. Alternativa que a descreva como “órgão da administração direta” ou “vinculada à Presidência da República” está retratando desenho revogado.

#### ATENÇÃO — Como a FGV arma a pegadinha

Troca **controlador × operador**: controlador **decide** sobre o tratamento; operador **executa** em nome do controlador. Troca **ANPD × CNPD**: a ANPD é autarquia federal de natureza especial que fiscaliza e aplica sanção; o CNPD é órgão consultivo — apenas propõe/sugere e dissemina (no Dataprev 2024 a alternativa certa era o CNPD “sugerir ações à ANPD”). Dizer que consentimento é a única base legal da LGPD é falso — há 10 bases.

## 19.2 Marco Civil (Lei 12.965/2014)

### 19.2.1 Guarda de registros

- **Registros de conexão**: guarda obrigatória por **1 ano** (provedor de conexão).
- **Registros de acesso a aplicações**: **6 meses** (provedor de aplicação).
- **Princípios**: neutralidade de rede, privacidade, liberdade de expressão.

**Neutralidade de rede (art. 9º)**. O responsável pela transmissão, comutação ou roteamento tem o dever de **tratar de forma isonômica quaisquer pacotes de dados, sem distinção por**

**conteúdo, origem e destino, serviço, terminal ou aplicação.** Ou seja: o provedor não pode degradar, nem priorizar, nem bloquear tráfego por causa *do que* ele carrega ou *de quem* ele vem. A discriminação só cabe nas **exceções do §1º**: requisitos técnicos indispensáveis à prestação adequada dos serviços e priorização de **serviços de emergência** — e ainda assim mediante regulamentação.

#### ATENÇÃO — Como a FGV arma a pegadinha

Os distratores de neutralidade descrevem justamente o que a lei proíbe, com cara de eficiência operacional: “priorizar o tráfego de quem paga mais”, “bloquear aplicações de concorrentes”, “monitorar o conteúdo dos pacotes para fins comerciais”, “reduzir a velocidade de vídeo no horário de pico para aliviar a rede”. Esse último é o mais perigoso: soa técnico, mas é discriminação **por aplicação**, que não está entre as exceções do §1º.

### 19.2.2 Sanções (art. 12)

As sanções do Marco Civil punem o descumprimento das regras de guarda e de tratamento de registros dos arts. 10 e 11 — exatamente a matéria da subseção anterior. Elas já caíram na Dataprev 2024, e o distrator era numérico.

#### As quatro sanções do art. 12

**I — advertência**, com indicação de **prazo para adoção de medidas corretivas** (é a que tem duas faces: repreende e orienta); **II — multa de até 10%** do faturamento do **grupo econômico no Brasil** no seu **último exercício**, excluídos os tributos, considerados a condição econômica do infrator e a proporcionalidade entre a gravidade da falta e a intensidade da sanção; **III — suspensão temporária** das atividades do art. 11; **IV — proibição** do exercício dessas atividades. Aplicam-se **de forma isolada ou cumulativa** e **sem prejuízo** das demais sanções cíveis, criminais ou administrativas.

**Parágrafo único:** tratando-se de **empresa estrangeira**, a **filial, sucursal, escritório ou estabelecimento situado no País** responde **solidariamente** pelo pagamento da multa — não existe vácuo legislativo para empresa de fora.

O par de números é o que a banca troca: **Marco Civil**, **até 10%** do faturamento do grupo econômico no Brasil no último exercício, **sem teto nominal**; **LGPD**, **até 2%**, com **teto de R\$ 50 milhões por infração**. Quem guardou só “dez por cento” erra a LGPD, e quem guardou só “dois por cento” erra o Marco Civil. Cuidado também com a janela: é o **último exercício**, não a média dos últimos três — foi assim que a alternativa errada da Dataprev 2024 foi montada.

### 19.2.3 Art. 19 — responsabilidade dos provedores (atualização crítica pós-STF, jun/2025)

O art. 19 do Marco Civil trata de quando uma plataforma/provedor responde civilmente por conteúdo **gerado por terceiros** (posts, comentários, vídeos de usuários). Até 2025, a redação original condicionava essa responsabilidade ao descumprimento de **ordem judicial específica** de remoção. Isso **mudou**.

#### O que o STF decidiu — Temas 987 e 533, 26/06/2025

Em **26/06/2025**, o STF julgou dois recursos extraordinários com repercussão geral (**Tema 987** e **Tema 533**) e, por **8 votos a 3**, declarou o **art. 19 parcial e progressivamente inconstitucional**. A tese fixada vale **a partir do julgamento** e continua valendo **até que o Congresso legisle** sobre o tema especificamente — ou seja, é a regra vigente **agora**, não uma

proposta.

|                                                                                                                                         | Antes (redação original)                                                       | Agora (pós-STF, 26/06/2025)                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Regra geral                                                                                                                             | provedor só responde se descumprir <b>ordem judicial específica</b> de remoção | regra geral virou <b>notice-and-takedown: notificação extrajudicial</b> já pode responsabilizar o provedor |
| Conteúdo íntimo não consentido (art. 21)                                                                                                | já bastava notificação (exceção histórica)                                     | continua bastando notificação (a exceção virou a regra geral)                                              |
| Crimes contra a honra (calúnia, injúria, difamação)                                                                                     | exigia ordem judicial                                                          | <b>continua exigindo ordem judicial</b> — é a única categoria que preserva a regra antiga                  |
| Conteúdo de risco grave (crime contra o Estado/ordem democrática, terrorismo, incitação ao racismo, conteúdo que ponha em risco a vida) | tratamento igual ao geral                                                      | exige atuação <b>proativa e imediata</b> do provedor, sob dever de cuidado — não espera nem notificação    |

Em resumo: o Marco Civil deixou de exigir ordem judicial como regra — a **notificação extrajudicial** (do usuário prejudicado, de terceiro ou de autoridade) já é suficiente para obrigar o provedor a agir na maioria dos casos. A **ordem judicial permanece obrigatória apenas para os crimes contra a honra**, e para conteúdo de risco grave o provedor deve agir de forma **proativa**, sem esperar sequer notificação.

#### ATENÇÃO — Como a FGV vai usar a lei antiga como distrator

A partir de 2026, espere que a banca ofereça a redação **original** do art. 19 como se ainda fosse a regra — e essa alternativa vai **soar correta** para quem estudou por material desatualizado. Reconheça o padrão:

- **Distrator típico:** “o provedor de aplicações de internet só pode ser responsabilizado civilmente por danos decorrentes de conteúdo gerado por terceiros se, após **ordem judicial específica**, não tomar as providências para tornar indisponível o conteúdo apontado como infringente” — isso era a regra **até 25/06/2025**. Hoje é a **exceção** (vale só para crimes contra a honra), não mais a regra geral. Se a alternativa não mencionar nenhuma ressalva de data ou de tipo de crime, desconfie.
- **Distrator inverso (superextrapolação):** dizer que “o STF extinguiu totalmente a exigência de ordem judicial” — também **falso**: a ordem judicial **continua obrigatória** para calúnia, injúria e difamação. A regra não é absoluta em nenhuma das duas direções.
- **Distrator de terminologia:** chamar o novo regime de “censura prévia” — é **notice-and-takedown** (reação a notificação sobre conteúdo já publicado), não filtragem antes da publicação.

- **Distrator de data/instrumento:** atribuir a mudança a uma **lei nova do Congresso** — foi uma **decisão do STF em controle de constitucionalidade** (Temas 987 e 533), com efeito **até que** o Congresso legisle sobre o tema; não confundir instância nem data.

### 19.3 LAI (Lei 12.527/2011) — prazos de sigilo

| Classificação | Prazo máximo de restrição             |
|---------------|---------------------------------------|
| Reservada     | 5 anos                                |
| Secreta       | 15 anos                               |
| Ultrasecreta  | 25 anos (prorrogável <b>uma vez</b> ) |

Regra geral: **publicidade é a regra; sigilo é exceção**. Prazos contam da **data de produção** da informação. Informação sobre **violação de direitos humanos** por agente público **não** pode ser objeto de restrição de acesso. **Transparência ativa** (o órgão publica de ofício, ex.: portal) × **passiva** (o cidadão pede via e-SIC).

**Desclassificação e reavaliação.** A classificação **não é definitiva nem discricionária**:

- **Findo o prazo** (ou consumado o evento que define o termo final), a informação torna-se **automaticamente** de acesso público — **não** é preciso procedimento próprio nem decisão específica para liberar.
- A lei permite **desclassificar e também reduzir o prazo**. A reavaliação cabe à **autoridade classificadora ou a autoridade hierarquicamente superior, de ofício ou mediante provocação**.
- A decisão de classificar é vinculada a balizas legais (grau, prazo máximo, competência por autoridade), não é escolha livre do agente.

**Decretos que o edital cita** (regulam a LAI no Executivo federal):

- **Decreto 7.724/2012:** regulamenta a LAI — procedimentos de acesso, e-SIC, prazos de resposta ao pedido (**20 dias**, prorrogáveis por **10**), recursos, transparência ativa, competências de classificação.
- **Decreto 7.845/2012:** procedimentos de **credenciamento de segurança** e tratamento de **informação classificada** (custódia, acesso, controle).

#### ATENÇÃO — Como a FGV arma a pegadinha

Inverter prazos (LAI, Marco Civil) e competências (ANPD × CNPD).

Na Dataprev 2024 os distratores da LAI foram exatamente estes: trocar os nomes dos graus (“ultrassigilosas, sigilosas ou reservadas” — é **ultrassecrta, secreta, reservada**) e dizer que a decisão de classificar é **discricionária**, “porquanto a lei não prevê balizas”; afirmar que, findo o prazo, “são necessários procedimento próprio e decisão específica” (é **automático**); e afirmar que a lei “permite a desclassificação, **mas não** a redução de prazo” (permite as duas).

### 19.4 Delitos Informáticos (Lei 12.737/2012, art. 154-A do CP)

Crime de **invasão de dispositivo informático**. **Independente** de o dispositivo estar conectado à rede (a conexão não é elementar do tipo). Aumento de pena conforme o sujeito passivo (autoridades)



e o prejuízo. (A pena exata foi alterada pela Lei 14.155/2021 — foque no conceito.)

### ATENÇÃO — Como a FGV arma a pegadinha

Dizer que a invasão do art. 154-A exige o aparelho conectado à rede é **falso** — o Dataprev derrubou exatamente essa alternativa.

### O que já caiu nas provas

**Em prova real da FGV: ANPD × CNPD** — o gabarito foi justamente “o CNPD tem atribuição de *sugerir ações* a serem realizadas pela ANPD”; **classificação na LAI** (nomes dos graus, decisão *vinculada* a balizas, acesso automático findo o prazo, desclassificação e redução de prazo); **sanções administrativas da LGPD** (parâmetros de dosimetria, destino da arrecadação das multas); **sanções do Marco Civil** (advertência com prazo para adoção de medidas); **invasão de dispositivo** do art. 154-A — não exige conexão à rede, e o aumento de pena varia com o sujeito passivo — **Dataprev 2024. Princípio da adequação** (o enunciado descreve “compatível com os fins informados, de acordo com o contexto”); **explicabilidade da decisão automatizada**, com a redação do art. 20; **IA generativa × LGPD**, incluindo raspagem da web — **TJ-RJ**.

**No nosso banco** (previsto pelo edital, ainda não visto na amostra de provas): controlador × operador; as dez bases legais; dado pessoal sensível; os prazos numéricos da LAI (5/15/25); transparência ativa × passiva; guarda de registros no Marco Civil (1 ano / 6 meses); neutralidade de rede; e todo o regime do art. 19 pós-STF de 26/06/2025 (notice-and-takedown, exceção dos crimes contra a honra, dever proativo, repercussão geral).

Rode `./quiz.py legislacao`.

## 19.5 Alta probabilidade / pesquisa extra

- **GDPR** (regulamento europeu) aparece como paralelo à LGPD.
- Atenção a novas decisões do STF/ANPD em 2026 — legislação é o bloco que mais muda; confira antes da prova.

### Como se sair melhor

LAI, prazos máximos contados da produção da informação: reservada 5 anos, secreta 15, ultrassecreta 25. Só a ultrassecreta prorroga, uma única vez, por igual período. LGPD, multa simples: até 2% do faturamento no Brasil no último exercício, excluídos tributos, limitada a R\$ 50 milhões por infração. Marco Civil art. 19 (STF, 26/06/2025): regra geral virou notice-and-takedown — notificação extrajudicial já basta, exceto para crimes contra a honra (ordem judicial). Guarde: ANPD fiscaliza/sanciona; CNPD só aconselha.

## Capítulo A

# Mapa de pesos e prioridades

### A.1 Estrutura oficial da prova

| Módulo       | Disciplina                                       | Questões  | Peso | Pontos     |
|--------------|--------------------------------------------------|-----------|------|------------|
| I            | Língua Portuguesa                                | 12        | 1    | 12         |
| I            | Língua Inglesa                                   | 12        | 1    | 12         |
| I            | Raciocínio Lógico Matemático                     | 5         | 1    | 5          |
| I            | Atualidades e Inteligência Artificial            | 6         | 1    | 6          |
| I            | Legislação (Seg. Informação e Proteção de Dados) | 5         | 1    | 5          |
| II           | Conhecimentos Específicos                        | 30        | 2,5  | 75         |
| <b>Total</b> |                                                  | <b>70</b> |      | <b>115</b> |

### A.2 Distribuição real do Módulo II na Dataprev 2024 (30 questões)

| Bloco                       | Questões | Pontos |
|-----------------------------|----------|--------|
| Engenharia de Software      | 10       | 25     |
| Programação                 | 4        | 10     |
| Banco de Dados / BI         | 4        | 10     |
| Segurança da Informação     | 3        | 7,5    |
| Redes de Computadores       | 3        | 7,5    |
| Arquitetura de Software     | 2        | 5      |
| Noções de Informática       | 2        | 5      |
| Arquitetura de Computadores | 1        | 2,5    |
| Sistemas de Informação      | 1        | 2,5    |



### A.3 Comportamento da FGV em outras provas de TI

| Prova                       | Data     | Blocos dominantes                                        |
|-----------------------------|----------|----------------------------------------------------------|
| MPU, Desenv. de Sistemas    | mai/2025 | Português 15, <b>Banco de Dados 12</b> , Eng. Software 8 |
| AgSUS, Analista de TI       | out/2025 | Português 12, Informática 8, RL 7                        |
| TJ-RJ, Analista de Sistemas | jan/2026 | Português 19, Eng. Software 7, <b>Banco de Dados 7</b>   |

### A.4 As cinco conclusões que moldam a priorização

1. **Engenharia de Software e Banco de Dados são um eixo duplo.** Na Dataprev 2024 o BD caiu com apenas 4 questões, mas isso foi flutuação, não padrão. No MPU o BD superou Engenharia de Software. Trate os dois como pilares, sem eleger vencedor.
2. **Redes de Computadores cai mesmo não estando no edital do perfil.** Foram 3 questões em 2024 (protocolo, segurança de redes, modelo OSI). O edital 2026 só cita HTTPS e SSL/TLS de passagem. Ignorar isso custa 7,5 pontos.
3. **Raciocínio Lógico da FGV é matemática, não lógica formal (mas o edital 2026 pede as duas).** O que caiu em 2024 foi aritmética, álgebra, estatística, análise combinatória e geometria plana. Só uma questão de proposições. O edital 2026 lista literalmente “problemas aritméticos, geométricos e matriciais” e lógica formal com peso próprio.
4. **Inglês vale 12 questões, o mesmo que Português.** Interpretação de texto foi o assunto mais cobrado da prova inteira, nas duas línguas. Para quem lê documentação técnica em inglês, é a disciplina de melhor retorno por hora estudada de toda a prova.
5. **A nota de corte real vai ser alta.** Desenvolvimento de Software teve 6.257 inscritos em 2024, o perfil mais concorrido de 23.423 candidatos. Os 57,5 pontos mínimos do edital são piso de eliminação, não meta. A meta é pontuar alto.

## A.5 Onde investir o tempo de revisão

| Bloco                   | Peso             | Por quê                                                                     |
|-------------------------|------------------|-----------------------------------------------------------------------------|
| Engenharia de Software  | muito alto       | Maior bloco em 2024. Requisitos, ágil, testes, ponto de função caem sempre. |
| Banco de Dados          | alto             | Eixo duplo com Eng. Software. No MPU superou tudo.                          |
| Arquitetura de Software | alto             | Microserviços, REST×SOAP, nuvem, containers, hexagonal.                     |
| Segurança               | alto             | ISO 27001/27002:2022, OWASP, OAuth2/SSO, SAST/DAST.                         |
| Programação             | médio-alto       | Spring, XML/JSON/REST, DevOps, Git, mobile, low-code.                       |
| BI                      | médio            | DW, OLAP, ETL, data mining.                                                 |
| Governança              | médio            | ITIL 4, COBIT 2019, Scrum/Kanban, BPMN.                                     |
| Frontend                | médio            | SPA×PWA, React/Angular/Vue, HTTPS/TLS.                                      |
| Java                    | médio            | Linguagem-base do edital, mas caiu pouco na amostra.                        |
| Redes                   | (fora do edital) | Cai mesmo assim; ~7,5 pontos históricos.                                    |

## A.6 Checklist do dia da prova

- Chegar com 1h30 de antecedência. Portões fecham às 12h30, sem exceção.
- Documento de identidade original com foto. CPF e certidão de nascimento **não** são aceitos.
- Comprovante de inscrição ou de pagamento.
- Caneta esferográfica azul ou preta, corpo transparente.
- **Proibido:** relógio de qualquer tipo, celular, lápis, lapiseira, borracha, corretivo, boné, óculos escuros.
- Lanche e bebida só em embalagem transparente e sem rótulo.
- Permanência mínima obrigatória: 2 horas.
- Caderno de questões só pode ser levado se você sair nos últimos 30 minutos.
- Haverá detector de metais na entrada da sala e dos sanitários.

## Capítulo B

# Glossário de pares que a FGV inverte

Revisão de última hora: uma tabela única com os pares conceituais que a FGV troca de lugar em cada bloco. Se você reconhece os dois lados de cada linha sem pensar, a maior fatia dos distratores da prova deixa de funcionar. A coluna “bloco” remete ao capítulo com o detalhe.

| Par invertido                       | Regra para não errar                                                         | Bloco         |
|-------------------------------------|------------------------------------------------------------------------------|---------------|
| Cascata × Incremental/Ágil          | fases sequenciais + requisitos congelados = cascata                          | Eng. Software |
| Requisito funcional × não funcional | RF = o que faz; RNF = qualidade/restrrição (“em tempo real” é RNF)           | Eng. Software |
| Sprint Review × Retrospective       | Review = produto; Retro = processo                                           | Eng. Software |
| Scrum × Kanban                      | Scrum tem sprint (time-box); Kanban não tem sprint                           | Eng. Software |
| Ponto de Função × Story Points      | PF = objetivo/comparável; SP = relativo do time                              | Eng. Software |
| TDD × BDD                           | TDD = teste antes, foco em design; BDD = linguagem de negócio (Gherkin)      | Eng. Software |
| CI × CD                             | CI integra/testa; CD leva a produção                                         | Eng. Software |
| Design alto × baixo nível           | alto = estrutura/módulos; baixo = função/algoritmo                           | Eng. Software |
| Factory Method × Abstract Factory   | Method = um produto via herança; Abstract = famílias via composição          | Padrões       |
| Adapter × Facade × Decorator        | Adapter compatibiliza; Facade simplifica; Decorator adiciona função          | Padrões       |
| Strategy × State                    | Strategy troca algoritmo; State troca conforme estado interno                | Padrões       |
| Agregação × Composição              | losango vazio (independente) × cheio (morre com o todo)                      | UML           |
| Include × Extend                    | include = sempre; extend = opcional                                          | UML           |
| Estrutural × Comportamental (UML)   | Classe = estrutural; Sequência/Atividade/Estado/Caso de Uso = comportamental | UML           |
| Servidor web × de aplicação         | web = HTTP; aplicação = lógica de negócio                                    | Arquitetura   |
| REST × SOAP                         | REST stateless/JSON; SOAP contrato WSDL/XML                                  | Arquitetura   |

| Par invertido                                     | Regra para não errar                                                     | Bloco                |
|---------------------------------------------------|--------------------------------------------------------------------------|----------------------|
| PUT × PATCH                                       | PUT substitui inteiro; PATCH atualiza parcial                            | Arquitetura          |
| Fila × Tópico (pub/sub)                           | fila = um consumidor; tópico = vários                                    | Arquitetura          |
| Escalar vertical × horizontal                     | vertical = mais recurso na instância; horizontal = mais instâncias       | Arquitetura          |
| Container × VM                                    | container compartilha kernel; VM tem SO completo                         | Arquitetura          |
| Hipervisor Tipo 1 × Tipo 2                        | Tipo 1 bare-metal; Tipo 2 sobre SO hospedeiro                            | Arquitetura / Órfãos |
| Intranet × Extranet × Internet                    | intranet = interna; extranet = + parceiros externos; internet = pública  | Arquitetura / Redes  |
| OLTP × OLAP                                       | OLTP = transacional/linha; OLAP = analítico/cubo                         | Banco de Dados / BI  |
| TRUNCATE × DELETE                                 | TRUNCATE é DDL, auto-commit; DELETE é DML, WHERE + roll-back             | Banco de Dados       |
| ETL × ELT                                         | ETL transforma antes de carregar; ELT transforma no destino              | Banco de Dados / BI  |
| Star Schema × Snowflake                           | Star = desnormalizado/rápido; Snowflake = normalizado/mais joins         | BI                   |
| Semiaditivo × Aditivo                             | semiaditivo não soma no tempo (saldo/estoque)                            | BI                   |
| Suporte × Confiança                               | suporte = frequência do conjunto; confiança = força de $A \rightarrow B$ | BI                   |
| Drill-down × Roll-up                              | down = mais detalhe; up = mais agregação                                 | BI                   |
| Slice × Dice                                      | slice = uma dimensão; dice = várias                                      | BI                   |
| Confidencialidade × Integridade × Disponibilidade | ver = confid.; íntegro = integridade; acessível = disponibilidade        | Segurança            |
| Simétrica × Assimétrica                           | simétrica = uma chave, rápida; assimétrica = par de chaves, lenta        | Segurança            |
| Hash × Cifra reversível                           | hash é unidirecional, não descriptografa                                 | Segurança            |
| SSL × TLS                                         | TLS sucedeu e corrigiu o SSL (nunca o contrário)                         | Segurança            |
| DAC × MAC × RBAC                                  | DAC = dono decide; MAC = rótulos do sistema; RBAC = por papel            | Segurança            |
| Autenticação × Autorização (OAuth2)               | OAuth2 = autorização; OIDC = autenticação                                | Segurança            |
| SAST × DAST                                       | SAST = código parado; DAST = app rodando                                 | Segurança            |
| IDS × IPS                                         | IDS alerta (passivo); IPS bloqueia (ativo)                               | Segurança            |

| Par invertido                                       | Regra para não errar                                                                  | Bloco                  |
|-----------------------------------------------------|---------------------------------------------------------------------------------------|------------------------|
| Spring Boot × Spring Cloud                          | Boot = app individual; Cloud = distribuído/microserviços                              | Programação            |
| Hibernate × JUnit                                   | Hibernate = ORM; JUnit = teste de unidade                                             | Programação            |
| XSLT sobre XML × JSON                               | XSLT só transforma XML, nunca JSON                                                    | Programação            |
| Flutter (Dart) × outros frameworks                  | Flutter = Dart; React Native = JS; Xamarin = C#                                       | Programação / Frontend |
| Kotlin/Android × Swift/iOS                          | não trocar a linguagem nativa de cada SO                                              | Programação / Frontend |
| Checked × Unchecked (Java)                          | checked = Exception, exige throws/catch; unchecked = RuntimeException                 | Java                   |
| Overload × Override                                 | overload = mesma classe, assinatura diferente; override = subclasse, mesma assinatura | Java                   |
| ISP × SRP (SOLID)                                   | ISP = interface enxuta; SRP = uma responsabilidade                                    | Java                   |
| Thread virtual × de plataforma                      | muitas virtuais compartilham uma carrier de plataforma                                | Java                   |
| SPA × PWA                                           | Service Worker/offline/instalável são da PWA, não da SPA                              | Frontend               |
| Padding × Margin                                    | padding = interno; margin = externo                                                   | Frontend               |
| React × Angular                                     | React = biblioteca (JS/JSX); Angular = framework completo (TypeScript)                | Frontend               |
| Governança × Gestão (COBIT)                         | EDM = governança; APO/BAI/DSS/MEA = gestão                                            | Governança             |
| Incidente × Problema (ITIL)                         | incidente = restaurar serviço; problema = causa-raiz                                  | Governança             |
| Grupo de processos × Área de conhecimento (PMBOK 6) | 5 grupos × 10 áreas — não confundir com os 12 princípios do PMBOK 7                   | Governança             |
| Modelo OSI por camada                               | sessão = diálogo/checkpoints; transporte = portas/TCP-UDP                             | Redes                  |
| TCP × UDP                                           | TCP = conexão/confiável; UDP = sem conexão/rápido                                     | Redes                  |
| Distância administrativa (Cisco)                    | menor vence: conectada 0, estática 1, OSPF 110, RIP 120                               | Redes                  |
| MPLS × roteamento IP                                | MPLS = rótulo (“camada 2.5”), não olha IP a cada salto                                | Redes                  |
| AD × DBA                                            | AD = negócio/conceitual; DBA = técnico/físico                                         | Órfãos                 |

| Par invertido                       | Regra para não errar                                                                  | Bloco                   |
|-------------------------------------|---------------------------------------------------------------------------------------|-------------------------|
| Incremental × Diferencial (backup)  | incremental = desde o último qual-quer; diferencial = desde o último full             | Órfãos                  |
| Analysis → Redo → Undo (ARIES)      | essa é a ordem; a banca inverte Redo/Undo                                             | Órfãos                  |
| Bitmap × B+ Tree (índices)          | bitmap = baixa cardinalidade; B+ Tree = alta cardinalidade/faixas                     | Banco de Dados / Órfãos |
| Supervisionado × Não supervisionado | supervisionado = dados rotulados; não supervisionado = sem rótulo                     | Atualidades / Órfãos    |
| Data drift × Concept drift          | data drift = muda a entrada; concept drift = muda a relação entrada→saída             | Órfãos                  |
| L1 (Lasso) × L2 (Ridge)             | L1 zera coeficientes; L2 encolhe sem zerar                                            | Órfãos                  |
| Overfitting × Underfitting          | overfitting decora o treino; regularização combate overfitting                        | Atualidades / Órfãos    |
| Contrapositiva × Recíproca          | só a contrapositiva ( $\neg q \rightarrow \neg p$ ) é equivalente a $p \rightarrow q$ | RLM                     |
| Negação de “todo”/“algum”           | nega “todo A é B” com “algum A não é B”                                               | RLM                     |
| Controlador × Operador (LGPD)       | controlador decide; operador executa em nome dele                                     | Legislação              |
| ANPD × CNPD                         | ANPD fiscaliza/sanciona; CNPD só é consultivo                                         | Legislação              |

### Como usar esta tabela na reta final

Não releia a apostila inteira na última semana — releia esta tabela. Para cada linha que você hesitar, volte ao capítulo indicado na terceira coluna e releia só a caixa vermelha (“Como a FGV arma a pegadinha”) daquele assunto. Depois volte ao quiz: `./quiz.py -erradas` filtra exatamente as questões em que você ainda tropeça nesses pares.